Federated Learning Optimization on UCI HAR

A PhD-Level, Beginner-Friendly Technical Report from Data Loading to Evaluation

Sriram M.

April 26, 2026

Abstract

This report documents the full federated learning optimization project in a detailed way. It explains the project for a reader who does not yet know machine learning, while preserving enough mathematical and experimental detail for graduate-level review. The report follows the entire pipeline: loading the UCI HAR dataset, transforming subjects into federated clients, scaling features, training a neural network with FedAvg, using early stopping, measuring communication, testing CVaR-style fairness, solving an LP communication model, running differential-evolution hyperparameter search, validating with 10 random seeds, and evaluating results with classification, fairness, calibration, statistical, and optimization diagnostics.

Contents

1 Executive Summary 4

2 Research Problem and Motivation 5

3 Machine Learning From First Principles 6

4 Dataset Loading 7

5 Subject-to-Client Conversion 8

6 Preprocessing and Scaling 9

7 Exploratory Data Analysis 10

8 Model Architecture 11

9 FedAvg Training 12

10 Early Stopping 13

11 Communication Accounting 14

12 Fairness and CVaR 15

13 Linear Programming and Duality 16

14 Differential Evolution Search 17

15 Baselines 18

16 Evaluation Metrics 19

17 Statistical Validation 20

18 Results and Interpretation 21

19 Professor Feedback and Extensions 22

20 Limitations and Future Work 23

21 Figure-by-Figure Explanation **24**
21.1 Class distribution . . . 24
21.2 Train/test class distribution . . . 24
21.3 Client sample counts . . . 25
21.4 Client label heatmap . . . 25
21.5 Client entropy . . . 26
21.6 Feature correlation subset . . . 27
21.7 PCA by activity . . . 27
21.8 3D PCA by activity . . . 28
21.9 PCA by client . . . 29
21.10 3D PCA by client . . . 30
21.11 Loss versus rounds . . . 31
21.12 Accuracy versus rounds . . . 32
21.13 Worst-client accuracy versus rounds . . . 32
21.14 Best loss so far . . . 33
21.15 Rounds since improvement . . . 33
21.16 Communication per round . . . 34
21.17 Accuracy versus communication . . . 35
21.18 Raw confusion matrix . . . 35
21.19 Normalized confusion matrix . . . 36
21.20 Per-class error rate . . . 37
21.21 Top confused pairs . . . 38
21.22 Per-client accuracy . . . 38
21.23 Baseline accuracy . . . 39
21.24 Baseline worst-client accuracy . . . 39
21.25 Baseline communication cost . . . 40
21.26 Rounds to convergence . . . 40
21.27 Shadow price versus budget . . . 41
21.28 Loss versus budget . . . 41
21.29 KKT residuals . . . 42
21.30 3D LP tradeoff . . . 43
21.31 Search accuracy versus communication . . . 44
21.32 3D hyperparameter fitness . . . 45
21.33 GA fitness over evaluations . . . 46
21.34 Accuracy per MB . . . 47
21.35 Accuracy-fairness-communication tradeoff . . . 47

22 Mathematical Formulation **49**

23 Core Tables and Actual Outputs **50**

24 Optimization, Search, and Statistical Tables **56**

25 Round-Level Trace Example **68**

A Source Code Appendix **73**
A.1 flopt/config.py . . . 73
A.2 flopt/data.py . . . 73
A.3 flopt/models.py . . . 75
A.4 flopt/fedavg.py . . . 75
A.5 flopt/duality.py . . . 79
A.6 flopt/search.py . . . 80
A.7 flopt/metrics.py . . . 81
A.8 flopt/eda.py . . . 82
A.9 flopt/calibration.py . . . 83
A.10 flopt/baselines.py . . . 84
A.11 flopt/stats.py . . . 85
A.12 flopt/analysis.py . . . 87
A.13 flopt/plots.py . . . 88
A.14 experiments/run_full_v2.py . . . 91

B Final Conclusion **99**

List of Figures

21.1 Class distribution. Shows whether class imbalance can bias accuracy. 24
21.2 Train/test class distribution. Checks that all activities are represented in both splits. 25
21.3 Client sample counts. Shows which clients have more influence under sample-weighted FedAvg. 25
21.4 Client label heatmap. Displays non-IID label structure across subjects. 26
21.5 Client entropy. Quantifies how diverse each client's labels are. 26
21.6 Feature correlation subset. Shows dependence among engineered sensor features. . . 27
21.7 PCA by activity. Visualizes activity separability in two dimensions. 28
21.8 3D PCA by activity. Provides a richer low-dimensional view of class structure. . . . 29
21.9 PCA by client. Visualizes subject-level heterogeneity. 30
21.10 3D PCA by client. Shows client variation in three dimensions. 31
21.11 Loss versus rounds. Shows convergence behavior. 32
21.12 Accuracy versus rounds. Shows performance improvement over FL rounds. 32
21.13 Worst-client accuracy versus rounds. Shows fairness behavior over training. 33
21.14 Best loss so far. Shows the early-stopping monitor. 33
21.15 Rounds since improvement. Explains patience accumulation. 34
21.16 Communication per round. Connects FL training to byte cost. 34
21.17 Accuracy versus communication. Shows the performance-cost tradeoff. 35
21.18 Raw confusion matrix. Shows absolute classification mistakes. 36
21.19 Normalized confusion matrix. Shows per-class behavior independent of support. . . . 37
21.20 Per-class error rate. Ranks difficult activities. 37
21.21 Top confused pairs. Identifies common activity confusions. 38
21.22 Per-client accuracy. Shows client-level fairness. 38
21.23 Baseline accuracy. Compares method performance. 39
21.24 Baseline worst-client accuracy. Compares fairness across methods. 39
21.25 Baseline communication cost. Compares byte budgets. 40
21.26 Rounds to convergence. Compares early-stopped training length. 40
21.27 Shadow price versus budget. Shows communication scarcity in the LP model. 41
21.28 Loss versus budget. Shows objective improvement with budget. 42
21.29 KKT residuals. Checks LP numerical optimality. 43
21.30 3D LP tradeoff. Shows budget, loss, and dual value together. 44
21.31 Search accuracy versus communication. Shows hyperparameter tradeoffs. 45
21.32 3D hyperparameter fitness. Shows the search landscape. 46
21.33 GA fitness over evaluations. Shows differential-evolution progress. 46
21.34 Accuracy per MB. Shows communication efficiency. 47
21.35 Accuracy-fairness-communication tradeoff. Shows the central multi-objective tension. 48

List of Tables

1.1 Headline method comparison. 4

23.1 Full method summary. 50
23.2 Class distribution. 50
23.3 Client counts and entropy. 51
23.4 Non-IID client metrics. 51
23.5 Classification report. 52
23.6 Per-client metrics. 52
23.7 Per-class error rates. 53
23.8 Top confusion pairs. 53
23.9 Calibration bins. 54
23.10 Calibration summary. 54
23.11 Runtime by stage. 54
23.12 Theoretical alignment. 55

24.1 Grid-search candidates. 56
24.2 Differential-evolution objective evaluations. 56
24.3 GA best-so-far history. 58
24.4 LP shadow-price and KKT diagnostics. 60
24.5 Confidence intervals. 61
24.6 Paired tests against FedAvg. 62
24.7 Ablation-style deltas. 63
24.8 Communication efficiency. 65
24.9 Failure-mode records. 67

25.1 Round-level FedAvg trace for seed 7. 68

Chapter 1

Executive Summary

The project asks whether a useful activity-recognition model can be trained without centralizing all client data. In the simulated federated setting, each UCI HAR subject is treated as a client. The default FedAvg method achieved mean accuracy 0.9697 and worst-client accuracy 0.8912 across 10 seeds.

The centralized model reached 0.9847 mean accuracy, which gives an upper benchmark because it can use all training data directly. Grid-validated FedAvg reached 0.9726, and GA-validated FedAvg reached 0.9654 while using substantially less communication. The results show that FedAvg is strong, tuning helps, and GA is valuable mainly as a communication-efficiency tool.

The key scientific caution is fairness. Average accuracy is high, but worst-client accuracy is lower. CVaR-style weighting was investigated because it focuses on hard clients, but it did not reliably improve worst-client accuracy in this run. This is an important honest result: fairness remains the main unresolved issue.

Table 1.1: Headline method comparison.

method	n	final_loss_mean	final_accuracy_mean	final_worst_client_accuracy_mean	total_
fedavg_default	10.0000	0.084107	0.969708	0.891248	1.460e
cvar_0	10.0000	0.084477	0.969549	0.883922	1.440e
cvar_0.5	10.0000	0.086034	0.970123	0.874004	1.459e
cvar_0.75	10.0000	0.094282	0.967479	0.873944	1.490e
cvar_0.9	10.0000	0.082170	0.970669	0.876384	1.557e
cvar_0.95	10.0000	0.081762	0.970815	0.878853	1.557e
centralized	10.0000	0.062112	0.984703	0.940667	0.000
local_only	1.0000	0.041006	0.987801	0.944444	0.000
grid_best_validated	10.0000	0.089625	0.972580	0.882650	1.562e
ga_best_validated	10.0000	0.117797	0.965375	0.837289	3.473e

Chapter 2

Research Problem and Motivation

The reason this report spends time on why federated learning is relevant and why this is an optimization problem is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why federated learning is relevant and why this is an optimization problem, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why federated learning is relevant and why this is an optimization problem is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why federated learning is relevant and why this is an optimization problem answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 3

Machine Learning From First Principles

The reason this report spends time on what features, labels, loss, accuracy, and model parameters mean is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For what features, labels, loss, accuracy, and model parameters mean, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, what features, labels, loss, accuracy, and model parameters mean is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that what features, labels, loss, accuracy, and model parameters mean answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 4

Dataset Loading

The reason this report spends time on why UCI HAR is loaded with features, labels, activity names, and subject identifiers is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why UCI HAR is loaded with features, labels, activity names, and subject identifiers, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why UCI HAR is loaded with features, labels, activity names, and subject identifiers is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why UCI HAR is loaded with features, labels, activity names, and subject identifiers answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 5

Subject-to-Client Conversion

The reason this report spends time on why each subject becomes a federated client and why this creates non-IID data is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why each subject becomes a federated client and why this creates non-IID data, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why each subject becomes a federated client and why this creates non-IID data is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why each subject becomes a federated client and why this creates non-IID data answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 6

Preprocessing and Scaling

The reason this report spends time on why StandardScaler is fit on training data and reused at inference is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why StandardScaler is fit on training data and reused at inference, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why StandardScaler is fit on training data and reused at inference is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why StandardScaler is fit on training data and reused at inference answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 7

Exploratory Data Analysis

The reason this report spends time on why class balance, client counts, heatmaps, entropy, and PCA are inspected is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why class balance, client counts, heatmaps, entropy, and PCA are inspected, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why class balance, client counts, heatmaps, entropy, and PCA are inspected is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why class balance, client counts, heatmaps, entropy, and PCA are inspected answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 8

Model Architecture

The reason this report spends time on why the HARMLP model is small, nonlinear, and suitable for repeated FL runs is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why the HARMLP model is small, nonlinear, and suitable for repeated FL runs, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why the HARMLP model is small, nonlinear, and suitable for repeated FL runs is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why the HARMLP model is small, nonlinear, and suitable for repeated FL runs answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 9

FedAvg Training

The reason this report spends time on why local training and weighted server aggregation define the core FL algorithm is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why local training and weighted server aggregation define the core FL algorithm, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why local training and weighted server aggregation define the core FL algorithm is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why local training and weighted server aggregation define the core FL algorithm answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 10

Early Stopping

The reason this report spends time on why max rounds alone is not enough and why convergence should determine stopping is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why max rounds alone is not enough and why convergence should determine stopping, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why max rounds alone is not enough and why convergence should determine stopping is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why max rounds alone is not enough and why convergence should determine stopping answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 11

Communication Accounting

The reason this report spends time on why communication must be measured and what assumptions are in the byte model is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why communication must be measured and what assumptions are in the byte model, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why communication must be measured and what assumptions are in the byte model is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why communication must be measured and what assumptions are in the byte model answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 12

Fairness and CVaR

The reason this report spends time on why worst-client accuracy matters and why tail-risk weighting was tested is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why worst-client accuracy matters and why tail-risk weighting was tested, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why worst-client accuracy matters and why tail-risk weighting was tested is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why worst-client accuracy matters and why tail-risk weighting was tested answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 13

Linear Programming and Duality

The reason this report spends time on why communication budgets can be analyzed with shadow prices is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why communication budgets can be analyzed with shadow prices, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why communication budgets can be analyzed with shadow prices is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why communication budgets can be analyzed with shadow prices answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 14

Differential Evolution Search

The reason this report spends time on why hyperparameter search is expensive and how it was validated is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why hyperparameter search is expensive and how it was validated, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why hyperparameter search is expensive and how it was validated is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why hyperparameter search is expensive and how it was validated answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 15

Baselines

The reason this report spends time on why centralized, local-only, default FedAvg, grid, and GA baselines are all needed is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why centralized, local-only, default FedAvg, grid, and GA baselines are all needed, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why centralized, local-only, default FedAvg, grid, and GA baselines are all needed is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why centralized, local-only, default FedAvg, grid, and GA baselines are all needed answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 16

Evaluation Metrics

The reason this report spends time on why loss, accuracy, macro F1, confusion matrices, calibration, and per-client metrics answer different questions is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why loss, accuracy, macro F1, confusion matrices, calibration, and per-client metrics answer different questions, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why loss, accuracy, macro F1, confusion matrices, calibration, and per-client metrics answer different questions is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why loss, accuracy, macro F1, confusion matrices, calibration, and per-client metrics answer different questions answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 17

Statistical Validation

The reason this report spends time on why multiple seeds, confidence intervals, paired tests, and effect sizes are reported is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For why multiple seeds, confidence intervals, paired tests, and effect sizes are reported, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, why multiple seeds, confidence intervals, paired tests, and effect sizes are reported is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that why multiple seeds, confidence intervals, paired tests, and effect sizes are reported answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 18

Results and Interpretation

The reason this report spends time on what the final numbers imply and what they do not imply is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For what the final numbers imply and what they do not imply, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, what the final numbers imply and what they do not imply is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that what the final numbers imply and what they do not imply answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 19

Professor Feedback and Extensions

The reason this report spends time on how to improve the communication model, search efficiency, and non-convex DNN extension is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For how to improve the communication model, search efficiency, and non-convex DNN extension, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, how to improve the communication model, search efficiency, and non-convex DNN extension is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that how to improve the communication model, search efficiency, and non-convex DNN extension answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 20

Limitations and Future Work

The reason this report spends time on what is still missing before this becomes a real deployed FL system is that a federated learning result is not a single accuracy number. It is a chain of choices. If one choice is misunderstood, the final conclusion can be misleading.

For what is still missing before this becomes a real deployed FL system, the project asks four questions: what is being done, why it is needed, how it affects optimization, and what evidence the v2 run provides. This structure makes the report readable for a beginner while still preserving the technical argument expected in a graduate optimization project.

The implementation is therefore treated as part of the scientific method. Code is not only a tool used to produce numbers; it encodes assumptions about clients, data, communication, fairness, convergence, and evaluation. Those assumptions are made explicit throughout the report.

In this project, what is still missing before this becomes a real deployed FL system is connected to the actual v2 outputs rather than being a theoretical side note. The output directory contains raw round logs, summary CSVs, model checkpoints, plots, and diagnostics, so each claim can be traced back to a produced artifact.

The beginner-level interpretation is that what is still missing before this becomes a real deployed FL system answers a practical question: what decision did we make, why did we make it, and how did it change the final model? The PhD-level interpretation is that each decision changes the empirical objective, the feasible communication budget, the stochastic training process, or the reliability of the evaluation.

A common mistake in machine-learning reports is to jump directly from dataset to final accuracy. This report avoids that mistake. It explains the causal chain: dataset structure creates clients; clients create heterogeneity; heterogeneity affects local gradients; local gradients affect FedAvg aggregation; aggregation affects convergence; convergence affects communication; communication interacts with hyperparameter search; and all of these determine the final evaluation.

Chapter 21

Figure-by-Figure Explanation

This chapter explains every major generated visualization. The purpose is not to decorate the report. Each figure answers a specific diagnostic question. Together, the figures show data quality, client heterogeneity, training convergence, fairness behavior, communication efficiency, classification quality, search behavior, and optimization diagnostics.

21.1 Class distribution

Shows whether class imbalance can bias accuracy. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

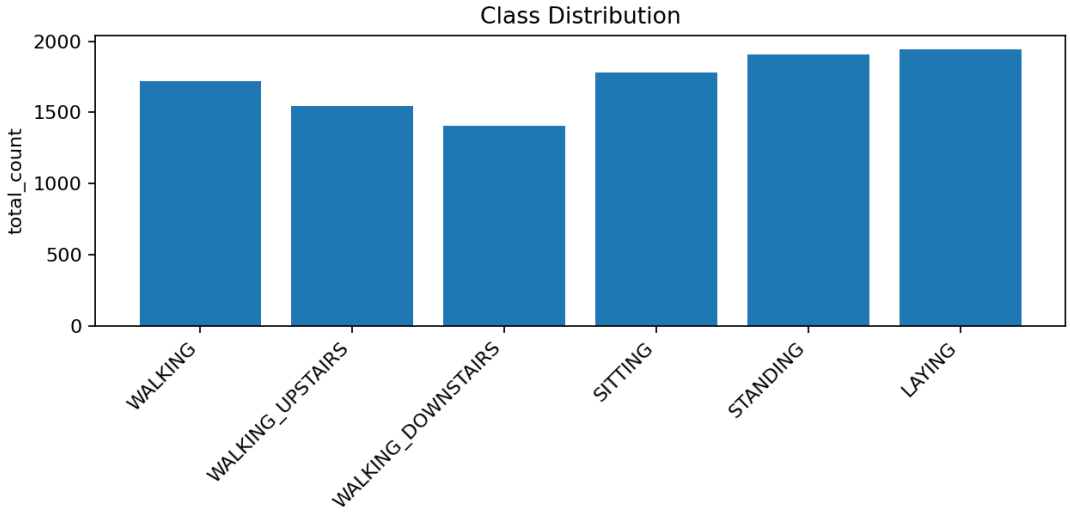

Figure 21.1: Class distribution. Shows whether class imbalance can bias accuracy.

21.2 Train/test class distribution

Checks that all activities are represented in both splits. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

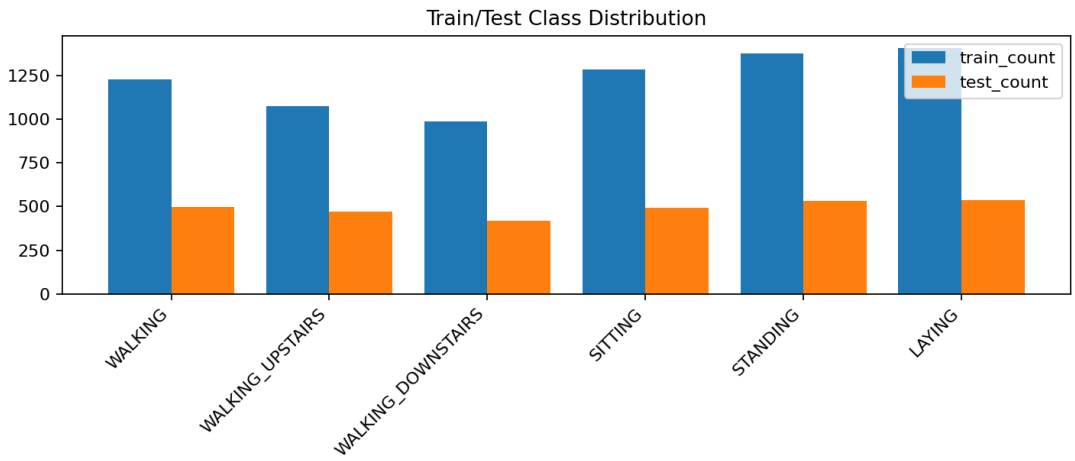

Figure 21.2: Train/test class distribution. Checks that all activities are represented in both splits.

21.3 Client sample counts

Shows which clients have more influence under sample-weighted FedAvg. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

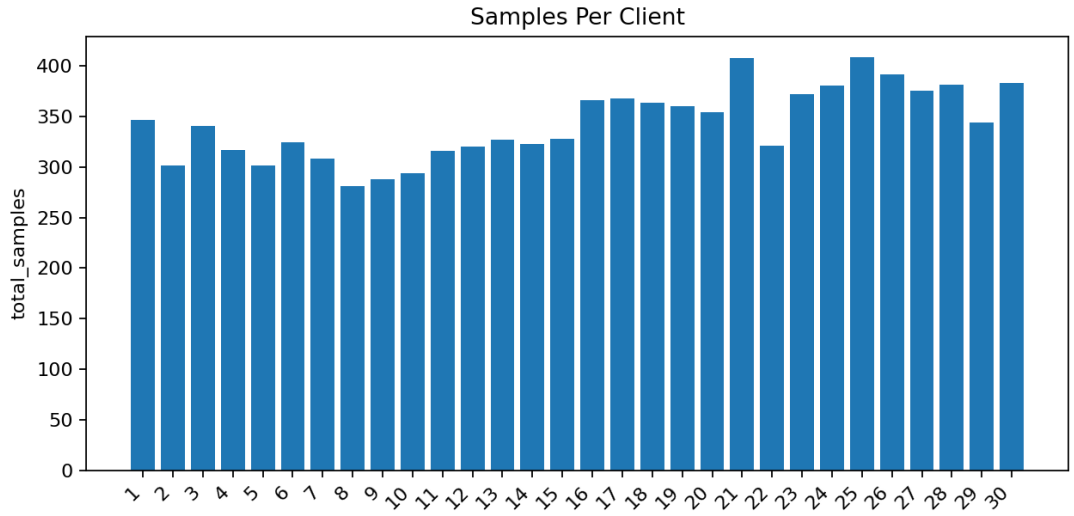

Figure 21.3: Client sample counts. Shows which clients have more influence under sample-weighted FedAvg.

21.4 Client label heatmap

Displays non-IID label structure across subjects. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

Figure 21.4: Client label heatmap. Displays non-IID label structure across subjects.

21.5 Client entropy

Quantifies how diverse each client's labels are. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

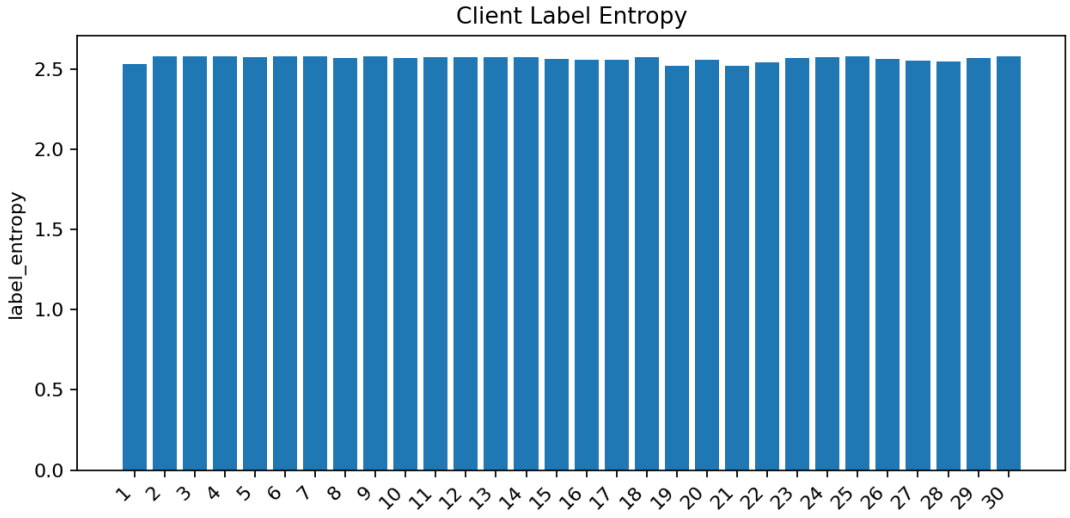

Figure 21.5: Client entropy. Quantifies how diverse each client's labels are.

21.6 Feature correlation subset

Shows dependence among engineered sensor features. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

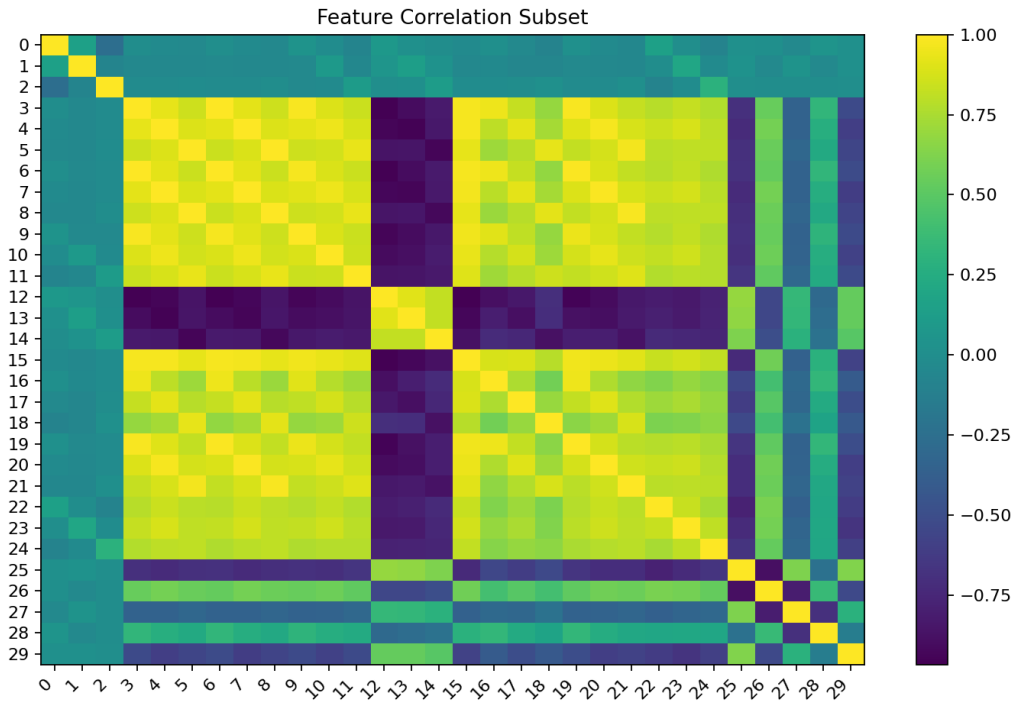

Figure 21.6: Feature correlation subset. Shows dependence among engineered sensor features.

21.7 PCA by activity

Visualizes activity separability in two dimensions. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

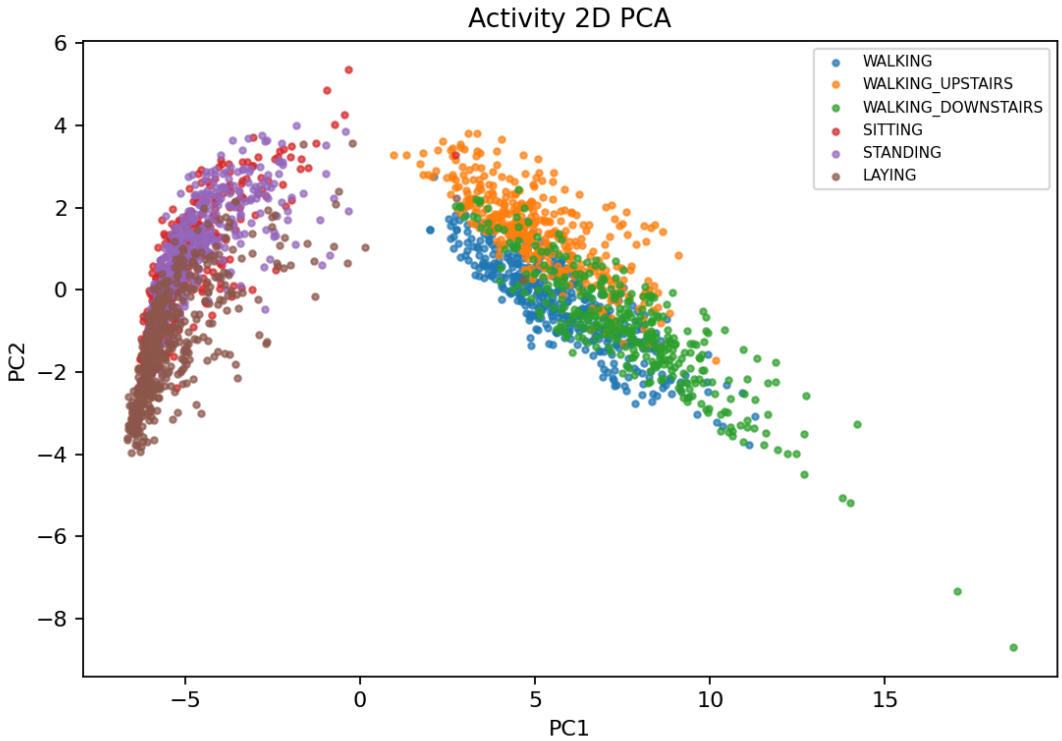

Figure 21.7: PCA by activity. Visualizes activity separability in two dimensions.

21.8 3D PCA by activity

Provides a richer low-dimensional view of class structure. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

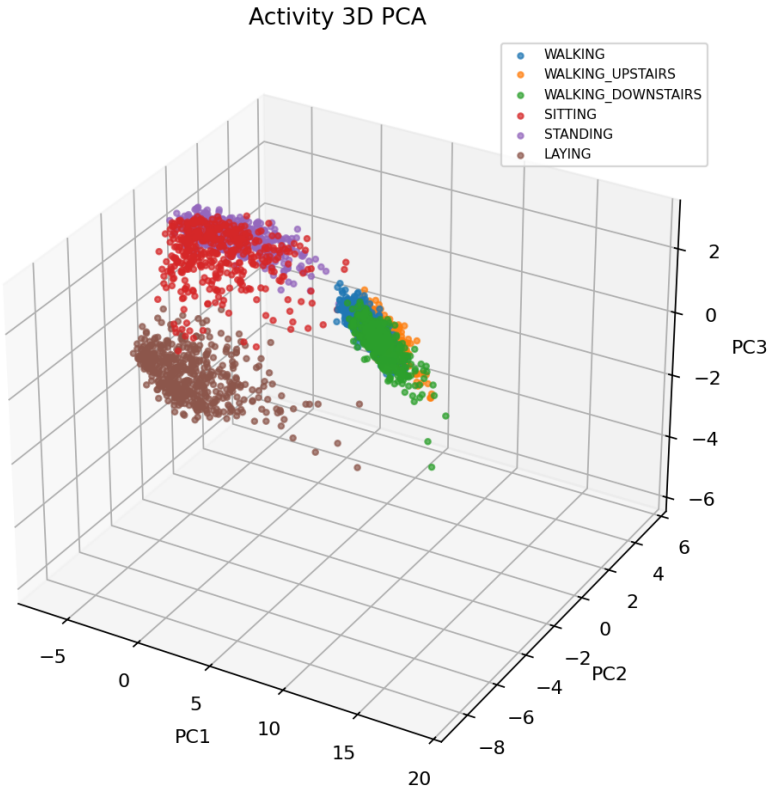

Figure 21.8: 3D PCA by activity. Provides a richer low-dimensional view of class structure.

21.9 PCA by client

Visualizes subject-level heterogeneity. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

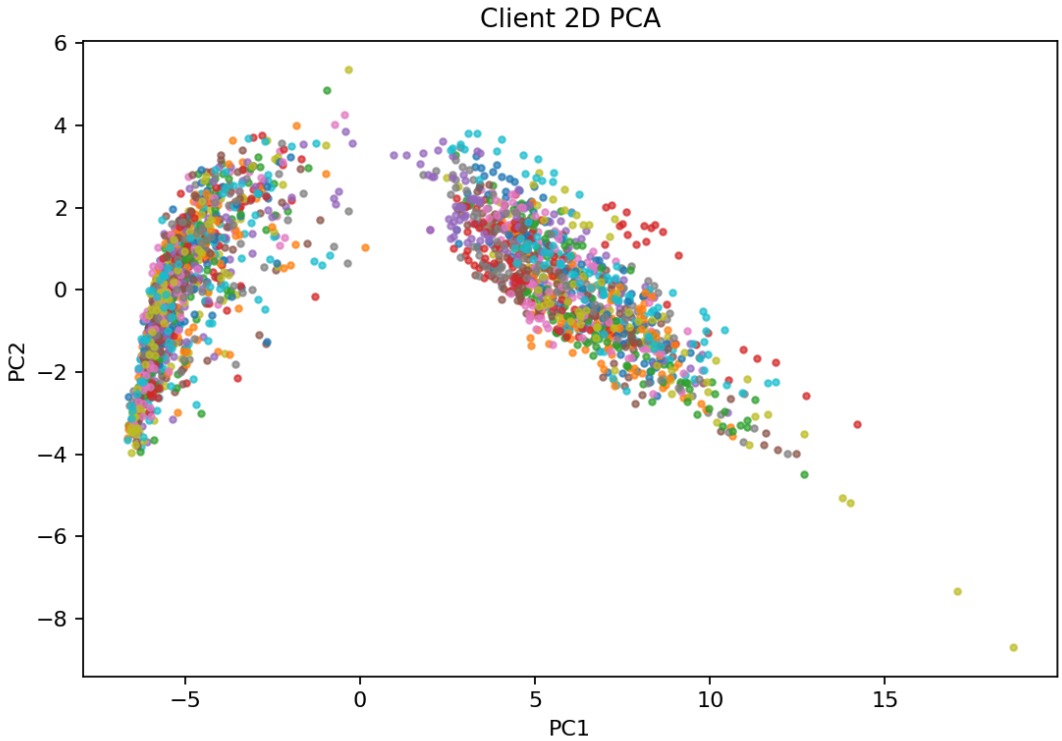

Figure 21.9: PCA by client. Visualizes subject-level heterogeneity.

21.10 3D PCA by client

Shows client variation in three dimensions. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

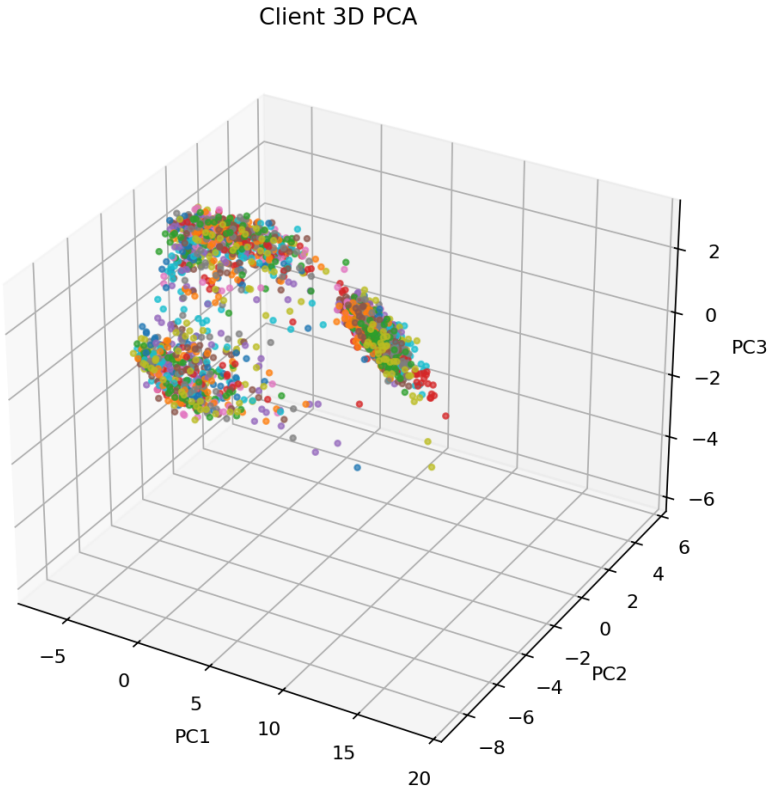

Figure 21.10: 3D PCA by client. Shows client variation in three dimensions.

21.11 Loss versus rounds

Shows convergence behavior. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

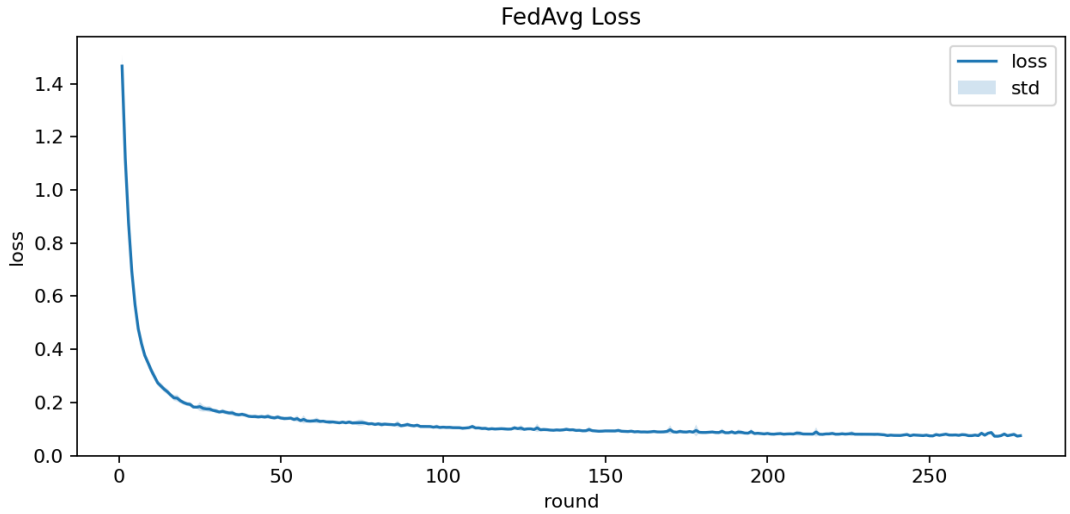

Figure 21.11: Loss versus rounds. Shows convergence behavior.

21.12 Accuracy versus rounds

Shows performance improvement over FL rounds. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

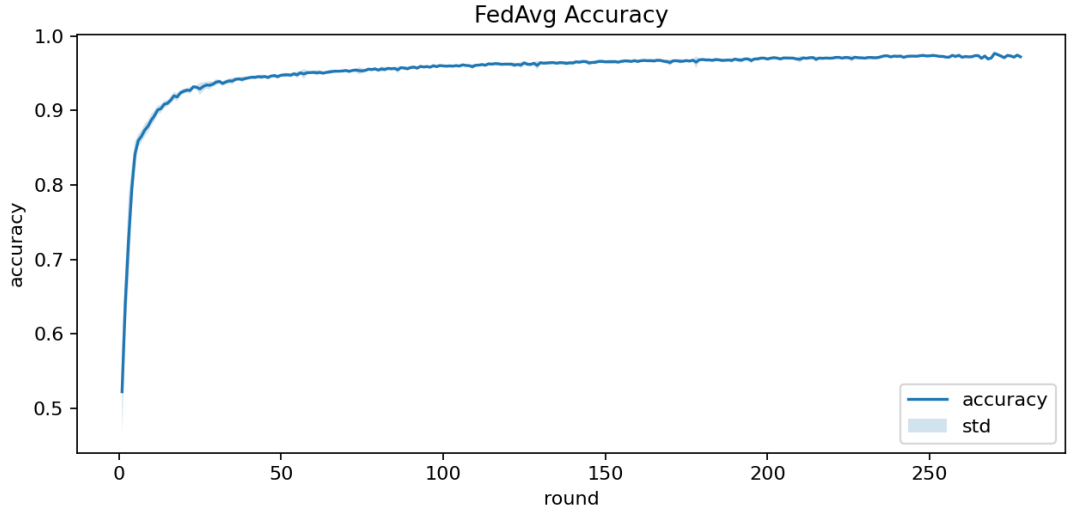

Figure 21.12: Accuracy versus rounds. Shows performance improvement over FL rounds.

21.13 Worst-client accuracy versus rounds

Shows fairness behavior over training. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

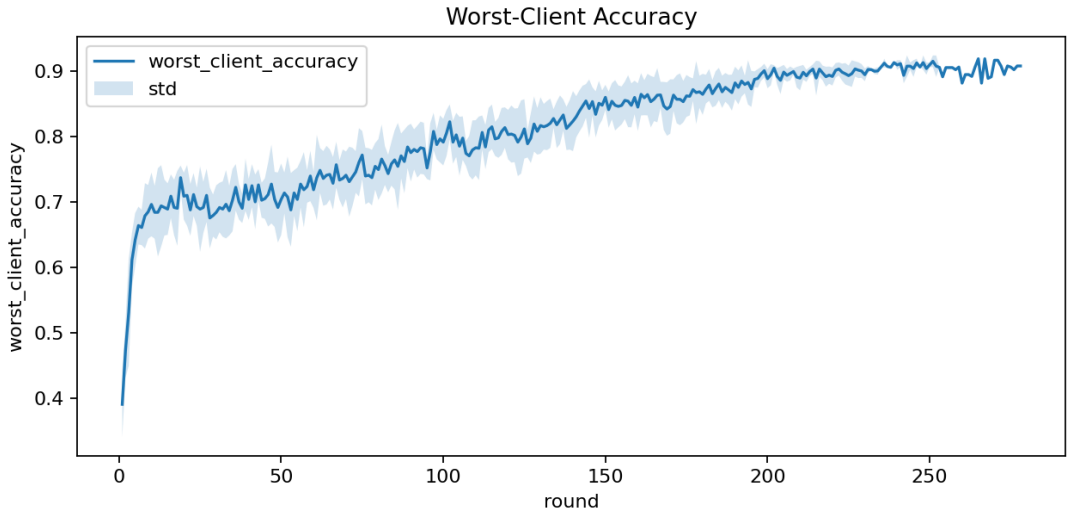

Figure 21.13: Worst-client accuracy versus rounds. Shows fairness behavior over training.

21.14 Best loss so far

Shows the early-stopping monitor. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

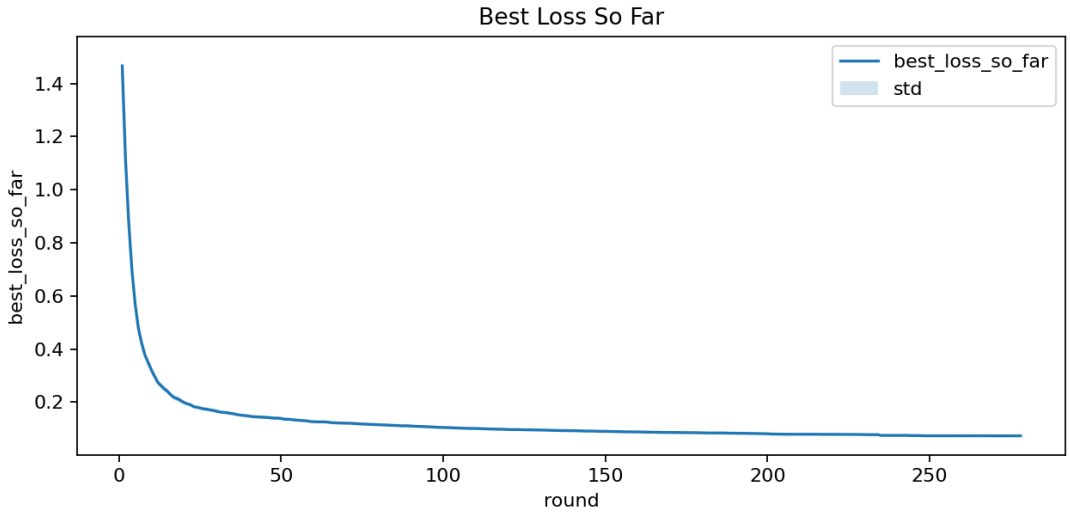

Figure 21.14: Best loss so far. Shows the early-stopping monitor.

21.15 Rounds since improvement

Explains patience accumulation. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

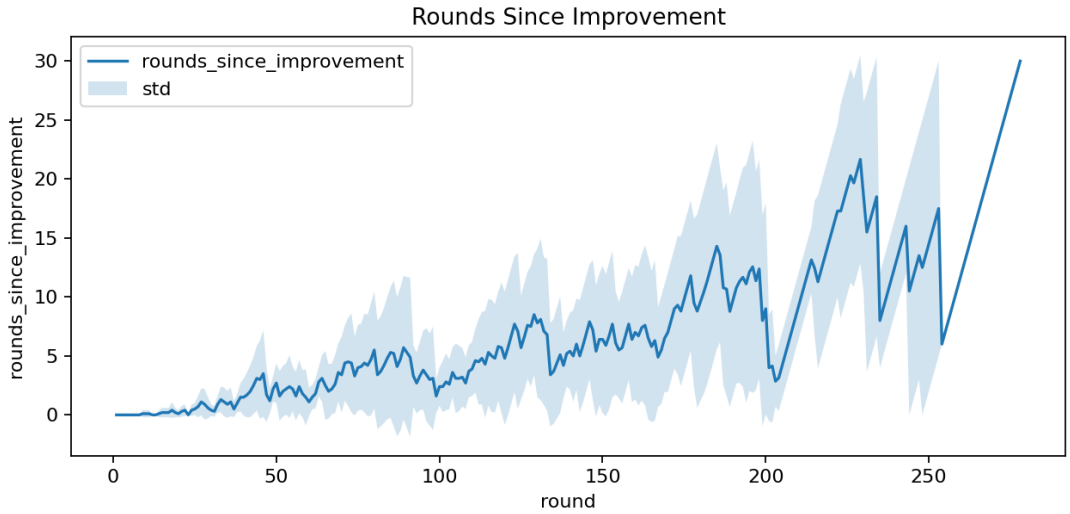

Figure 21.15: Rounds since improvement. Explains patience accumulation.

21.16 Communication per round

Connects FL training to byte cost. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

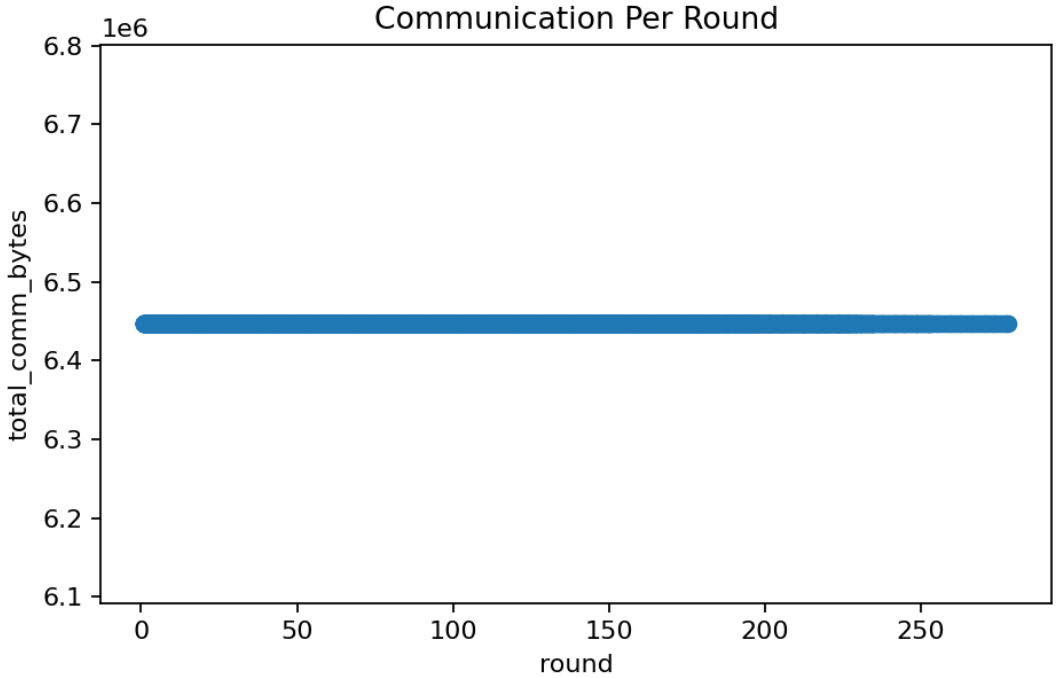

Figure 21.16: Communication per round. Connects FL training to byte cost.

21.17 Accuracy versus communication

Shows the performance-cost tradeoff. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

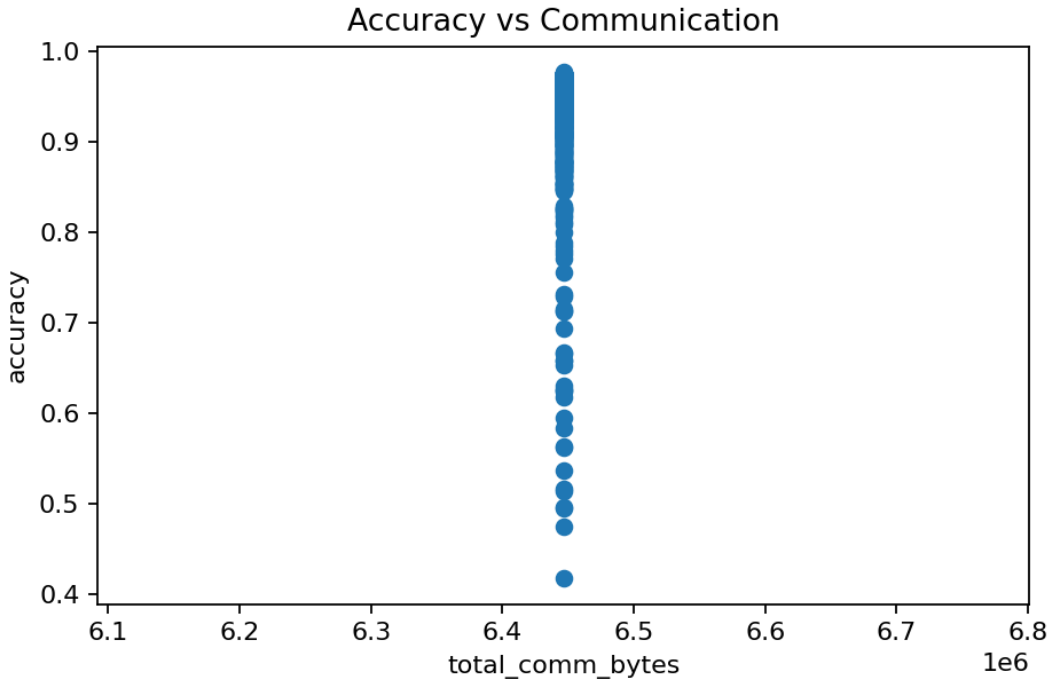

Figure 21.17: Accuracy versus communication. Shows the performance-cost tradeoff.

21.18 Raw confusion matrix

Shows absolute classification mistakes. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

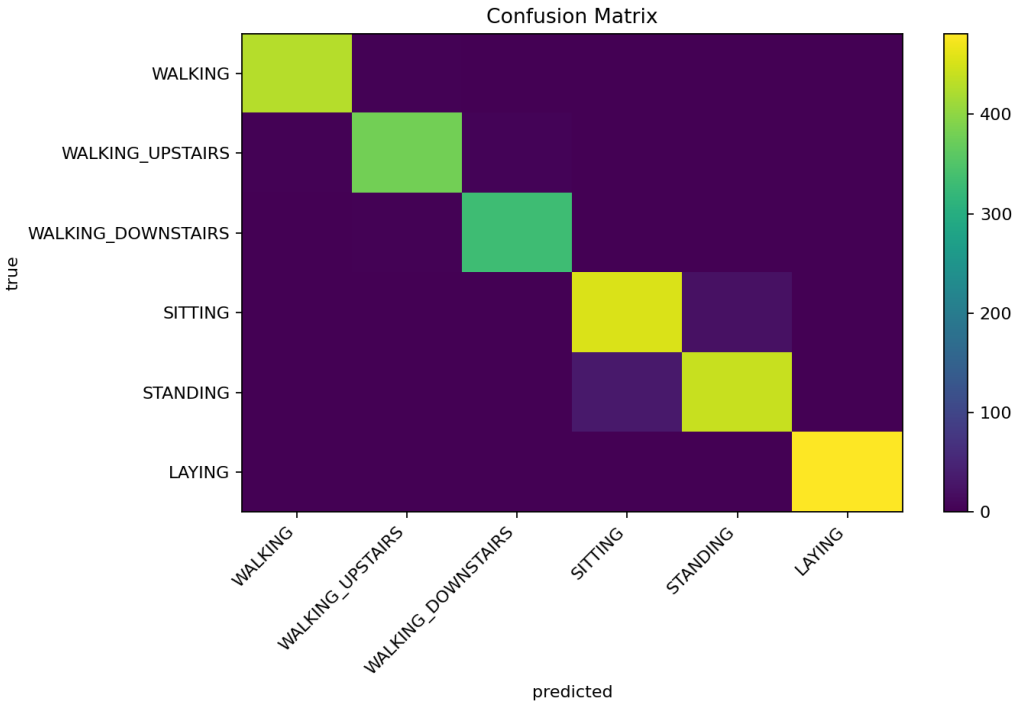

Figure 21.18: Raw confusion matrix. Shows absolute classification mistakes.

21.19 Normalized confusion matrix

Shows per-class behavior independent of support. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

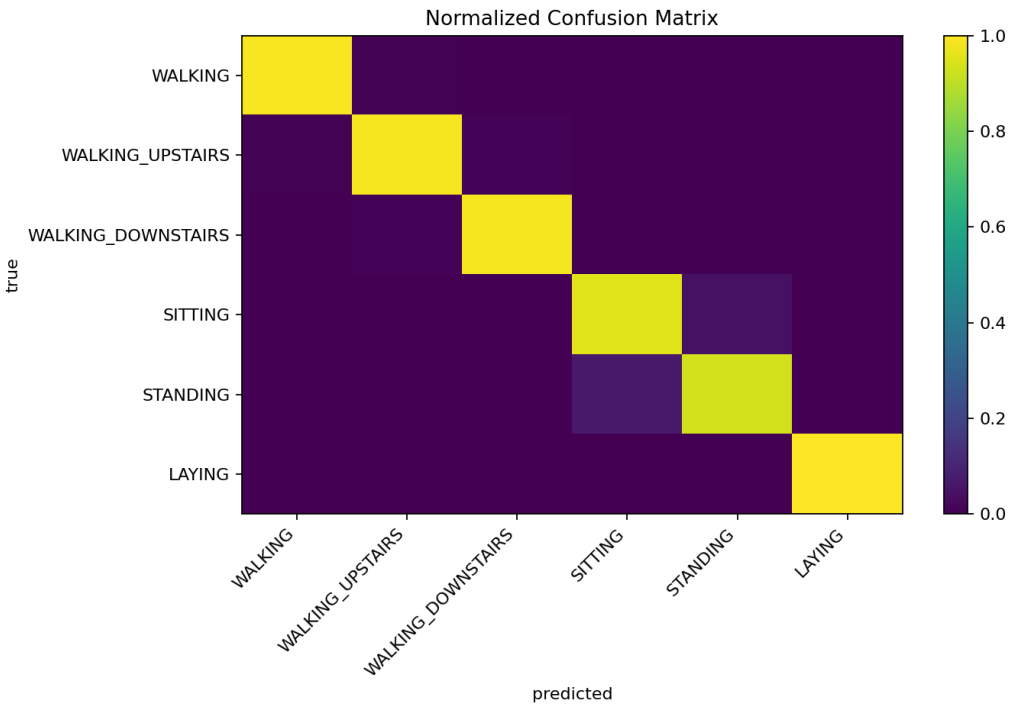

Figure 21.19: Normalized confusion matrix. Shows per-class behavior independent of support.

21.20 Per-class error rate

Ranks difficult activities. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

Figure 21.20: Per-class error rate. Ranks difficult activities.

21.21 Top confused pairs

Identifies common activity confusions. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

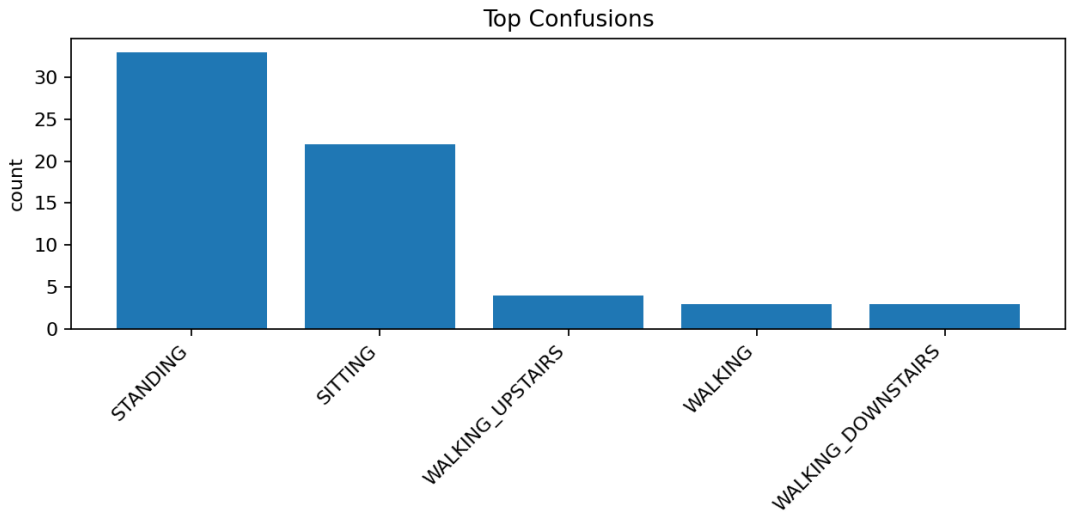

Figure 21.21: Top confused pairs. Identifies common activity confusions.

21.22 Per-client accuracy

Shows client-level fairness. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

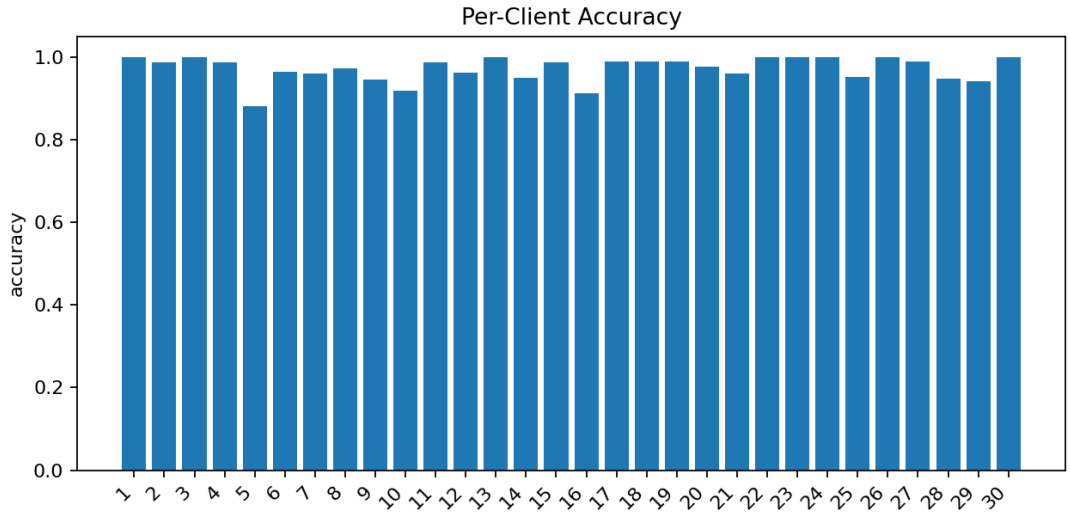

Figure 21.22: Per-client accuracy. Shows client-level fairness.

21.23 Baseline accuracy

Compares method performance. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

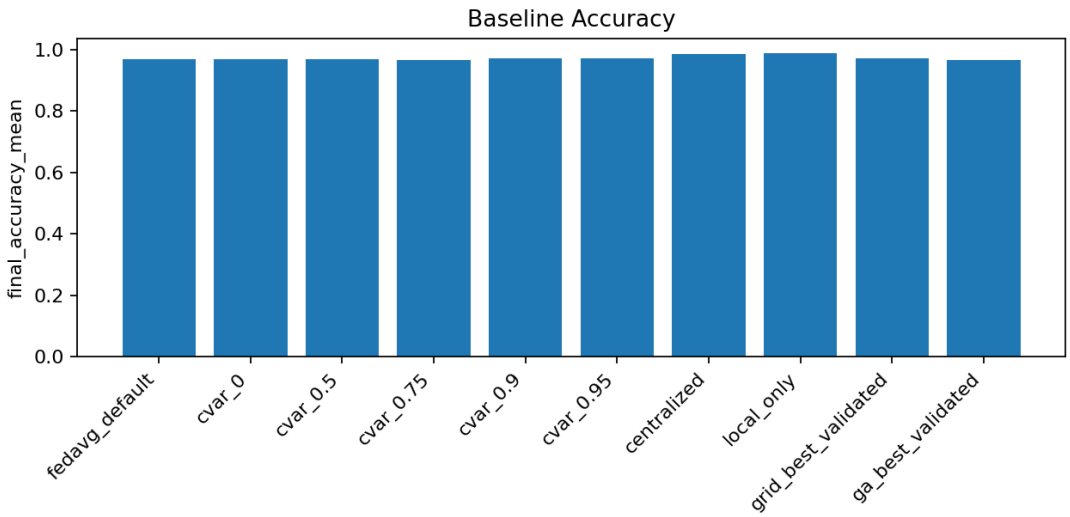

Figure 21.23: Baseline accuracy. Compares method performance.

21.24 Baseline worst-client accuracy

Compares fairness across methods. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

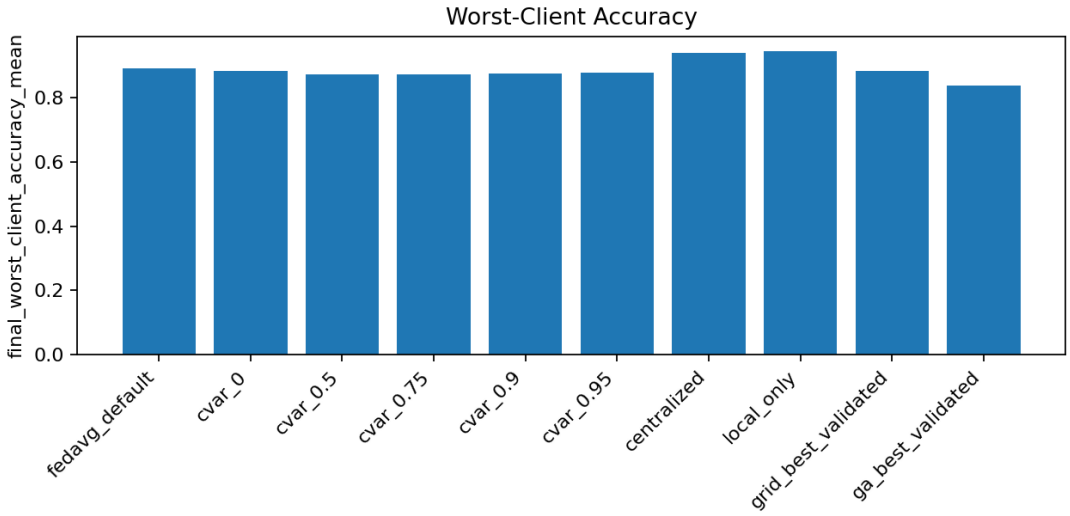

Figure 21.24: Baseline worst-client accuracy. Compares fairness across methods.

21.25 Baseline communication cost

Compares byte budgets. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

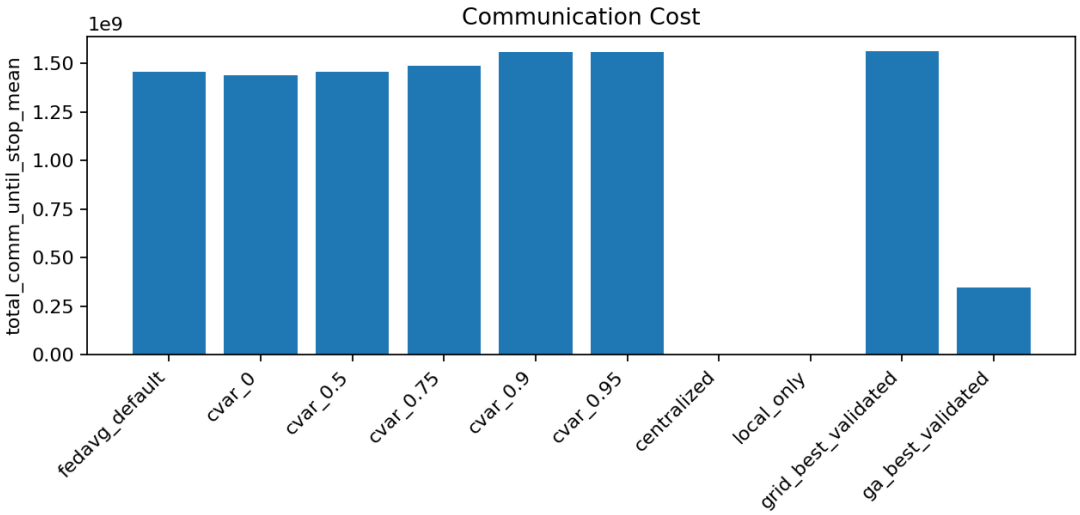

Figure 21.25: Baseline communication cost. Compares byte budgets.

21.26 Rounds to convergence

Compares early-stopped training length. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

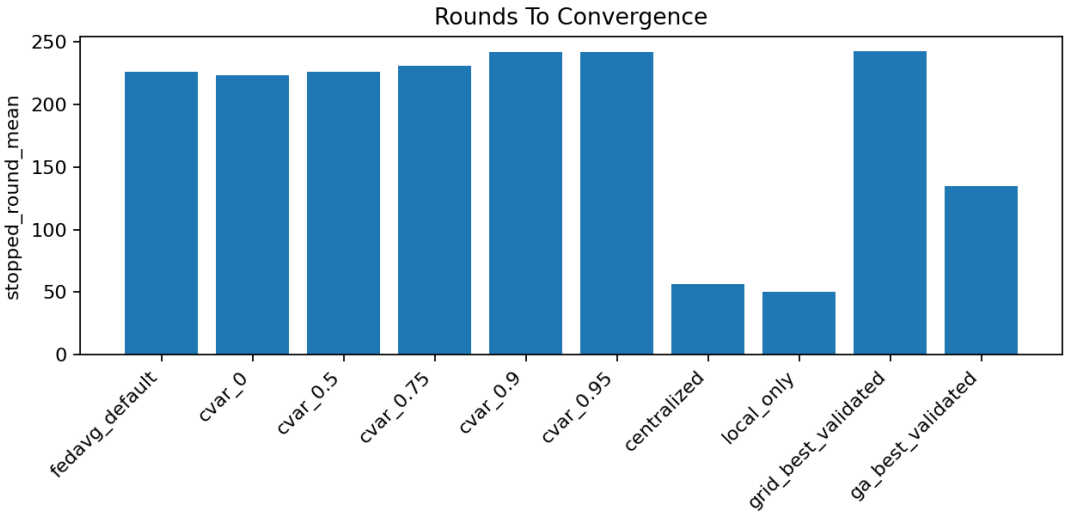

Figure 21.26: Rounds to convergence. Compares early-stopped training length.

21.27 Shadow price versus budget

Shows communication scarcity in the LP model. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

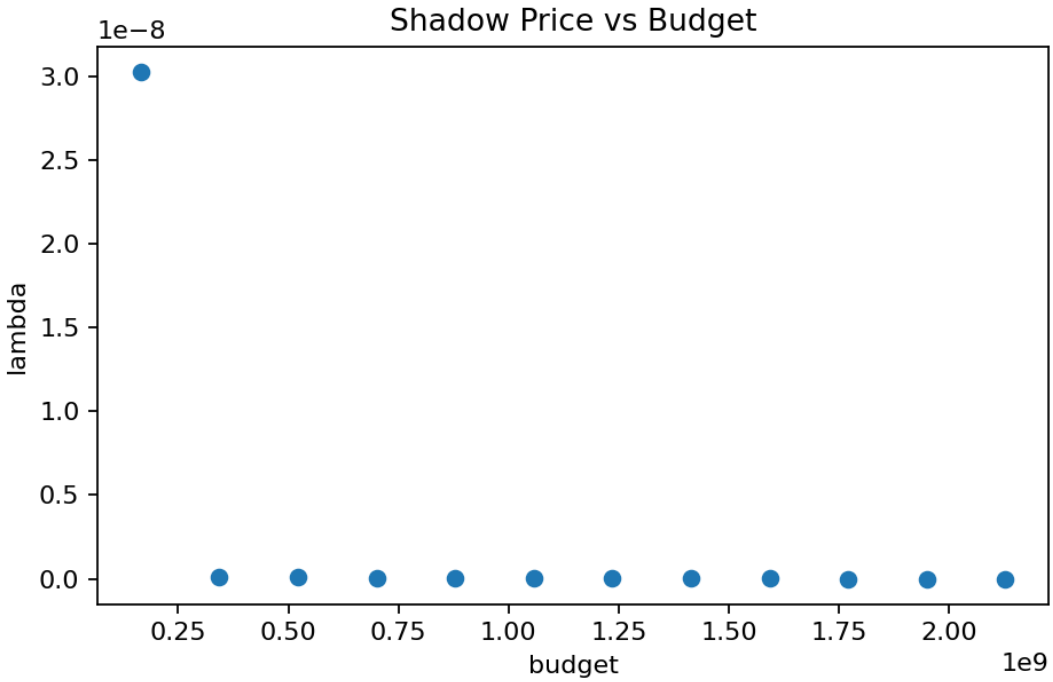

Figure 21.27: Shadow price versus budget. Shows communication scarcity in the LP model.

21.28 Loss versus budget

Shows objective improvement with budget. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

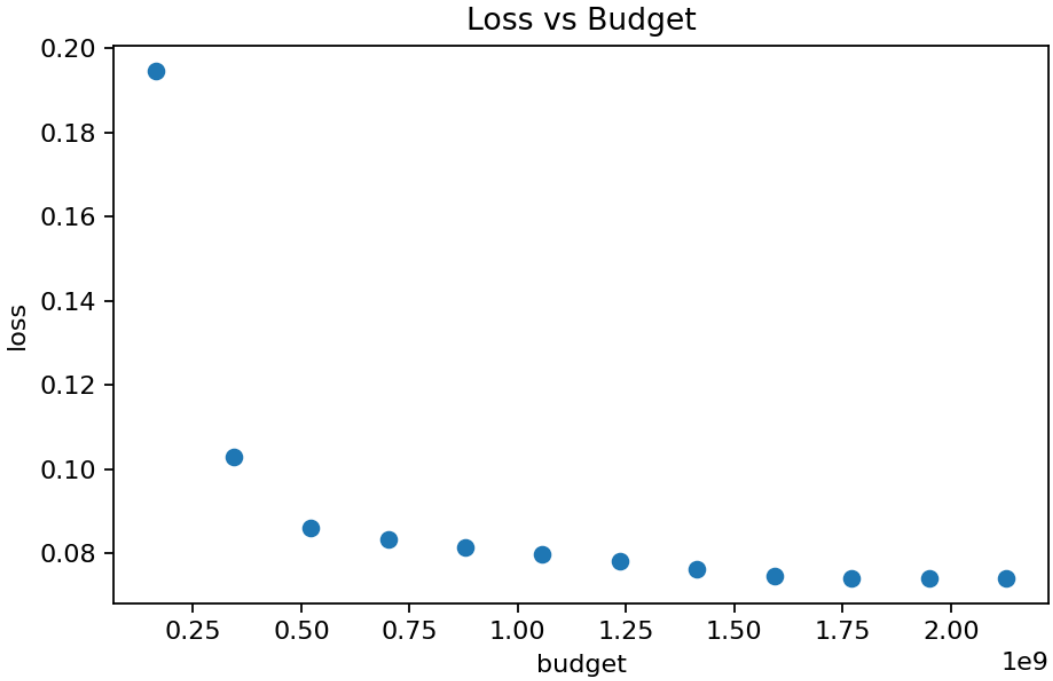

Figure 21.28: Loss versus budget. Shows objective improvement with budget.

21.29 KKT residuals

Checks LP numerical optimality. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

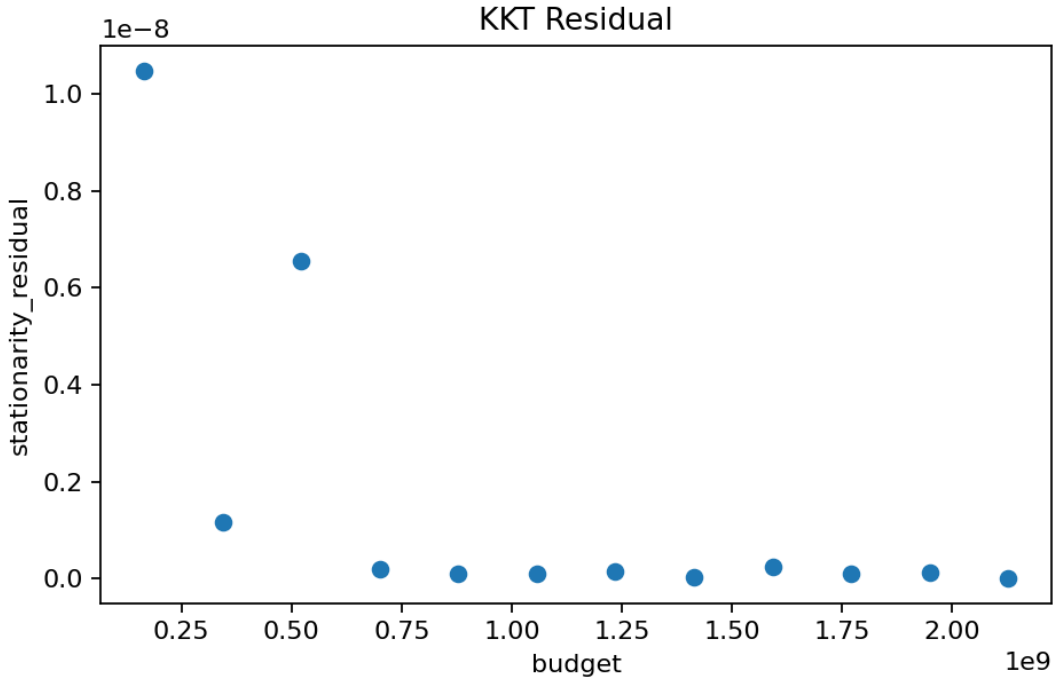

Figure 21.29: KKT residuals. Checks LP numerical optimality.

21.30 3D LP tradeoff

Shows budget, loss, and dual value together. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

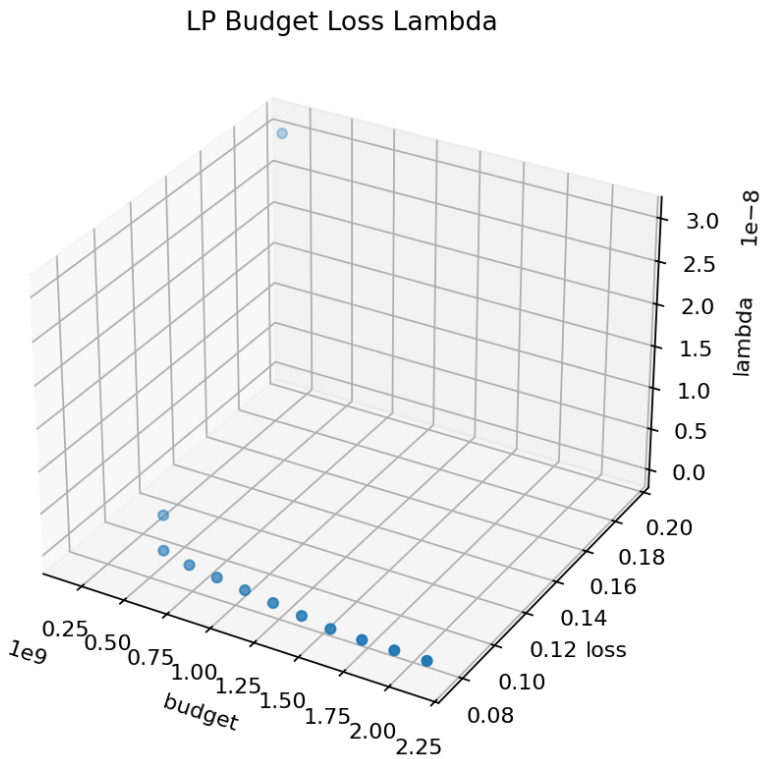

Figure 21.30: 3D LP tradeoff. Shows budget, loss, and dual value together.

21.31 Search accuracy versus communication

Shows hyperparameter tradeoffs. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

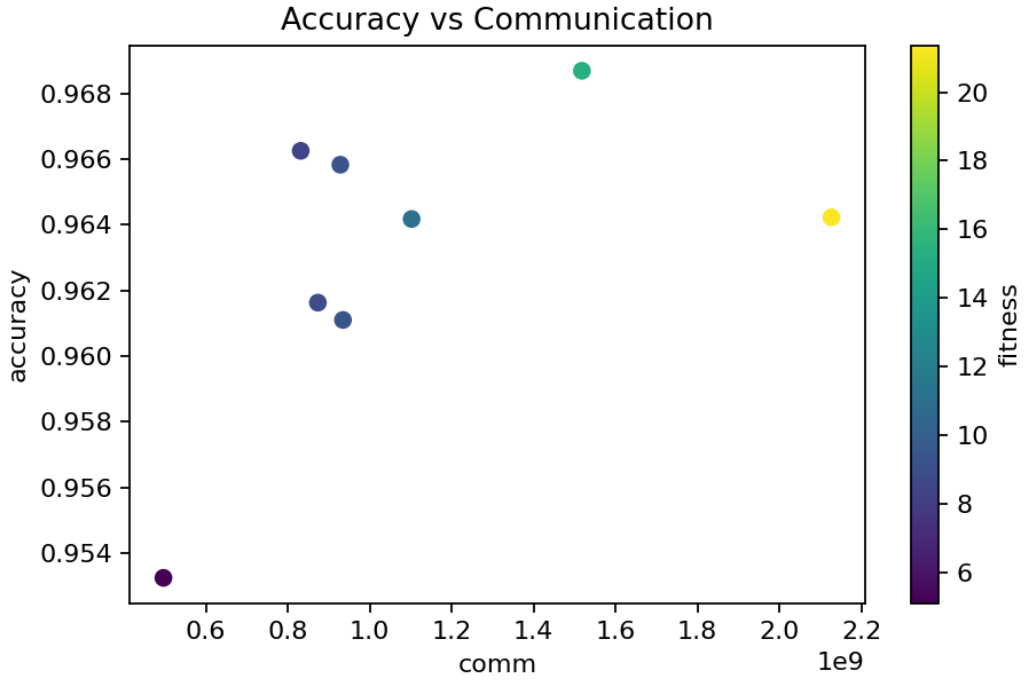

Figure 21.31: Search accuracy versus communication. Shows hyperparameter tradeoffs.

21.32 3D hyperparameter fitness

Shows the search landscape. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

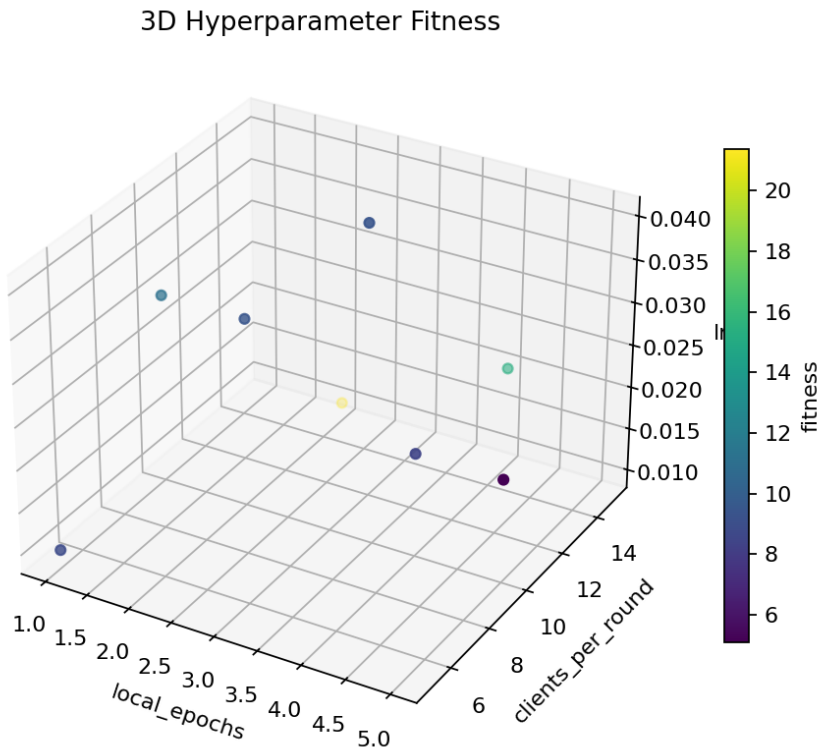

Figure 21.32: 3D hyperparameter fitness. Shows the search landscape.

21.33 GA fitness over evaluations

Shows differential-evolution progress. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

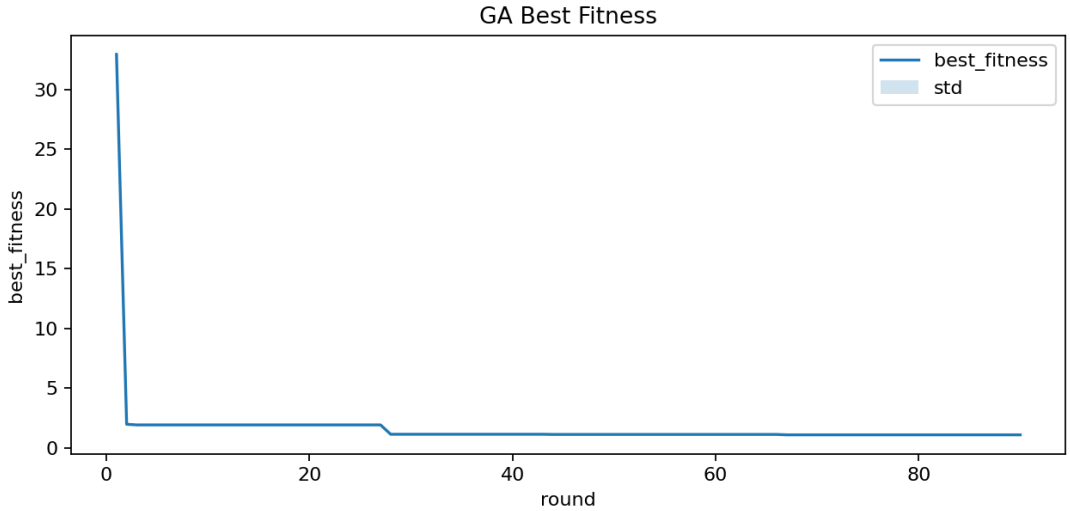

Figure 21.33: GA fitness over evaluations. Shows differential-evolution progress.

21.34 Accuracy per MB

Shows communication efficiency. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

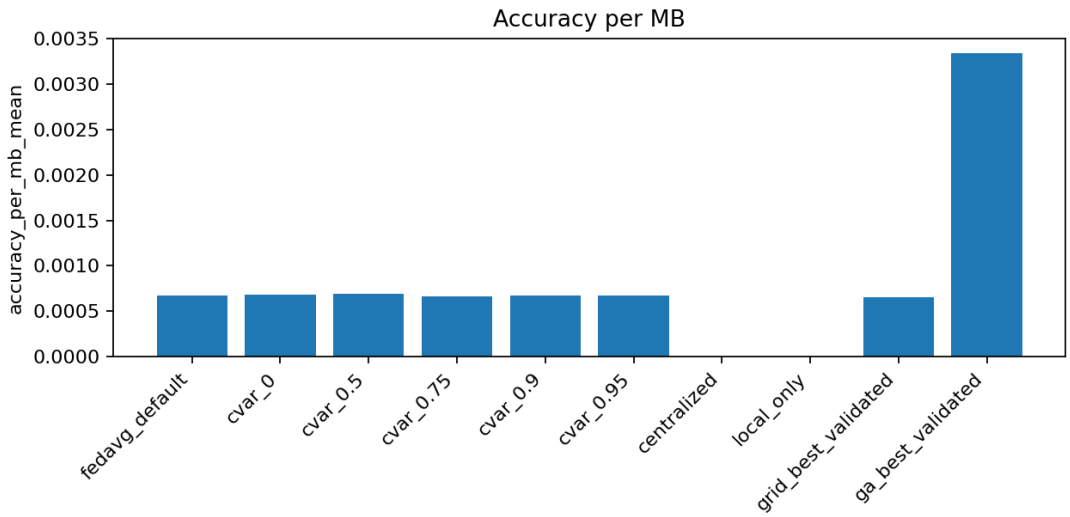

Figure 21.34: Accuracy per MB. Shows communication efficiency.

21.35 Accuracy-fairness-communication tradeoff

Shows the central multi-objective tension. The reason this is included is that a final accuracy number alone cannot explain the behavior of a federated system. Federated learning must be judged through convergence, fairness, communication, and robustness.

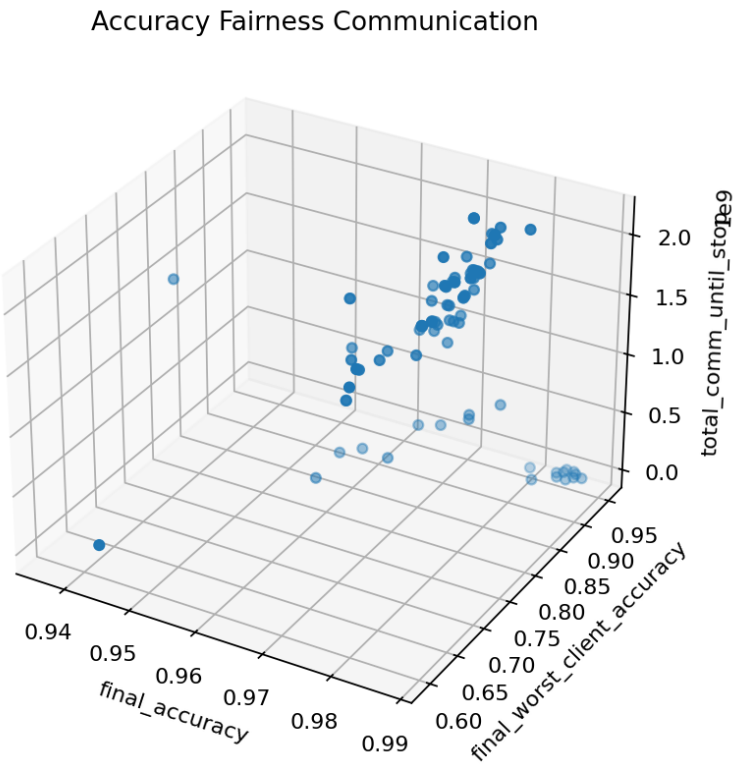

Figure 21.35: Accuracy-fairness-communication tradeoff. Shows the central multi-objective tension.

Chapter 22

Mathematical Formulation

The supervised learning objective minimizes empirical risk over labeled examples. If x is a 561-dimensional input vector and y is one of six activity labels, a neural network f_w maps x to class logits. Cross-entropy loss penalizes wrong predictions, especially confident wrong predictions.

The federated objective is a weighted sum of client objectives: $F(w) = \sum_i (n_i/N) F_i(w)$. FedAvg approximates this objective by selecting clients, training local models, and aggregating returned weights using sample-size weights.

The communication model uses $C_t = 2K_t(4d)$ bytes for round t, where d is the number of model parameters and K_t is the number of selected clients. This is interpretable but simplified.

The LP communication model solves $\min_x \sum_k L_k x_k$ subject to $\sum_k C_k x_k \leq B$, $\sum_k x_k = 1$, and $x_k \geq 0$. The dual variable of the budget constraint is the communication shadow price.

For non-convex DNNs, the correct goal is not a global optimality claim. The better theoretical language is convergence toward stationarity, measured with quantities like $\mathbb{E}\|\nabla F(w)\|^2$, plus empirical validation across seeds.

Chapter 23

Core Tables and Actual Outputs

The following tables are actual outputs from the v2 run. They are included so that the report is auditable. A reader can connect claims in the narrative to CSV artifacts produced by the experiment runner.

Table 23.1: Full

method	n	final_loss_mean	final_loss_std	final_accuracy_mean	final_accuracy_std	final_
fedavg_default	10.0000	0.084107	0.009160	0.969708	0.004035	0.891
cvar_0	10.0000	0.084477	0.008690	0.969549	0.004726	0.883
cvar_0.5	10.0000	0.086034	0.014758	0.970123	0.004895	0.874
cvar_0.75	10.0000	0.094282	0.038510	0.967479	0.011497	0.873
cvar_0.9	10.0000	0.082170	0.011273	0.970669	0.004367	0.876
cvar_0.95	10.0000	0.081762	0.011062	0.970815	0.004191	0.878
centralized	10.0000	0.062112	0.005889	0.984703	0.002026	0.940
local_only	1.0000	0.041006	0.000000	0.987801	0.000000	0.944
grid_best_validated	10.0000	0.089625	0.009678	0.972580	0.003750	0.882
ga_best_validated	10.0000	0.117797	0.033193	0.965375	0.009992	0.837

Table 23.2: Class distribution.

class_id	activity	train_count	test_count	total_count
0.000000	WALKING	1226.0000	496.0000	1722.0000
1.0000	WALKING_UPSTAIRS	1073.0000	471.0000	1544.0000
2.0000	WALKING_DOWNSTAIRS	986.0000	420.0000	1406.0000
3.0000	SITTING	1286.0000	491.0000	1777.0000
4.0000	STANDING	1374.0000	532.0000	1906.0000
5.0000	LAYING	1407.0000	537.0000	1944.0000

Table 23.3: Client counts and entropy.

client_id	train_samples	test_samples	total_samples	label_entropy	dominant_class_ratio
1.0000	347.0000	0.000000	347.0000	2.5319	0.273775
2.0000	0.000000	302.0000	302.0000	2.5790	0.195364
3.0000	341.0000	0.000000	341.0000	2.5799	0.181818
4.0000	0.000000	317.0000	317.0000	2.5792	0.189274
5.0000	302.0000	0.000000	302.0000	2.5788	0.185430
6.0000	325.0000	0.000000	325.0000	2.5819	0.175385
7.0000	308.0000	0.000000	308.0000	2.5820	0.185065
8.0000	281.0000	0.000000	281.0000	2.5730	0.192171
9.0000	0.000000	288.0000	288.0000	2.5812	0.180556
10.0000	0.000000	294.0000	294.0000	2.5711	0.197279
11.0000	316.0000	0.000000	316.0000	2.5790	0.186709
12.0000	0.000000	320.0000	320.0000	2.5776	0.190625
13.0000	0.000000	327.0000	327.0000	2.5786	0.189602
14.0000	323.0000	0.000000	323.0000	2.5786	0.185759
15.0000	328.0000	0.000000	328.0000	2.5642	0.219512
16.0000	366.0000	0.000000	366.0000	2.5582	0.213115
17.0000	368.0000	0.000000	368.0000	2.5593	0.211957
18.0000	0.000000	364.0000	364.0000	2.5772	0.200549
19.0000	360.0000	0.000000	360.0000	2.5244	0.230556
20.0000	0.000000	354.0000	354.0000	2.5624	0.206215
21.0000	408.0000	0.000000	408.0000	2.5204	0.220588
22.0000	321.0000	0.000000	321.0000	2.5425	0.224299
23.0000	372.0000	0.000000	372.0000	2.5734	0.193548
24.0000	0.000000	381.0000	381.0000	2.5776	0.188976
25.0000	409.0000	0.000000	409.0000	2.5793	0.180929
26.0000	392.0000	0.000000	392.0000	2.5641	0.198980
27.0000	376.0000	0.000000	376.0000	2.5538	0.212766
28.0000	382.0000	0.000000	382.0000	2.5509	0.209424
29.0000	344.0000	0.000000	344.0000	2.5712	0.200581
30.0000	383.0000	0.000000	383.0000	2.5829	0.182768

Table 23.4: Non-IID client metrics.

client_id	kl_divergence	js_divergence	label_entropy	dominant_class_ratio	samples
1.0000	0.041605	0.009927	2.5319	0.273775	347.0000
2.0000	0.007626	0.001908	2.5790	0.195364	302.0000
3.0000	0.003352	0.000834	2.5799	0.181818	341.0000
4.0000	0.003926	0.000978	2.5792	0.189274	317.0000
5.0000	0.005302	0.001332	2.5788	0.185430	302.0000
6.0000	0.001579	0.000395	2.5819	0.175385	325.0000
7.0000	0.005018	0.001253	2.5820	0.185065	308.0000
8.0000	0.000495	0.000124	2.5730	0.192171	281.0000
9.0000	0.005135	0.001293	2.5812	0.180556	288.0000

client_id	kl_divergence	js_divergence	label_entropy	dominant_class_ratio	samples
10.0000	0.005186	0.001325	2.5711	0.197279	294.0000
11.0000	0.006875	0.001740	2.5790	0.186709	316.0000
12.0000	0.001671	0.000418	2.5776	0.190625	320.0000
13.0000	0.003291	0.000827	2.5786	0.189602	327.0000
14.0000	0.004439	0.001122	2.5786	0.185759	323.0000
15.0000	0.004422	0.001099	2.5642	0.219512	328.0000
16.0000	0.005849	0.001468	2.5582	0.213115	366.0000
17.0000	0.003748	0.000936	2.5593	0.211957	368.0000
18.0000	0.003262	0.000815	2.5772	0.200549	364.0000
19.0000	0.017928	0.004532	2.5244	0.230556	360.0000
20.0000	0.003866	0.000971	2.5624	0.206215	354.0000
21.0000	0.021039	0.005323	2.5204	0.220588	408.0000
22.0000	0.010052	0.002521	2.5425	0.224299	321.0000
23.0000	0.001431	0.000359	2.5734	0.193548	372.0000
24.0000	0.001136	0.000286	2.5776	0.188976	381.0000
25.0000	0.001799	0.000450	2.5793	0.180929	409.0000
26.0000	0.003513	0.000872	2.5641	0.198980	392.0000
27.0000	0.005589	0.001399	2.5538	0.212766	376.0000
28.0000	0.007067	0.001778	2.5509	0.209424	382.0000
29.0000	0.001166	0.000293	2.5712	0.200581	344.0000
30.0000	0.006685	0.001670	2.5829	0.182768	383.0000

Table 23.5: Classification report.

label	precision	recall	f1-score	support
WALKING	0.993039	0.990741	0.991889	432.0000
WALKING_UPSTAIRS	0.981818	0.984375	0.983095	384.0000
WALKING_DOWNSTAIRS	0.985119	0.988060	0.986587	335.0000
SITTING	0.932238	0.949791	0.940933	478.0000
STANDING	0.952484	0.930380	0.941302	474.0000
LAYING	0.997925	1.0000	0.998962	481.0000
accuracy				
macro avg	0.973771	0.973891	0.973795	2584.0000
weighted avg	0.972568	0.972523	0.972506	2584.0000

Table 23.6: Per-client metrics.

client_id	test_samples	train_samples	accuracy	error_rate
1.0000	87.0000	260.0000	1.0000	0.000000
2.0000	76.0000	226.0000	0.986842	0.013158
3.0000	86.0000	255.0000	1.0000	0.000000
4.0000	80.0000	237.0000	0.987500	0.012500
5.0000	76.0000	226.0000	0.881579	0.118421

client_id	test_samples	train_samples	accuracy	error_rate
6.0000	82.0000	243.0000	0.963415	0.036585
7.0000	77.0000	231.0000	0.961039	0.038961
8.0000	71.0000	210.0000	0.971831	0.028169
9.0000	72.0000	216.0000	0.944444	0.055556
10.0000	74.0000	220.0000	0.918919	0.081081
11.0000	79.0000	237.0000	0.987342	0.012658
12.0000	80.0000	240.0000	0.962500	0.037500
13.0000	82.0000	245.0000	1.0000	0.000000
14.0000	81.0000	242.0000	0.950617	0.049383
15.0000	82.0000	246.0000	0.987805	0.012195
16.0000	92.0000	274.0000	0.913043	0.086957
17.0000	92.0000	276.0000	0.989130	0.010870
18.0000	91.0000	273.0000	0.989011	0.010989
19.0000	90.0000	270.0000	0.988889	0.011111
20.0000	89.0000	265.0000	0.977528	0.022472
21.0000	102.0000	306.0000	0.960784	0.039216
22.0000	81.0000	240.0000	1.0000	0.000000
23.0000	93.0000	279.0000	1.0000	0.000000
24.0000	96.0000	285.0000	1.0000	0.000000
25.0000	103.0000	306.0000	0.951456	0.048544
26.0000	98.0000	294.0000	1.0000	0.000000
27.0000	94.0000	282.0000	0.989362	0.010638
28.0000	96.0000	286.0000	0.947917	0.052083
29.0000	86.0000	258.0000	0.941860	0.058140
30.0000	96.0000	287.0000	1.0000	0.000000

Table 23.7: Per-class error rates.

label	support	error_rate	recall
WALKING	432.0000	0.009259	0.990741
WALKING_UPSTAIRS	384.0000	0.015625	0.984375
WALKING_DOWNSTAIRS	335.0000	0.011940	0.988060
SITTING	478.0000	0.050209	0.949791
STANDING	474.0000	0.069620	0.930380
LAYING	481.0000	0.000000	1.0000

Table 23.8: Top confusion pairs.

true_label	pred_label	count
STANDING	SITTING	33.0000
SITTING	STANDING	22.0000
WALKING_UPSTAIRS	WALKING_DOWNSTAIRS	4.0000
WALKING	WALKING_UPSTAIRS	3.0000

true_label	pred_label	count
WALKING_DOWNSTAIRS	WALKING_UPSTAIRS	3.0000
WALKING_UPSTAIRS	WALKING	2.0000
WALKING	WALKING_DOWNSTAIRS	1.0000
WALKING_DOWNSTAIRS	WALKING	1.0000
SITTING	WALKING_UPSTAIRS	1.0000
SITTING	LAYING	1.0000

Table 23.9: Calibration bins.

bin	lower	upper	count	accuracy	confidence	gap
0.000000	0.000000	0.100000	0.000000	0.000000	0.000000	0.000000
1.0000	0.100000	0.200000	0.000000	0.000000	0.000000	0.000000
2.0000	0.200000	0.300000	0.000000	0.000000	0.000000	0.000000
3.0000	0.300000	0.400000	0.000000	0.000000	0.000000	0.000000
4.0000	0.400000	0.500000	0.000000	0.000000	0.000000	0.000000
5.0000	0.500000	0.600000	13.0000	0.461538	0.549167	0.087629
6.0000	0.600000	0.700000	19.0000	0.631579	0.649460	0.017881
7.0000	0.700000	0.800000	31.0000	0.548387	0.752284	0.203897
8.0000	0.800000	0.900000	50.0000	0.760000	0.849821	0.089821
9.0000	0.900000	1.0000	2471.0000	0.987454	0.997041	0.009587

Table 23.10: Calibration summary.

ece	mce	mean_confidence	accuracy
0.013924	0.203897	0.986447	0.972523

Table 23.11: Runtime by stage.

stage	seconds
load_data	0.300682
eda	0.998332
fedavg_default	190.3501
cvar_sweep	978.7105
baselines	68.5492
search	2275.9283
predictions_metrics	0.051374
lp_diagnostics	0.193287
diagnostics	0.621627
plots	1.2861
artifacts	0.008271

Table 23.12: Theoretical alignment.

component	optimization_concept
FedAvg	local SGD and stochastic optimization
LP shadow price	duality and constrained optimization
KKT residuals	optimality diagnostics
CVaR weighting	robust and fairness-aware optimization
Differential evolution	global search
Early stopping	empirical convergence criterion

Chapter 24

Optimization, Search, and Statistical Tables

This chapter includes the optimization outputs: grid search, differential evolution, LP diagnostics, confidence intervals, paired tests, ablation deltas, and communication efficiency. These tables show how the project connects empirical ML training with optimization analysis.

Table 24.1: Grid-search candidates.

local_epochs	clients_per_round	lr	loss	accuracy	comm	fitness
5.0000	10.0000	0.020000	0.141079	0.953244	4.964e+08	5.1049
4.0000	10.0000	0.020000	0.096551	0.966259	8.316e+08	8.4126
1.0000	5.0000	0.010000	0.097887	0.961631	8.735e+08	8.8330
3.0000	12.0000	0.040000	0.095697	0.965835	9.283e+08	9.3787
2.0000	10.0000	0.030000	0.097295	0.961103	9.348e+08	9.4448
1.0000	10.0000	0.030000	0.092500	0.964181	1.102e+09	11.1161
4.0000	15.0000	0.020000	0.091335	0.968699	1.518e+09	15.2730
2.0000	15.0000	0.010000	0.089977	0.964232	2.127e+09	21.3636

Table 24.2: Differential-evolution objective evaluations.

evaluation	local_epochs	clients_per_round	lr	loss	accuracy	comm	fitness
1.0000	2.0000	17.0000	0.007752	0.080299	0.970504	3.288e+09	32.9578
2.0000	2.0000	4.0000	0.073076	0.122883	0.957465	1.857e+08	1.9795
3.0000	4.0000	5.0000	0.034129	0.153980	0.949575	1.773e+08	1.9268
4.0000	4.0000	20.0000	0.017694	0.089476	0.970166	2.630e+09	26.3914
5.0000	1.0000	7.0000	0.038408	0.105864	0.956330	4.828e+08	4.9343
6.0000	4.0000	15.0000	0.052955	0.090736	0.970335	1.431e+09	14.4021
7.0000	5.0000	13.0000	0.056180	0.106655	0.964765	9.219e+08	9.3252
8.0000	7.0000	8.0000	0.064459	0.168334	0.951351	2.011e+08	2.1797
9.0000	3.0000	12.0000	0.010956	0.090634	0.965358	1.501e+09	15.0982
10.0000	3.0000	9.0000	0.029969	0.092039	0.967779	1.108e+09	11.1737
11.0000	2.0000	11.0000	0.002924	0.117272	0.960869	2.127e+09	21.3909
12.0000	8.0000	6.0000	0.046523	0.163256	0.949081	1.857e+08	2.0199

evaluation	local_epochs	clients_per_round	lr	loss	accuracy	comm	fitness
13.0000	5.0000	19.0000	0.059605	0.113641	0.965048	1.029e+09	10.4024
14.0000	5.0000	5.0000	0.042553	0.159492	0.949634	1.773e+08	1.9323
15.0000	6.0000	16.0000	0.077906	0.180080	0.947851	3.094e+08	3.2744
16.0000	6.0000	10.0000	0.019820	0.141150	0.951581	4.964e+08	5.1050
17.0000	7.0000	18.0000	0.024929	0.105753	0.968380	1.706e+09	17.1634
18.0000	7.0000	14.0000	0.070812	0.110152	0.962747	9.115e+08	9.2256
19.0000	5.0000	15.0000	0.057337	0.109562	0.963440	8.896e+08	9.0058
20.0000	7.0000	9.0000	0.062824	0.127854	0.960960	4.932e+08	5.0595
21.0000	5.0000	17.0000	0.047329	0.097135	0.971436	1.644e+09	16.5359
22.0000	2.0000	11.0000	0.026061	0.093173	0.965979	1.085e+09	10.9427
23.0000	1.0000	11.0000	0.027903	0.092962	0.965542	1.525e+09	15.3391
24.0000	4.0000	13.0000	0.005309	0.101262	0.960862	1.500e+09	15.1024
25.0000	5.0000	9.0000	0.056180	0.126800	0.960499	4.990e+08	5.1164
26.0000	5.0000	15.0000	0.048001	0.113451	0.961719	8.896e+08	9.0097
27.0000	5.0000	7.0000	0.019591	0.118459	0.961211	4.828e+08	4.9469
28.0000	7.0000	4.0000	0.036610	0.187373	0.940745	9.541e+07	1.1415
29.0000	2.0000	19.0000	0.002924	0.117796	0.956289	3.675e+09	36.8632
30.0000	7.0000	8.0000	0.056689	0.157105	0.954020	2.011e+08	2.1684
31.0000	5.0000	4.0000	0.005576	0.120016	0.956674	3.094e+08	3.2144
32.0000	5.0000	5.0000	0.042553	0.156768	0.951111	1.773e+08	1.9296
33.0000	7.0000	10.0000	0.047879	0.138545	0.956897	4.964e+08	5.1024
34.0000	6.0000	16.0000	0.009684	0.102667	0.966516	1.681e+09	16.9153
35.0000	7.0000	4.0000	0.024929	0.142338	0.954694	1.753e+08	1.8958
36.0000	8.0000	14.0000	0.011891	0.113383	0.962297	1.038e+09	10.4923
37.0000	7.0000	4.0000	0.014927	0.141744	0.955564	1.753e+08	1.8952
38.0000	7.0000	8.0000	0.050211	0.158446	0.951943	2.011e+08	2.1698
39.0000	8.0000	17.0000	0.058618	0.092166	0.971216	1.644e+09	16.5309
40.0000	3.0000	11.0000	0.030384	0.100287	0.964328	9.360e+08	9.4607
41.0000	3.0000	15.0000	0.061051	0.081464	0.973120	1.518e+09	15.2631
42.0000	5.0000	11.0000	0.033963	0.128646	0.962659	5.602e+08	5.7307
43.0000	6.0000	17.0000	0.056180	0.093900	0.972447	1.644e+09	16.5326
44.0000	7.0000	4.0000	0.072310	0.173609	0.946842	9.541e+07	1.1277
45.0000	4.0000	5.0000	0.019591	0.161706	0.945464	1.773e+08	1.9345
46.0000	3.0000	11.0000	0.030465	0.098493	0.962848	9.360e+08	9.4589
47.0000	5.0000	11.0000	0.005707	0.085521	0.969243	1.624e+09	16.3244
48.0000	4.0000	7.0000	0.031012	0.114623	0.962528	4.828e+08	4.9431
49.0000	5.0000	15.0000	0.044107	0.111170	0.960939	8.896e+08	9.0074
50.0000	5.0000	9.0000	0.072145	0.122408	0.960536	4.990e+08	5.1120
51.0000	6.0000	16.0000	0.021257	0.121685	0.959878	9.180e+08	9.3016
52.0000	2.0000	4.0000	0.019820	0.126813	0.954259	3.197e+08	3.3243
53.0000	3.0000	8.0000	0.058195	0.143495	0.947422	2.011e+08	2.1548
54.0000	7.0000	9.0000	0.043943	0.130929	0.964230	5.570e+08	5.7008
55.0000	2.0000	4.0000	0.026926	0.129366	0.952788	1.857e+08	1.9860
56.0000	3.0000	6.0000	0.073076	0.167687	0.946676	2.011e+08	2.1790
57.0000	5.0000	5.0000	0.041363	0.149433	0.953183	1.773e+08	1.9222
58.0000	5.0000	13.0000	0.076217	0.110922	0.963534	9.219e+08	9.3295

evaluation	local_epochs	clients_per_round	lr	loss	accuracy	comm	fitness
59.0000	1.0000	7.0000	0.005846	0.123020	0.952962	1.354e+09	13.6608
60.0000	5.0000	11.0000	0.005846	0.101201	0.961540	1.106e+09	11.1635
61.0000	5.0000	6.0000	0.075095	0.158441	0.954238	2.050e+08	2.2084
62.0000	4.0000	9.0000	0.046485	0.096732	0.969545	9.167e+08	9.2637
63.0000	4.0000	3.0000	0.015214	0.149849	0.948879	1.605e+08	1.7550
64.0000	7.0000	14.0000	0.054789	0.114381	0.963265	9.657e+08	9.7713
65.0000	5.0000	11.0000	0.005707	0.085709	0.967737	1.603e+09	16.1119
66.0000	7.0000	6.0000	0.065399	0.153557	0.955538	1.895e+08	2.0488
67.0000	5.0000	4.0000	0.074823	0.165271	0.947484	9.283e+07	1.0936
68.0000	7.0000	13.0000	0.077783	0.140175	0.957390	5.028e+08	5.1685
69.0000	2.0000	3.0000	0.054654	0.170943	0.940493	9.670e+07	1.1379
70.0000	2.0000	10.0000	0.070343	0.088640	0.965741	9.348e+08	9.4362
71.0000	2.0000	4.0000	0.024929	0.113064	0.958552	2.965e+08	3.0785
72.0000	5.0000	4.0000	0.037901	0.134747	0.955497	1.779e+08	1.9140
73.0000	5.0000	11.0000	0.076859	0.126160	0.963501	5.602e+08	5.7282
74.0000	3.0000	5.0000	0.076743	0.131963	0.955757	1.837e+08	1.9692
75.0000	5.0000	9.0000	0.051612	0.087491	0.971713	9.515e+08	9.6026
76.0000	7.0000	13.0000	0.047152	0.101158	0.965280	9.219e+08	9.3197
77.0000	4.0000	4.0000	0.075643	0.140333	0.957675	1.521e+08	1.6617
78.0000	3.0000	4.0000	0.051098	0.135009	0.954622	2.140e+08	2.2753
79.0000	2.0000	12.0000	0.009076	0.095513	0.961860	1.733e+09	17.4239
80.0000	7.0000	4.0000	0.036610	0.132117	0.957604	1.753e+08	1.8856
81.0000	3.0000	10.0000	0.070831	0.124348	0.956281	4.964e+08	5.0882
82.0000	4.0000	9.0000	0.078561	0.113231	0.963011	5.106e+08	5.2189
83.0000	7.0000	11.0000	0.055978	0.174317	0.949396	2.269e+08	2.4435
84.0000	7.0000	5.0000	0.066479	0.165025	0.953060	1.547e+08	1.7122
85.0000	7.0000	5.0000	0.020623	0.172415	0.947318	1.773e+08	1.9452
86.0000	5.0000	8.0000	0.075630	0.163132	0.953099	2.011e+08	2.1745
87.0000	5.0000	4.0000	0.077209	0.133099	0.961245	1.779e+08	1.9123
88.0000	4.0000	5.0000	0.030699	0.151269	0.949710	1.773e+08	1.9241
89.0000	7.0000	3.0000	0.045442	0.157622	0.951361	9.476e+07	1.1053
90.0000	5.0000	4.0000	0.064217	0.134197	0.956515	1.470e+08	1.6040

Table 24.3: GA best-so-far history.

round	best_fitness
1.0000	32.9578
2.0000	1.9795
3.0000	1.9268
4.0000	1.9268
5.0000	1.9268
6.0000	1.9268
7.0000	1.9268
8.0000	1.9268
9.0000	1.9268

round	best_fitness
10.0000	1.9268
11.0000	1.9268
12.0000	1.9268
13.0000	1.9268
14.0000	1.9268
15.0000	1.9268
16.0000	1.9268
17.0000	1.9268
18.0000	1.9268
19.0000	1.9268
20.0000	1.9268
21.0000	1.9268
22.0000	1.9268
23.0000	1.9268
24.0000	1.9268
25.0000	1.9268
26.0000	1.9268
27.0000	1.9268
28.0000	1.1415
29.0000	1.1415
30.0000	1.1415
31.0000	1.1415
32.0000	1.1415
33.0000	1.1415
34.0000	1.1415
35.0000	1.1415
36.0000	1.1415
37.0000	1.1415
38.0000	1.1415
39.0000	1.1415
40.0000	1.1415
41.0000	1.1415
42.0000	1.1415
43.0000	1.1415
44.0000	1.1277
45.0000	1.1277
46.0000	1.1277
47.0000	1.1277
48.0000	1.1277
49.0000	1.1277
50.0000	1.1277
51.0000	1.1277
52.0000	1.1277
53.0000	1.1277
54.0000	1.1277
55.0000	1.1277

round	best_fitness
56.0000	1.1277
57.0000	1.1277
58.0000	1.1277
59.0000	1.1277
60.0000	1.1277
61.0000	1.1277
62.0000	1.1277
63.0000	1.1277
64.0000	1.1277
65.0000	1.1277
66.0000	1.1277
67.0000	1.0936
68.0000	1.0936
69.0000	1.0936
70.0000	1.0936
71.0000	1.0936
72.0000	1.0936
73.0000	1.0936
74.0000	1.0936
75.0000	1.0936
76.0000	1.0936
77.0000	1.0936
78.0000	1.0936
79.0000	1.0936
80.0000	1.0936
81.0000	1.0936
82.0000	1.0936
83.0000	1.0936
84.0000	1.0936
85.0000	1.0936
86.0000	1.0936
87.0000	1.0936
88.0000	1.0936
89.0000	1.0936
90.0000	1.0936

Table 24.4: LP shadow-price and KKT diagnostics.

budget	loss	cost	lambda	status	primal_feasible	dual_feasible	complementary_slackn
1.650e+08	0.194650	1.650e+08	0.000000	optimal	True	True	0.000000
3.434e+08	0.103093	3.434e+08	0.000000	optimal	True	True	0.000000
5.218e+08	0.086011	5.218e+08	0.000000	optimal	True	True	0.000000
7.002e+08	0.083351	7.002e+08	0.000000	optimal	True	True	0.000000
8.786e+08	0.081608	8.786e+08	0.000000	optimal	True	True	0.000000
1.057e+09	0.079864	1.057e+09	0.000000	optimal	True	True	0.000000

budget	loss	cost	lambda	status	primal_feasible	dual_feasible	complementary_slackn
1.235e+09	0.078121	1.235e+09	0.000000	optimal	True	True	0.000000
1.414e+09	0.076377	1.414e+09	0.000000	optimal	True	True	0.000000
1.592e+09	0.074634	1.592e+09	0.000000	optimal	True	True	0.000000
1.771e+09	0.074254	1.631e+09	0.000000	optimal	True	True	0.000000
1.949e+09	0.074254	1.631e+09	0.000000	optimal	True	True	0.000000
2.127e+09	0.074254	1.631e+09	0.000000	optimal	True	True	0.000000

Table 24.5: Confidence intervals.

group	metric	n	mean	std	ci95_low	ci95_high
fedavg_default	final_accuracy	10.0000	0.969708	0.004035	0.967207	0.972209
fedavg_default	final_worst_client_accuracy	10.0000	0.891248	0.017209	0.880582	0.901914
fedavg_default	final_loss	10.0000	0.084107	0.009160	0.078429	0.089785
fedavg_default	total_comm_until_stop	10.0000	1.460e+09	1.776e+08	1.349e+09	1.570e+09
fedavg_default	stopped_round	10.0000	226.4000	27.5568	209.3201	243.4799
cvar_0	final_accuracy	10.0000	0.969549	0.004726	0.966620	0.972478
cvar_0	final_worst_client_accuracy	10.0000	0.883922	0.033206	0.863341	0.904503
cvar_0	final_loss	10.0000	0.084477	0.008690	0.079091	0.089863
cvar_0	total_comm_until_stop	10.0000	1.440e+09	1.761e+08	1.331e+09	1.549e+09
cvar_0	stopped_round	10.0000	223.4000	27.3097	206.4732	240.3268
cvar_0.5	final_accuracy	10.0000	0.970123	0.004895	0.967089	0.973157
cvar_0.5	final_worst_client_accuracy	10.0000	0.874004	0.038472	0.850159	0.897849
cvar_0.5	final_loss	10.0000	0.086034	0.014758	0.076887	0.095182
cvar_0.5	total_comm_until_stop	10.0000	1.459e+09	2.961e+08	1.275e+09	1.642e+09
cvar_0.5	stopped_round	10.0000	226.3000	45.9276	197.8338	254.7662
cvar_0.75	final_accuracy	10.0000	0.967479	0.011497	0.960353	0.974605
cvar_0.75	final_worst_client_accuracy	10.0000	0.873944	0.043294	0.847111	0.900778
cvar_0.75	final_loss	10.0000	0.094282	0.038510	0.070414	0.118151
cvar_0.75	total_comm_until_stop	10.0000	1.490e+09	2.711e+08	1.322e+09	1.658e+09
cvar_0.75	stopped_round	10.0000	231.1000	42.0488	205.0379	257.1621
cvar_0.9	final_accuracy	10.0000	0.970669	0.004367	0.967963	0.973376
cvar_0.9	final_worst_client_accuracy	10.0000	0.876384	0.045831	0.847977	0.904791
cvar_0.9	final_loss	10.0000	0.082170	0.011273	0.075183	0.089157
cvar_0.9	total_comm_until_stop	10.0000	1.557e+09	4.054e+08	1.306e+09	1.809e+09
cvar_0.9	stopped_round	10.0000	241.6000	62.8847	202.6236	280.5764
cvar_0.95	final_accuracy	10.0000	0.970815	0.004191	0.968218	0.973413
cvar_0.95	final_worst_client_accuracy	10.0000	0.878853	0.041087	0.853387	0.904319
cvar_0.95	final_loss	10.0000	0.081762	0.011062	0.074906	0.088618
cvar_0.95	total_comm_until_stop	10.0000	1.557e+09	4.054e+08	1.306e+09	1.809e+09
cvar_0.95	stopped_round	10.0000	241.6000	62.8847	202.6236	280.5764
centralized	final_accuracy	10.0000	0.984703	0.002026	0.983447	0.985959
centralized	final_worst_client_accuracy	10.0000	0.940667	0.009813	0.934585	0.946749
centralized	final_loss	10.0000	0.062112	0.005889	0.058462	0.065762
centralized	total_comm_until_stop	10.0000	0.000000	0.000000	0.000000	0.000000
centralized	stopped_round	10.0000	56.4000	3.9215	53.9695	58.8305

group	metric	n	mean	std	ci95_low	ci95_high
local_only	final_accuracy	1.0000	0.987801	0.000000	0.987801	0.987801
local_only	final_worst_client_accuracy	1.0000	0.944444	0.000000	0.944444	0.944444
local_only	final_loss	1.0000	0.041006	0.000000	0.041006	0.041006
local_only	total_comm_until_stop	1.0000	0.000000	0.000000	0.000000	0.000000
local_only	stopped_round	1.0000	50.0000	0.000000	50.0000	50.0000
grid_best_validated	final_accuracy	10.0000	0.972580	0.003750	0.970256	0.974904
grid_best_validated	final_worst_client_accuracy	10.0000	0.882650	0.048662	0.852490	0.912811
grid_best_validated	final_loss	10.0000	0.089625	0.009678	0.083626	0.095623
grid_best_validated	total_comm_until_stop	10.0000	1.562e+09	3.630e+08	1.337e+09	1.787e+09
grid_best_validated	stopped_round	10.0000	242.3000	56.3107	207.3982	277.2018
ga_best_validated	final_accuracy	10.0000	0.965375	0.009992	0.959182	0.971567
ga_best_validated	final_worst_client_accuracy	10.0000	0.837289	0.094306	0.778837	0.895740
ga_best_validated	final_loss	10.0000	0.117797	0.033193	0.097223	0.138370
ga_best_validated	total_comm_until_stop	10.0000	3.473e+08	1.519e+08	2.532e+08	4.415e+08
ga_best_validated	stopped_round	10.0000	134.7000	58.8936	98.1974	171.2026

Table 24.6: Paired tests against FedAvg.

baseline	method	metric	n	mean_diff	paired_t_p	wilcoxon
fedavg_default	centralized	final_accuracy	10.0000	0.014995	0.000002	0.001953
fedavg_default	centralized	final_worst_client_accuracy	10.0000	0.049419	0.000016	0.001953
fedavg_default	centralized	final_loss	10.0000	-0.021995	0.000128	0.001953
fedavg_default	centralized	total_comm_until_stop	10.0000	-1.460e+09	0.000000	0.001953
fedavg_default	centralized	stopped_round	10.0000	-170.0000	0.000000	0.001953
fedavg_default	cvar_0	final_accuracy	10.0000	-0.000159	0.809736	0.875000
fedavg_default	cvar_0	final_worst_client_accuracy	10.0000	-0.007326	0.451455	0.750000
fedavg_default	cvar_0	final_loss	10.0000	0.000370	0.683026	0.625000
fedavg_default	cvar_0	total_comm_until_stop	10.0000	-1.934e+07	0.279609	0.500000
fedavg_default	cvar_0	stopped_round	10.0000	-3.0000	0.279609	0.500000
fedavg_default	cvar_0.5	final_accuracy	10.0000	0.000415	0.850306	1.0000
fedavg_default	cvar_0.5	final_worst_client_accuracy	10.0000	-0.017244	0.241894	0.312500
fedavg_default	cvar_0.5	final_loss	10.0000	0.001927	0.749102	0.556641
fedavg_default	cvar_0.5	total_comm_until_stop	10.0000	-6.447e+05	0.994986	0.812500
fedavg_default	cvar_0.5	stopped_round	10.0000	-0.100000	0.994986	0.812500
fedavg_default	cvar_0.75	final_accuracy	10.0000	-0.002229	0.377259	0.695312
fedavg_default	cvar_0.75	final_worst_client_accuracy	10.0000	-0.017304	0.145948	0.230469
fedavg_default	cvar_0.75	final_loss	10.0000	0.010175	0.320120	0.375000
fedavg_default	cvar_0.75	total_comm_until_stop	10.0000	3.030e+07	0.559430	1.0000
fedavg_default	cvar_0.75	stopped_round	10.0000	4.7000	0.559430	1.0000
fedavg_default	cvar_0.9	final_accuracy	10.0000	0.000961	0.645407	0.769531
fedavg_default	cvar_0.9	final_worst_client_accuracy	10.0000	-0.014864	0.380289	0.625000
fedavg_default	cvar_0.9	final_loss	10.0000	-0.001937	0.683862	0.695312
fedavg_default	cvar_0.9	total_comm_until_stop	10.0000	9.799e+07	0.421339	0.375000
fedavg_default	cvar_0.9	stopped_round	10.0000	15.2000	0.421339	0.375000
fedavg_default	cvar_0.95	final_accuracy	10.0000	0.001107	0.590879	0.769531

baseline	method	metric	n	mean_diff	paired_t_p	wilcoxon
fedavg_default	cvar_0.95	final_worst_client_accuracy	10.0000	-0.012395	0.424598	0.625000
fedavg_default	cvar_0.95	final_loss	10.0000	-0.002345	0.618983	0.556641
fedavg_default	cvar_0.95	total_comm_until_stop	10.0000	9.799e+07	0.421339	0.375000
fedavg_default	cvar_0.95	stopped_round	10.0000	15.2000	0.421339	0.375000
fedavg_default	ga_best_validated	final_accuracy	10.0000	-0.004334	0.107415	0.105469
fedavg_default	ga_best_validated	final_worst_client_accuracy	10.0000	-0.053959	0.072234	0.027344
fedavg_default	ga_best_validated	final_loss	10.0000	0.033690	0.004416	0.003906
fedavg_default	ga_best_validated	total_comm_until_stop	10.0000	-1.112e+09	0.000000	0.001953
fedavg_default	ga_best_validated	stopped_round	10.0000	-91.7000	0.002047	0.005859
fedavg_default	grid_best_validated	final_accuracy	10.0000	0.002872	0.122539	0.193359
fedavg_default	grid_best_validated	final_worst_client_accuracy	10.0000	-0.008598	0.632310	1.0000
fedavg_default	grid_best_validated	final_loss	10.0000	0.005518	0.242693	0.193359
fedavg_default	grid_best_validated	total_comm_until_stop	10.0000	1.025e+08	0.362268	0.382812
fedavg_default	grid_best_validated	stopped_round	10.0000	15.9000	0.362268	0.382812

Table 24.7: Ablation-style deltas.

method	seed	final_loss	final_accuracy	final_worst_client_accuracy	delta_accuracy	delta_w
fedavg_default	7.0000	0.084216	0.969140	0.888889	0.000000	0.00000
fedavg_default	11.0000	0.091356	0.969271	0.881579	0.000000	0.00000
fedavg_default	19.0000	0.074832	0.972361	0.907895	0.000000	0.00000
fedavg_default	23.0000	0.084768	0.970192	0.881579	0.000000	0.00000
fedavg_default	29.0000	0.082076	0.969563	0.905405	0.000000	0.00000
fedavg_default	31.0000	0.084339	0.968746	0.888889	0.000000	0.00000
fedavg_default	37.0000	0.105881	0.959307	0.851852	0.000000	0.00000
fedavg_default	41.0000	0.074254	0.973141	0.906250	0.000000	0.00000
fedavg_default	43.0000	0.080542	0.972976	0.894737	0.000000	0.00000
fedavg_default	47.0000	0.078805	0.972386	0.905405	0.000000	0.00000
cvar_0	7.0000	0.089309	0.963568	0.802469	-0.005572	-0.0864
cvar_0	11.0000	0.087877	0.969861	0.881579	0.000590	0.00000
cvar_0	19.0000	0.079776	0.974282	0.894737	0.001922	-0.0131
cvar_0	23.0000	0.081911	0.971662	0.907895	0.001470	0.02631
cvar_0	29.0000	0.082076	0.969563	0.905405	0.000000	0.00000
cvar_0	31.0000	0.084339	0.968746	0.888889	0.000000	0.00000
cvar_0	37.0000	0.105881	0.959307	0.851852	0.000000	0.00000
cvar_0	41.0000	0.074254	0.973141	0.906250	0.000000	0.00000
cvar_0	43.0000	0.080542	0.972976	0.894737	0.000000	0.00000
cvar_0	47.0000	0.078805	0.972386	0.905405	0.000000	0.00000
cvar_0.5	7.0000	0.089560	0.965354	0.790123	-0.003785	-0.0987
cvar_0.5	11.0000	0.091136	0.969393	0.881579	0.000122	0.00000
cvar_0.5	19.0000	0.080303	0.974633	0.894737	0.002273	-0.0131
cvar_0.5	23.0000	0.081984	0.972417	0.894737	0.002225	0.01315
cvar_0.5	29.0000	0.091316	0.966033	0.839506	-0.003530	-0.0658
cvar_0.5	31.0000	0.070820	0.976797	0.907895	0.008051	0.01900
cvar_0.5	37.0000	0.074752	0.971662	0.894737	0.012355	0.04288

method	seed	final_loss	final_accuracy	final_worst_client_accuracy	delta_accuracy	delta_w
cvar_0.5	41.0000	0.075580	0.972615	0.913043	-0.000526	0.00679
cvar_0.5	43.0000	0.122915	0.960438	0.842105	-0.012537	-0.0526
cvar_0.5	47.0000	0.081977	0.971886	0.881579	-0.000500	-0.0238
cvar_0.75	7.0000	0.089708	0.965740	0.790123	-0.003400	-0.0987
cvar_0.75	11.0000	0.091487	0.968954	0.868421	-0.000317	-0.0131
cvar_0.75	19.0000	0.080245	0.975084	0.894737	0.002723	-0.0131
cvar_0.75	23.0000	0.081734	0.971594	0.894737	0.001402	0.01315
cvar_0.75	29.0000	0.082332	0.967933	0.901235	-0.001631	-0.0041
cvar_0.75	31.0000	0.086776	0.967533	0.881579	-0.001213	-0.0073
cvar_0.75	37.0000	0.202530	0.936280	0.802469	-0.023026	-0.0493
cvar_0.75	41.0000	0.071455	0.976601	0.916667	0.003459	0.01041
cvar_0.75	43.0000	0.075664	0.973476	0.907895	0.000500	0.01315
cvar_0.75	47.0000	0.080894	0.971595	0.881579	-0.000792	-0.0238
cvar_0.9	7.0000	0.089223	0.965350	0.790123	-0.003789	-0.0987
cvar_0.9	11.0000	0.092229	0.968848	0.868421	-0.000423	-0.0131
cvar_0.9	19.0000	0.080607	0.972632	0.881579	0.000272	-0.0263
cvar_0.9	23.0000	0.087067	0.968234	0.876543	-0.001958	-0.0050
cvar_0.9	29.0000	0.071558	0.976433	0.907895	0.006870	0.00248
cvar_0.9	31.0000	0.081355	0.970171	0.894737	0.001425	0.00584
cvar_0.9	37.0000	0.074990	0.973946	0.913580	0.014639	0.06172
cvar_0.9	41.0000	0.070810	0.974483	0.927083	0.001341	0.02083
cvar_0.9	43.0000	0.069222	0.973787	0.901408	0.000811	0.00667
cvar_0.9	47.0000	0.104640	0.962809	0.802469	-0.009577	-0.1029
cvar_0.95	7.0000	0.085141	0.966808	0.814815	-0.002331	-0.0740
cvar_0.95	11.0000	0.092229	0.968848	0.868421	-0.000423	-0.0131
cvar_0.95	19.0000	0.080607	0.972632	0.881579	0.000272	-0.0263
cvar_0.95	23.0000	0.087067	0.968234	0.876543	-0.001958	-0.0050
cvar_0.95	29.0000	0.071558	0.976433	0.907895	0.006870	0.00248
cvar_0.95	31.0000	0.081355	0.970171	0.894737	0.001425	0.00584
cvar_0.95	37.0000	0.074990	0.973946	0.913580	0.014639	0.06172
cvar_0.95	41.0000	0.070810	0.974483	0.927083	0.001341	0.02083
cvar_0.95	43.0000	0.069222	0.973787	0.901408	0.000811	0.00667
cvar_0.95	47.0000	0.104640	0.962809	0.802469	-0.009577	-0.1029
centralized	7.0000	0.062550	0.986646	0.948052	0.017506	0.05916
centralized	11.0000	0.056431	0.986111	0.951456	0.016840	0.06987
centralized	19.0000	0.055696	0.986057	0.934783	0.013697	0.02688
centralized	23.0000	0.066330	0.986763	0.941748	0.016571	0.06016
centralized	29.0000	0.074862	0.980479	0.937500	0.010915	0.03209
centralized	31.0000	0.066689	0.982090	0.918919	0.013344	0.03003
centralized	37.0000	0.060545	0.984695	0.935065	0.025388	0.08321
centralized	41.0000	0.061730	0.984193	0.941748	0.011052	0.03549
centralized	43.0000	0.056892	0.984968	0.945946	0.011992	0.05120
centralized	47.0000	0.059397	0.985030	0.951456	0.012644	0.04605
local_only	7.0000	0.041006	0.987801	0.944444	0.018662	0.05555
grid_best_validated	7.0000	0.096162	0.970334	0.839506	0.001194	-0.0493
grid_best_validated	11.0000	0.079148	0.974990	0.918919	0.005719	0.03734

method	seed	final_loss	final_accuracy	final_worst_client_accuracy	delta_accuracy	delta_w
grid_best_validated	19.0000	0.077124	0.980446	0.923913	0.008085	0.01601
grid_best_validated	23.0000	0.090883	0.970979	0.894737	0.000787	0.01315
grid_best_validated	29.0000	0.106726	0.966304	0.765432	-0.003259	-0.1399
grid_best_validated	31.0000	0.098940	0.971908	0.868421	0.003163	-0.0204
grid_best_validated	37.0000	0.083818	0.973548	0.907895	0.014241	0.05604
grid_best_validated	41.0000	0.088985	0.971532	0.891892	-0.001609	-0.0143
grid_best_validated	43.0000	0.094574	0.970736	0.894737	-0.002239	0.00000
grid_best_validated	47.0000	0.079887	0.975026	0.921053	0.002640	0.01564
ga_best_validated	7.0000	0.096275	0.974182	0.894737	0.005042	0.00584
ga_best_validated	11.0000	0.086836	0.977261	0.916667	0.007990	0.03508
ga_best_validated	19.0000	0.140236	0.959203	0.790123	-0.013157	-0.1177
ga_best_validated	23.0000	0.127606	0.962116	0.842105	-0.008076	-0.0394
ga_best_validated	29.0000	0.113908	0.966215	0.839506	-0.003348	-0.0658
ga_best_validated	31.0000	0.133541	0.959675	0.828947	-0.009071	-0.0599
ga_best_validated	37.0000	0.194650	0.943192	0.592593	-0.016115	-0.2592
ga_best_validated	41.0000	0.084978	0.974219	0.894737	0.001077	-0.0115
ga_best_validated	43.0000	0.102485	0.967534	0.881579	-0.005442	-0.0131
ga_best_validated	47.0000	0.097456	0.970149	0.891892	-0.002237	-0.0135

Table 24.8: Communication efficiency.

method	seed	final_accuracy	final_worst_client_accuracy	comm_mb	accuracy_per_mb	wors
fedavg_default	7.0000	0.969140	0.888889	1192.6136	0.000813	0.00
fedavg_default	11.0000	0.969271	0.881579	1476.2622	0.000657	0.00
fedavg_default	19.0000	0.972361	0.907895	1792.1437	0.000543	0.00
fedavg_default	23.0000	0.970192	0.881579	1289.3120	0.000752	0.00
fedavg_default	29.0000	0.969563	0.905405	1508.4950	0.000643	0.00
fedavg_default	31.0000	0.968746	0.888889	1263.5258	0.000767	0.00
fedavg_default	37.0000	0.959307	0.851852	1456.9226	0.000658	0.00
fedavg_default	41.0000	0.973141	0.906250	1630.9797	0.000597	0.00
fedavg_default	43.0000	0.972976	0.894737	1508.4950	0.000645	0.00
fedavg_default	47.0000	0.972386	0.905405	1476.2622	0.000659	0.00
cvar_0	7.0000	0.963568	0.802469	1199.0602	0.000804	0.00
cvar_0	11.0000	0.969861	0.881579	1308.6517	0.000741	0.00
cvar_0	19.0000	0.974282	0.894737	1759.9109	0.000554	0.00
cvar_0	23.0000	0.971662	0.907895	1289.3120	0.000754	0.00
cvar_0	29.0000	0.969563	0.905405	1508.4950	0.000643	0.00
cvar_0	31.0000	0.968746	0.888889	1263.5258	0.000767	0.00
cvar_0	37.0000	0.959307	0.851852	1456.9226	0.000658	0.00
cvar_0	41.0000	0.973141	0.906250	1630.9797	0.000597	0.00
cvar_0	43.0000	0.972976	0.894737	1508.4950	0.000645	0.00
cvar_0	47.0000	0.972386	0.905405	1476.2622	0.000659	0.00
cvar_0.5	7.0000	0.965354	0.790123	1199.0602	0.000805	0.00
cvar_0.5	11.0000	0.969393	0.881579	1308.6517	0.000741	0.00
cvar_0.5	19.0000	0.974633	0.894737	1759.9109	0.000554	0.00

method	seed	final_accuracy	final_worst_client_accuracy	comm_mb	accuracy_per_mb	wors
cvar_0.5	23.0000	0.972417	0.894737	1289.3120	0.000754	0.00
cvar_0.5	29.0000	0.966033	0.839506	1166.8274	0.000828	0.00
cvar_0.5	31.0000	0.976797	0.907895	1998.4336	0.000489	0.00
cvar_0.5	37.0000	0.971662	0.894737	1656.7659	0.000586	0.00
cvar_0.5	41.0000	0.972615	0.913043	1637.4262	0.000594	0.00
cvar_0.5	43.0000	0.960438	0.842105	1095.9152	0.000876	0.00
cvar_0.5	47.0000	0.971886	0.881579	1476.2622	0.000658	0.00
cvar_0.75	7.0000	0.965740	0.790123	1199.0602	0.000805	0.00
cvar_0.75	11.0000	0.968954	0.868421	1308.6517	0.000740	0.00
cvar_0.75	19.0000	0.975084	0.894737	1759.9109	0.000554	0.00
cvar_0.75	23.0000	0.971594	0.894737	1289.3120	0.000754	0.00
cvar_0.75	29.0000	0.967933	0.901235	1508.4950	0.000642	0.00
cvar_0.75	31.0000	0.967533	0.881579	1205.5067	0.000803	0.00
cvar_0.75	37.0000	0.936280	0.802469	1456.9226	0.000643	0.00
cvar_0.75	41.0000	0.976601	0.916667	2062.8992	0.000473	0.00
cvar_0.75	43.0000	0.973476	0.907895	1630.9797	0.000597	0.00
cvar_0.75	47.0000	0.971595	0.881579	1476.2622	0.000658	0.00
cvar_0.9	7.0000	0.965350	0.790123	1199.0602	0.000805	0.00
cvar_0.9	11.0000	0.968848	0.868421	1308.6517	0.000740	0.00
cvar_0.9	19.0000	0.972632	0.881579	1682.5522	0.000578	0.00
cvar_0.9	23.0000	0.968234	0.876543	1269.9723	0.000762	0.00
cvar_0.9	29.0000	0.976433	0.907895	2030.6664	0.000481	0.00
cvar_0.9	31.0000	0.970171	0.894737	1566.5141	0.000619	0.00
cvar_0.9	37.0000	0.973946	0.913580	1656.7659	0.000588	0.00
cvar_0.9	41.0000	0.974483	0.927083	1863.0558	0.000523	0.00
cvar_0.9	43.0000	0.973787	0.901408	2153.1510	0.000452	0.00
cvar_0.9	47.0000	0.962809	0.802469	844.4994	0.001140	0.00
cvar_0.95	7.0000	0.966808	0.814815	1199.0602	0.000806	0.00
cvar_0.95	11.0000	0.968848	0.868421	1308.6517	0.000740	0.00
cvar_0.95	19.0000	0.972632	0.881579	1682.5522	0.000578	0.00
cvar_0.95	23.0000	0.968234	0.876543	1269.9723	0.000762	0.00
cvar_0.95	29.0000	0.976433	0.907895	2030.6664	0.000481	0.00
cvar_0.95	31.0000	0.970171	0.894737	1566.5141	0.000619	0.00
cvar_0.95	37.0000	0.973946	0.913580	1656.7659	0.000588	0.00
cvar_0.95	41.0000	0.974483	0.927083	1863.0558	0.000523	0.00
cvar_0.95	43.0000	0.973787	0.901408	2153.1510	0.000452	0.00
cvar_0.95	47.0000	0.962809	0.802469	844.4994	0.001140	0.00
centralized	7.0000	0.986646	0.948052	0.000000	0.000000	0.00
centralized	11.0000	0.986111	0.951456	0.000000	0.000000	0.00
centralized	19.0000	0.986057	0.934783	0.000000	0.000000	0.00
centralized	23.0000	0.986763	0.941748	0.000000	0.000000	0.00
centralized	29.0000	0.980479	0.937500	0.000000	0.000000	0.00
centralized	31.0000	0.982090	0.918919	0.000000	0.000000	0.00
centralized	37.0000	0.984695	0.935065	0.000000	0.000000	0.00
centralized	41.0000	0.984193	0.941748	0.000000	0.000000	0.00
centralized	43.0000	0.984968	0.945946	0.000000	0.000000	0.00

method	seed	final_accuracy	final_worst_client_accuracy	comm_mb	accuracy_per_mb	wor
centralized	47.0000	0.985030	0.951456	0.000000	0.000000	0.00
local_only	7.0000	0.987801	0.944444	0.000000	0.000000	0.00
grid_best_validated	7.0000	0.970334	0.839506	1199.0602	0.000809	0.00
grid_best_validated	11.0000	0.974990	0.918919	1734.1246	0.000562	0.00
grid_best_validated	19.0000	0.980446	0.923913	2075.7923	0.000472	0.00
grid_best_validated	23.0000	0.970979	0.894737	1289.3120	0.000753	0.00
grid_best_validated	29.0000	0.966304	0.765432	1160.3808	0.000833	0.00
grid_best_validated	31.0000	0.971908	0.868421	1927.5214	0.000504	0.00
grid_best_validated	37.0000	0.973548	0.907895	1534.2813	0.000635	0.00
grid_best_validated	41.0000	0.971532	0.891892	1630.9797	0.000596	0.00
grid_best_validated	43.0000	0.970736	0.894737	1095.9152	0.000886	0.00
grid_best_validated	47.0000	0.975026	0.921053	1972.6474	0.000494	0.00
ga_best_validated	7.0000	0.974182	0.894737	495.0958	0.001968	0.00
ga_best_validated	11.0000	0.977261	0.916667	580.1904	0.001684	0.00
ga_best_validated	19.0000	0.959203	0.790123	170.1892	0.005636	0.00
ga_best_validated	23.0000	0.962116	0.842105	257.8624	0.003731	0.00
ga_best_validated	29.0000	0.966215	0.839506	255.2838	0.003785	0.00
ga_best_validated	31.0000	0.959675	0.828947	237.2334	0.004045	0.00
ga_best_validated	37.0000	0.943192	0.592593	165.0319	0.005715	0.00
ga_best_validated	41.0000	0.974219	0.894737	533.7752	0.001825	0.00
ga_best_validated	43.0000	0.967534	0.881579	389.3722	0.002485	0.00
ga_best_validated	47.0000	0.970149	0.891892	389.3722	0.002492	0.00

Table 24.9: Failure-mode records.

method	seed	failure_type	description
ga_best_validated	37.0000	low_worst_client_accuracy	Worst-client accuracy remains below 0.70.

Chapter 25

Round-Level Trace Example

The table below gives a round-by-round trace for default FedAvg with seed 7. It proves that the final summary is not a single isolated measurement. It is built from a full sequence of evaluated communication rounds, each with loss, accuracy, worst-client accuracy, communication bytes, and early-stopping state.

Table 25.1: Round-level FedAvg trace for seed 7.

round	loss	accuracy	worst_client_accuracy	total_comm_bytes	best_loss_so_far	best_round	rour
1.0000	1.5446	0.516425	0.395062	6.447e+06	1.5446	1.0000	0.00
2.0000	1.1884	0.594430	0.444444	6.447e+06	1.1884	2.0000	0.00
3.0000	0.888710	0.712524	0.481481	6.447e+06	0.888710	3.0000	0.00
4.0000	0.701841	0.755426	0.592593	6.447e+06	0.701841	4.0000	0.00
5.0000	0.568953	0.821767	0.543210	6.447e+06	0.568953	5.0000	0.00
6.0000	0.479059	0.849367	0.660194	6.447e+06	0.479059	6.0000	0.00
7.0000	0.423216	0.854423	0.604938	6.447e+06	0.423216	7.0000	0.00
8.0000	0.391538	0.851458	0.555556	6.447e+06	0.391538	8.0000	0.00
9.0000	0.351517	0.874937	0.641975	6.447e+06	0.351517	9.0000	0.00
10.0000	0.332266	0.876936	0.617284	6.447e+06	0.332266	10.0000	0.00
11.0000	0.313855	0.887043	0.629630	6.447e+06	0.313855	11.0000	0.00
12.0000	0.291055	0.890327	0.555556	6.447e+06	0.291055	12.0000	0.00
13.0000	0.278879	0.896238	0.567901	6.447e+06	0.278879	13.0000	0.00
14.0000	0.257498	0.907337	0.617284	6.447e+06	0.257498	14.0000	0.00
15.0000	0.253728	0.903820	0.629630	6.447e+06	0.253728	15.0000	0.00
16.0000	0.236911	0.918466	0.716049	6.447e+06	0.236911	16.0000	0.00
17.0000	0.228381	0.917379	0.629630	6.447e+06	0.228381	17.0000	0.00
18.0000	0.224330	0.910930	0.604938	6.447e+06	0.224330	18.0000	0.00
19.0000	0.212467	0.922602	0.740741	6.447e+06	0.212467	19.0000	0.00
20.0000	0.208987	0.924726	0.740741	6.447e+06	0.208987	20.0000	0.00
21.0000	0.198558	0.924814	0.740741	6.447e+06	0.198558	21.0000	0.00
22.0000	0.214136	0.914847	0.691358	6.447e+06	0.198558	21.0000	1.00
23.0000	0.191267	0.931260	0.679012	6.447e+06	0.191267	23.0000	0.00
24.0000	0.190662	0.928724	0.691358	6.447e+06	0.191267	23.0000	1.00
25.0000	0.183138	0.929362	0.654321	6.447e+06	0.183138	25.0000	0.00
26.0000	0.180784	0.933169	0.679012	6.447e+06	0.180784	26.0000	0.00

round	loss	accuracy	worst_client_accuracy	total_comm_bytes	best_loss_so_far	best_round	rour
27.0000	0.179377	0.935094	0.740741	6.447e+06	0.179377	27.0000	0.00
28.0000	0.189476	0.928255	0.703704	6.447e+06	0.179377	27.0000	1.00
29.0000	0.178143	0.934556	0.728395	6.447e+06	0.178143	29.0000	0.00
30.0000	0.186075	0.928403	0.617284	6.447e+06	0.178143	29.0000	1.00
31.0000	0.170535	0.934504	0.679012	6.447e+06	0.170535	31.0000	0.00
32.0000	0.178520	0.930778	0.641975	6.447e+06	0.170535	31.0000	1.00
33.0000	0.168631	0.933997	0.666667	6.447e+06	0.168631	33.0000	0.00
34.0000	0.167520	0.933560	0.691358	6.447e+06	0.167520	34.0000	0.00
35.0000	0.175153	0.933173	0.654321	6.447e+06	0.167520	34.0000	1.00
36.0000	0.165129	0.936600	0.666667	6.447e+06	0.165129	36.0000	0.00
37.0000	0.162566	0.938145	0.654321	6.447e+06	0.162566	37.0000	0.00
38.0000	0.158312	0.938238	0.654321	6.447e+06	0.158312	38.0000	0.00
39.0000	0.162805	0.938278	0.777778	6.447e+06	0.158312	38.0000	1.00
40.0000	0.151182	0.941569	0.716049	6.447e+06	0.151182	40.0000	0.00
41.0000	0.152679	0.944279	0.703704	6.447e+06	0.151182	40.0000	1.00
42.0000	0.152189	0.944490	0.703704	6.447e+06	0.151182	40.0000	2.00
43.0000	0.155710	0.941231	0.666667	6.447e+06	0.151182	40.0000	3.00
44.0000	0.151891	0.942979	0.703704	6.447e+06	0.151182	40.0000	4.00
45.0000	0.150117	0.944008	0.703704	6.447e+06	0.150117	45.0000	0.00
46.0000	0.151463	0.943223	0.679012	6.447e+06	0.150117	45.0000	1.00
47.0000	0.147773	0.942752	0.691358	6.447e+06	0.147773	47.0000	0.00
48.0000	0.147917	0.944373	0.703704	6.447e+06	0.147773	47.0000	1.00
49.0000	0.151183	0.942640	0.641975	6.447e+06	0.147773	47.0000	2.00
50.0000	0.138333	0.948459	0.740741	6.447e+06	0.138333	50.0000	0.00
51.0000	0.144377	0.942872	0.740741	6.447e+06	0.138333	50.0000	1.00
52.0000	0.139575	0.948656	0.777778	6.447e+06	0.138333	50.0000	2.00
53.0000	0.146489	0.945628	0.679012	6.447e+06	0.138333	50.0000	3.00
54.0000	0.142318	0.947462	0.740741	6.447e+06	0.138333	50.0000	4.00
55.0000	0.142063	0.942161	0.753086	6.447e+06	0.138333	50.0000	5.00
56.0000	0.135074	0.949488	0.753086	6.447e+06	0.135074	56.0000	0.00
57.0000	0.131690	0.949502	0.765432	6.447e+06	0.131690	57.0000	0.00
58.0000	0.139557	0.943861	0.740741	6.447e+06	0.131690	57.0000	1.00
59.0000	0.132168	0.949592	0.679012	6.447e+06	0.131690	57.0000	2.00
60.0000	0.130383	0.948437	0.691358	6.447e+06	0.130383	60.0000	0.00
61.0000	0.127518	0.951110	0.740741	6.447e+06	0.127518	61.0000	0.00
62.0000	0.129664	0.947296	0.790123	6.447e+06	0.127518	61.0000	1.00
63.0000	0.132215	0.946771	0.740741	6.447e+06	0.127518	61.0000	2.00
64.0000	0.126530	0.949565	0.740741	6.447e+06	0.127518	61.0000	3.00
65.0000	0.124369	0.952464	0.716049	6.447e+06	0.124369	65.0000	0.00
66.0000	0.127178	0.949704	0.691358	6.447e+06	0.124369	65.0000	1.00
67.0000	0.124532	0.952094	0.703704	6.447e+06	0.124369	65.0000	2.00
68.0000	0.123620	0.952639	0.728395	6.447e+06	0.124369	65.0000	3.00
69.0000	0.129120	0.950933	0.716049	6.447e+06	0.124369	65.0000	4.00
70.0000	0.121550	0.952971	0.728395	6.447e+06	0.121550	70.0000	0.00
71.0000	0.125911	0.947781	0.753086	6.447e+06	0.121550	70.0000	1.00
72.0000	0.118015	0.955602	0.740741	6.447e+06	0.118015	72.0000	0.00

round	loss	accuracy	worst_client_accuracy	total_comm_bytes	best_loss_so_far	best_round	rour
73.0000	0.125425	0.953622	0.716049	6.447e+06	0.118015	72.0000	1.00
74.0000	0.153852	0.940313	0.654321	6.447e+06	0.118015	72.0000	2.00
75.0000	0.119530	0.952750	0.777778	6.447e+06	0.118015	72.0000	3.00
76.0000	0.118927	0.953516	0.753086	6.447e+06	0.118015	72.0000	4.00
77.0000	0.119474	0.953563	0.716049	6.447e+06	0.118015	72.0000	5.00
78.0000	0.122749	0.952148	0.728395	6.447e+06	0.118015	72.0000	6.00
79.0000	0.121218	0.954021	0.753086	6.447e+06	0.118015	72.0000	7.00
80.0000	0.126563	0.953945	0.679012	6.447e+06	0.118015	72.0000	8.00
81.0000	0.137847	0.947354	0.629630	6.447e+06	0.118015	72.0000	9.00
82.0000	0.121249	0.954249	0.703704	6.447e+06	0.118015	72.0000	10.0
83.0000	0.123278	0.954068	0.716049	6.447e+06	0.118015	72.0000	11.0
84.0000	0.122803	0.955426	0.691358	6.447e+06	0.118015	72.0000	12.0
85.0000	0.121271	0.956018	0.691358	6.447e+06	0.118015	72.0000	13.0
86.0000	0.129215	0.952476	0.654321	6.447e+06	0.118015	72.0000	14.0
87.0000	0.115867	0.957606	0.765432	6.447e+06	0.115867	87.0000	0.00
88.0000	0.120925	0.954798	0.691358	6.447e+06	0.115867	87.0000	1.00
89.0000	0.117927	0.956730	0.753086	6.447e+06	0.115867	87.0000	2.00
90.0000	0.109408	0.957895	0.753086	6.447e+06	0.109408	90.0000	0.00
91.0000	0.120892	0.954455	0.827160	6.447e+06	0.109408	90.0000	1.00
92.0000	0.126993	0.952452	0.827160	6.447e+06	0.109408	90.0000	2.00
93.0000	0.108448	0.958245	0.814815	6.447e+06	0.109408	90.0000	3.00
94.0000	0.108530	0.958567	0.765432	6.447e+06	0.109408	90.0000	4.00
95.0000	0.108757	0.958960	0.740741	6.447e+06	0.109408	90.0000	5.00
96.0000	0.106900	0.959431	0.753086	6.447e+06	0.106900	96.0000	0.00
97.0000	0.106955	0.957405	0.790123	6.447e+06	0.106900	96.0000	1.00
98.0000	0.104947	0.959531	0.753086	6.447e+06	0.104947	98.0000	0.00
99.0000	0.105319	0.960935	0.827160	6.447e+06	0.104947	98.0000	1.00
100.0000	0.111647	0.959424	0.740741	6.447e+06	0.104947	98.0000	2.00
101.0000	0.102652	0.959802	0.765432	6.447e+06	0.102652	101.0000	0.00
102.0000	0.099668	0.961105	0.839506	6.447e+06	0.099668	102.0000	0.00
103.0000	0.110819	0.956505	0.740741	6.447e+06	0.099668	102.0000	1.00
104.0000	0.100479	0.959451	0.777778	6.447e+06	0.099668	102.0000	2.00
105.0000	0.106305	0.958724	0.728395	6.447e+06	0.099668	102.0000	3.00
106.0000	0.099796	0.960843	0.777778	6.447e+06	0.099668	102.0000	4.00
107.0000	0.101597	0.961252	0.753086	6.447e+06	0.099668	102.0000	5.00
108.0000	0.100731	0.959612	0.765432	6.447e+06	0.099668	102.0000	6.00
109.0000	0.108864	0.960246	0.855263	6.447e+06	0.099668	102.0000	7.00
110.0000	0.098216	0.963764	0.851852	6.447e+06	0.098216	110.0000	0.00
111.0000	0.101429	0.963195	0.881579	6.447e+06	0.098216	110.0000	1.00
112.0000	0.097667	0.961954	0.827160	6.447e+06	0.098216	110.0000	2.00
113.0000	0.097552	0.962083	0.765432	6.447e+06	0.098216	110.0000	3.00
114.0000	0.094482	0.966092	0.839506	6.447e+06	0.094482	114.0000	0.00
115.0000	0.095440	0.966086	0.876543	6.447e+06	0.094482	114.0000	1.00
116.0000	0.093864	0.966372	0.851852	6.447e+06	0.094482	114.0000	2.00
117.0000	0.101797	0.958718	0.765432	6.447e+06	0.094482	114.0000	3.00
118.0000	0.099526	0.961100	0.753086	6.447e+06	0.094482	114.0000	4.00

round	loss	accuracy	worst_client_accuracy	total_comm_bytes	best_loss_so_far	best_round	rour
119.0000	0.092836	0.964659	0.839506	6.447e+06	0.092836	119.0000	0.00
120.0000	0.092412	0.964612	0.839506	6.447e+06	0.092836	119.0000	1.00
121.0000	0.094371	0.965382	0.864198	6.447e+06	0.092836	119.0000	2.00
122.0000	0.111968	0.956762	0.814815	6.447e+06	0.092836	119.0000	3.00
123.0000	0.096610	0.963583	0.864198	6.447e+06	0.092836	119.0000	4.00
124.0000	0.096360	0.964045	0.839506	6.447e+06	0.092836	119.0000	5.00
125.0000	0.099284	0.962654	0.864198	6.447e+06	0.092836	119.0000	6.00
126.0000	0.094920	0.962374	0.814815	6.447e+06	0.092836	119.0000	7.00
127.0000	0.096265	0.961927	0.851852	6.447e+06	0.092836	119.0000	8.00
128.0000	0.094637	0.964359	0.881579	6.447e+06	0.092836	119.0000	9.00
129.0000	0.092650	0.966639	0.881579	6.447e+06	0.092836	119.0000	10.0
130.0000	0.088831	0.967785	0.895833	6.447e+06	0.088831	130.0000	0.00
131.0000	0.093825	0.963994	0.864198	6.447e+06	0.088831	130.0000	1.00
132.0000	0.091279	0.966000	0.814815	6.447e+06	0.088831	130.0000	2.00
133.0000	0.097663	0.960521	0.728395	6.447e+06	0.088831	130.0000	3.00
134.0000	0.091185	0.965187	0.814815	6.447e+06	0.088831	130.0000	4.00
135.0000	0.094000	0.960875	0.753086	6.447e+06	0.088831	130.0000	5.00
136.0000	0.101068	0.960338	0.765432	6.447e+06	0.088831	130.0000	6.00
137.0000	0.094166	0.964533	0.827160	6.447e+06	0.088831	130.0000	7.00
138.0000	0.092632	0.965387	0.827160	6.447e+06	0.088831	130.0000	8.00
139.0000	0.104666	0.964439	0.881579	6.447e+06	0.088831	130.0000	9.00
140.0000	0.092235	0.967060	0.864198	6.447e+06	0.088831	130.0000	10.0
141.0000	0.091385	0.967069	0.881579	6.447e+06	0.088831	130.0000	11.0
142.0000	0.089528	0.966218	0.864198	6.447e+06	0.088831	130.0000	12.0
143.0000	0.089568	0.966189	0.864198	6.447e+06	0.088831	130.0000	13.0
144.0000	0.088481	0.967077	0.864198	6.447e+06	0.088831	130.0000	14.0
145.0000	0.091788	0.962666	0.777778	6.447e+06	0.088831	130.0000	15.0
146.0000	0.093839	0.963633	0.839506	6.447e+06	0.088831	130.0000	16.0
147.0000	0.092665	0.962167	0.765432	6.447e+06	0.088831	130.0000	17.0
148.0000	0.095199	0.963261	0.765432	6.447e+06	0.088831	130.0000	18.0
149.0000	0.096380	0.962854	0.827160	6.447e+06	0.088831	130.0000	19.0
150.0000	0.090284	0.964326	0.814815	6.447e+06	0.088831	130.0000	20.0
151.0000	0.091434	0.963531	0.777778	6.447e+06	0.088831	130.0000	21.0
152.0000	0.088447	0.967144	0.851852	6.447e+06	0.088831	130.0000	22.0
153.0000	0.090295	0.966940	0.881579	6.447e+06	0.088831	130.0000	23.0
154.0000	0.085437	0.965165	0.851852	6.447e+06	0.085437	154.0000	0.00
155.0000	0.084085	0.966282	0.851852	6.447e+06	0.084085	155.0000	0.00
156.0000	0.083940	0.969245	0.876543	6.447e+06	0.084085	155.0000	1.00
157.0000	0.085769	0.967051	0.888889	6.447e+06	0.084085	155.0000	2.00
158.0000	0.085538	0.968231	0.881579	6.447e+06	0.084085	155.0000	3.00
159.0000	0.085716	0.966688	0.876543	6.447e+06	0.084085	155.0000	4.00
160.0000	0.096250	0.961188	0.777778	6.447e+06	0.084085	155.0000	5.00
161.0000	0.085542	0.965770	0.839506	6.447e+06	0.084085	155.0000	6.00
162.0000	0.086987	0.967985	0.851852	6.447e+06	0.084085	155.0000	7.00
163.0000	0.089136	0.965327	0.827160	6.447e+06	0.084085	155.0000	8.00
164.0000	0.090710	0.963766	0.790123	6.447e+06	0.084085	155.0000	9.00

round	loss	accuracy	worst_client_accuracy	total_comm_bytes	best_loss_so_far	best_round	rour
165.0000	0.095898	0.962106	0.790123	6.447e+06	0.084085	155.0000	10.0
166.0000	0.094198	0.964426	0.753086	6.447e+06	0.084085	155.0000	11.0
167.0000	0.089300	0.966072	0.827160	6.447e+06	0.084085	155.0000	12.0
168.0000	0.092153	0.964831	0.790123	6.447e+06	0.084085	155.0000	13.0
169.0000	0.088304	0.968121	0.876543	6.447e+06	0.084085	155.0000	14.0
170.0000	0.140260	0.956152	0.855263	6.447e+06	0.084085	155.0000	15.0
171.0000	0.090282	0.968044	0.876543	6.447e+06	0.084085	155.0000	16.0
172.0000	0.092682	0.963081	0.765432	6.447e+06	0.084085	155.0000	17.0
173.0000	0.088498	0.965593	0.777778	6.447e+06	0.084085	155.0000	18.0
174.0000	0.088659	0.965989	0.827160	6.447e+06	0.084085	155.0000	19.0
175.0000	0.088018	0.967225	0.827160	6.447e+06	0.084085	155.0000	20.0
176.0000	0.088977	0.967205	0.827160	6.447e+06	0.084085	155.0000	21.0
177.0000	0.093289	0.967589	0.891304	6.447e+06	0.084085	155.0000	22.0
178.0000	0.092122	0.967489	0.894737	6.447e+06	0.084085	155.0000	23.0
179.0000	0.085927	0.969831	0.894737	6.447e+06	0.084085	155.0000	24.0
180.0000	0.085159	0.967821	0.851852	6.447e+06	0.084085	155.0000	25.0
181.0000	0.085552	0.969394	0.876543	6.447e+06	0.084085	155.0000	26.0
182.0000	0.090025	0.967314	0.894737	6.447e+06	0.084085	155.0000	27.0
183.0000	0.084698	0.966514	0.839506	6.447e+06	0.084085	155.0000	28.0
184.0000	0.084265	0.970185	0.894737	6.447e+06	0.084085	155.0000	29.0
185.0000	0.084216	0.969140	0.888889	6.447e+06	0.084085	155.0000	30.0

Appendix A

Source Code Appendix

This appendix includes the source code used to produce the results. Code listings are part of the evidence. They show exactly how the dataset was loaded, how clients were built, how FedAvg was trained, how metrics were computed, how LP duality was solved, and how the full v2 experiment was orchestrated.

A.1 flopt/config.py

```
from dataclasses import dataclass


@dataclass(frozen=True)
class FLConfig:
    rounds:int=20
    max_rounds:int|None=None
    local_epochs:int=2
    clients_per_round:int=10
    lr:float=0.01
    batch_size:int=32
    seed:int=7
    cvar_alpha:float=0.0
    fairness_strength:float=1.0
    patience:int=30
    min_delta:float=0.001
    early_stopping:bool=False
    monitor:str="loss"
```

A.2 flopt/data.py

```
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from urllib.request import urlretrieve
import zipfile

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


UCI_URL="https://archive.ics.uci.edu/static/public/240/human+activity+recognition+using+
    smartphones.zip"
```

```python


@dataclass
class ClientData:
    x_train:np.ndarray
    y_train:np.ndarray
    x_test:np.ndarray
    y_test:np.ndarray
    client_id:int|None=None


def load_clients(source:str="uci",data_dir:str="data",seed:int=7)->list[ClientData]:
    if source=="uci":
        return load_uci_har(Path(data_dir),seed).clients
    raise ValueError(f"unknown source: {source}")


@dataclass
class UCIBundle:
    clients:list[ClientData]
    scaler:StandardScaler
    activity_names:list[str]
    root:Path
    x_train_raw:np.ndarray
    y_train:np.ndarray
    subject_train:np.ndarray
    x_test_raw:np.ndarray
    y_test:np.ndarray
    subject_test:np.ndarray


def load_uci_har(data_dir:Path,seed:int=7)->UCIBundle:
    root=_ensure_uci_har(data_dir)
    x_train_raw=np.loadtxt(root/"train"/"X_train.txt",dtype="float32")
    y_train=np.loadtxt(root/"train"/"y_train.txt",dtype="int64")-1
    s_train=np.loadtxt(root/"train"/"subject_train.txt",dtype="int64")
    x_test_raw=np.loadtxt(root/"test"/"X_test.txt",dtype="float32")
    y_test=np.loadtxt(root/"test"/"y_test.txt",dtype="int64")-1
    s_test=np.loadtxt(root/"test"/"subject_test.txt",dtype="int64")
    labels=np.loadtxt(root/"activity_labels.txt",dtype=str)
    activity_names=[name for _,name in sorted((int(idx),name) for idx,name in labels)]
    scaler=StandardScaler().fit(x_train_raw)
    x_train=scaler.transform(x_train_raw).astype("float32")
    x_test=scaler.transform(x_test_raw).astype("float32")
    clients=[]
    for subject in sorted(set(s_train)|set(s_test)):
        tr=s_train==subject
        te=s_test==subject
        if tr.sum()==0 or te.sum()==0:
            x=np.concatenate([x_train[tr],x_test[te]])
            y=np.concatenate([y_train[tr],y_test[te]])
            xtr,xte,ytr,yte=train_test_split(x,y,test_size=0.25,random_state=seed+int(subject))
        else:
            xtr,xte,ytr,yte=x_train[tr],x_test[te],y_train[tr],y_test[te]
        clients.append(ClientData(xtr,ytr,xte,yte,int(subject)))
    return UCIBundle(clients,scaler,activity_names,root,x_train_raw,y_train,s_train,x_test_raw,y_test,s_test)


def _ensure_uci_har(data_dir:Path)->Path:
    root=data_dir/"UCI HAR Dataset"
    if root.exists():
        return root
    data_dir.mkdir(parents=True,exist_ok=True)
    zip_path=data_dir/"uci_har.zip"
    if not zip_path.exists():
        urlretrieve(UCI_URL,zip_path)
```

```
    with zipfile.ZipFile(zip_path) as zf:
        zf.extractall(data_dir)
    nested=data_dir/"UCI HAR Dataset.zip"
    if nested.exists() and not root.exists():
        with zipfile.ZipFile(nested) as zf:
            zf.extractall(data_dir)
    return root
```

A.3 flopt/models.py

```
import torch
from torch import nn


class HARMLP(nn.Module):
    def __init__(self,features:int=561,classes:int=6):
        super().__init__()
        self.net=nn.Sequential(
            nn.Linear(features,128),
            nn.ReLU(),
            nn.Linear(128,64),
            nn.ReLU(),
            nn.Linear(64,classes),
        )

    def forward(self,x:torch.Tensor)->torch.Tensor:
        return self.net(x)


class LogisticModel(nn.Module):
    def __init__(self,features:int=561,classes:int=6):
        super().__init__()
        self.linear=nn.Linear(features,classes)

    def forward(self,x:torch.Tensor)->torch.Tensor:
        return self.linear(x)


def count_parameters(model:nn.Module)->int:
    return sum(p.numel() for p in model.parameters())


def clone_model(model:nn.Module)->nn.Module:
    clone=type(model)()
    clone.load_state_dict(model.state_dict())
    return clone
```

A.4 flopt/fedavg.py

```
from __future__ import annotations

import random
from copy import deepcopy
from dataclasses import asdict

import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader,TensorDataset

from .config import FLConfig
```

```python
from .data import ClientData
from .models import count_parameters


def federated_train(model:nn.Module,clients:list[ClientData],cfg:FLConfig,track_drift:bool=False)->tuple[nn.Module,list[dict]]:
    _set_seed(cfg.seed)
    device=_device()
    global_model=deepcopy(model).to(device)
    records=[]
    client_ids=list(range(len(clients)))
    max_rounds=cfg.max_rounds or cfg.rounds
    best_value=float("inf")
    best_round=0
    stale_rounds=0
    best_state=None
    for round_id in range(1,max_rounds+1):
        selected=random.sample(client_ids,min(cfg.clients_per_round,len(client_ids)))
        local_states=[]
        local_sizes=[]
        local_losses=[]
        base_state={k:v.detach().cpu().clone() for k,v in global_model.state_dict().items()}
        for cid in selected:
            local_model=deepcopy(global_model)
            loss=train_one_client(local_model,clients[cid],cfg,device)
            local_states.append({k:v.detach().cpu() for k,v in local_model.state_dict().items()})
            local_sizes.append(len(clients[cid].x_train))
            local_losses.append(loss)
        weights=_aggregation_weights(np.array(local_sizes),np.array(local_losses),cfg)
        drift=_drift_stats(base_state,local_states,selected,weights) if track_drift else {}
        _load_weighted_state(global_model,local_states,weights,device)
        metrics=evaluate_all(global_model,clients,device)
        current=float(metrics[cfg.monitor])
        improved=current<best_value-cfg.min_delta
        if improved:
            best_value=current
            best_round=round_id
            stale_rounds=0
            best_state={k:v.detach().cpu().clone() for k,v in global_model.state_dict().items()}
        else:
            stale_rounds+=1
        stopped=bool(cfg.early_stopping and stale_rounds>=cfg.patience)
        metrics.update({
            "round":round_id,
            "selected_clients":selected,
            "upload_bytes":count_parameters(global_model)*4*len(selected),
            "download_bytes":count_parameters(global_model)*4*len(selected),
            "config":asdict(cfg),
            "best_loss_so_far":best_value,
            "best_round":best_round,
            "rounds_since_improvement":stale_rounds,
            "stopped_early":stopped,
            **drift,
        })
        records.append(metrics)
        if stopped:
            break
    if best_state is not None:
        global_model.load_state_dict({k:v.to(device) for k,v in best_state.items()})
    return global_model,records


def train_one_client(model:nn.Module,client:ClientData,cfg:FLConfig,device:torch.device)->float:
    model.train()
    x=torch.tensor(client.x_train,dtype=torch.float32)
```

```
    y=torch.tensor(client.y_train,dtype=torch.long)
    loader=DataLoader(TensorDataset(x,y),batch_size=cfg.batch_size,shuffle=True)
    opt=torch.optim.SGD(model.parameters(),lr=cfg.lr)
    loss_fn=nn.CrossEntropyLoss()
    last_loss=0.0
    for _ in range(cfg.local_epochs):
        for xb,yb in loader:
            xb=xb.to(device)
            yb=yb.to(device)
            opt.zero_grad()
            loss=loss_fn(model(xb),yb)
            loss.backward()
            opt.step()
            last_loss=float(loss.detach().cpu())
    return last_loss


@torch.no_grad()
def evaluate_all(model:nn.Module,clients:list[ClientData],device:torch.device)->dict:
    client_loss=[]
    client_acc=[]
    for client in clients:
        loss,acc=evaluate(model,client,device)
        client_loss.append(loss)
        client_acc.append(acc)
    return {
        "loss":float(np.mean(client_loss)),
        "accuracy":float(np.mean(client_acc)),
        "worst_client_accuracy":float(np.min(client_acc)),
        "client_loss":client_loss,
        "client_accuracy":client_acc,
    }


@torch.no_grad()
def evaluate(model:nn.Module,client:ClientData,device:torch.device)->tuple[float,float]:
    model.eval()
    x=torch.tensor(client.x_test,dtype=torch.float32,device=device)
    y=torch.tensor(client.y_test,dtype=torch.long,device=device)
    logits=model(x)
    loss=float(nn.CrossEntropyLoss()(logits,y).detach().cpu())
    acc=float((logits.argmax(1)==y).float().mean().detach().cpu())
    return loss,acc


@torch.no_grad()
def predict_clients(model:nn.Module,clients:list[ClientData],device:torch.device|None=None)
    ->list[dict]:
    device=device or _device()
    model=model.to(device)
    model.eval()
    rows=[]
    for idx,client in enumerate(clients):
        x=torch.tensor(client.x_test,dtype=torch.float32,device=device)
        y=torch.tensor(client.y_test,dtype=torch.long,device=device)
        logits=model(x)
        probs=torch.softmax(logits,dim=1)
        pred=probs.argmax(1)
        conf=probs.max(1).values
        cid=client.client_id if client.client_id is not None else idx
        for i in range(len(y)):
            rows.append({
                "client_id":int(cid),
                "row":int(i),
                "y_true":int(y[i].cpu()),
                "y_pred":int(pred[i].cpu()),
                "confidence":float(conf[i].cpu()),
                **{f"prob_{j}":float(probs[i,j].cpu()) for j in range(probs.shape[1])},
```

```
            })
    return rows


def _aggregation_weights(sizes:np.ndarray,losses:np.ndarray,cfg:FLConfig)->np.ndarray:
    weights=sizes.astype("float64")/sizes.sum()
    if cfg.cvar_alpha<=0:
        return weights
    tau=np.quantile(losses,cfg.cvar_alpha)
    tail=np.maximum(losses-tau,0)
    if tail.sum()==0:
        return weights
    # CVaR weighting keeps FedAvg's sample-size signal while lifting high-loss clients.
    weights=weights*(1+cfg.fairness_strength*tail/tail.sum())
    return weights/weights.sum()


def _load_weighted_state(model:nn.Module,states:list[dict],weights:np.ndarray,device:torch.device)->None:
    avg={}
    for key in states[0]:
        avg[key]=sum(weights[i]*states[i][key] for i in range(len(states))).to(device)
    model.load_state_dict(avg)


def _set_seed(seed:int)->None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)


def _device()->torch.device:
    if torch.backends.mps.is_available():
        return torch.device("mps")
    if torch.cuda.is_available():
        return torch.device("cuda")
    return torch.device("cpu")


def _drift_stats(base_state:dict,states:list[dict],selected:list[int],weights:np.ndarray)->dict:
    updates=[]
    norms=[]
    for state in states:
        parts=[(state[k]-base_state[k]).reshape(-1).float() for k in state]
        update=torch.cat(parts)
        updates.append(update)
        norms.append(float(torch.linalg.vector_norm(update)))
    avg=sum(float(weights[i])*updates[i] for i in range(len(updates)))
    avg_norm=float(torch.linalg.vector_norm(avg))
    cos=[]
    dist=[]
    for update in updates:
        denom=float(torch.linalg.vector_norm(update))*max(avg_norm,1e-12)
        cos.append(float(torch.dot(update,avg)/denom) if denom>0 else 0.0)
        dist.append(float(torch.linalg.vector_norm(update-avg)))
    return {
        "drift_client_ids":selected,
        "drift_update_norms":norms,
        "drift_cosine_to_mean":cos,
        "drift_distance_to_mean":dist,
        "drift_mean_update_norm":float(np.mean(norms)) if norms else 0.0,
        "drift_avg_update_norm":avg_norm,
    }
```

A.5 flopt/duality.py

```python
from __future__ import annotations

import numpy as np


def solve_policy_lp(losses:list[float],costs:list[float],budgets:list[float],rtol:float=1e-6,atol:float=1e-5)->list[dict]:
    import cvxpy as cp

    losses_np=np.array(losses,dtype="float64")
    costs_np=np.array(costs,dtype="float64")
    cost_scale=max(float(np.max(np.abs(costs_np))),1.0)
    scaled_costs=costs_np/cost_scale
    rows=[]
    for budget in budgets:
        scaled_budget=budget/cost_scale
        x=cp.Variable(len(losses_np),nonneg=True)
        simplex=sum(x)==1
        budget_constraint=scaled_costs@x<=scaled_budget
        problem=cp.Problem(cp.Minimize(losses_np@x),[simplex,budget_constraint])
        _solve(problem,cp)
        if problem.status not in {"optimal","optimal_inaccurate"}:
            rows.append({"budget":budget,"status":problem.status})
            continue
        weights=np.asarray(x.value).reshape(-1)
        lam=float(budget_constraint.dual_value)/cost_scale
        rows.append({
            "budget":float(budget),
            "loss":float(losses_np@weights),
            "cost":float(costs_np@weights),
            "lambda":lam,
            "weights":weights.tolist(),
            "kkt":_kkt(losses_np,costs_np,weights,budget,lam,float(simplex.dual_value),rtol,atol),
            "status":problem.status,
        })
    return rows


def _solve(problem,cp)->None:
    for solver in [cp.CLARABEL,cp.HIGHS,cp.OSQP,cp.SCS]:
        try:
            problem.solve(solver=solver,verbose=False)
            if problem.status in {"optimal","optimal_inaccurate"}:
                return
        except cp.SolverError:
            continue


def _kkt(losses:np.ndarray,costs:np.ndarray,x:np.ndarray,budget:float,lam:float,mu:float,rtol:float,atol:float)->dict:
    active=x>1e-6
    stationarity=losses[active]+lam*costs[active]+mu
    budget_slack=costs@x-budget
    budget_tol=atol+rtol*max(abs(float(budget)),abs(float(costs@x)),1.0)
    simplex_err=abs(x.sum()-1)
    comp=float(abs(lam*budget_slack))
    stat=float(np.max(np.abs(stationarity))) if stationarity.size else 0.0
    feasible=bool(simplex_err<=atol and budget_slack<=budget_tol and np.all(x>=-atol))
    dual=bool(lam>=-atol)
    near=bool((not feasible and budget_slack<=10*budget_tol) or stat<=1e-4 or comp<=1e-4)
    status="pass" if feasible and dual and comp<=1e-5 else "near_pass" if dual and near else "fail"
    return {
        "primal_feasible":feasible,
```

```
        "dual_feasible":dual,
        "complementary_slackness":comp,
        "stationarity_residual":stat,
        "budget_slack":float(budget_slack),
        "budget_tolerance":float(budget_tol),
        "kkt_status":status,
    }
```

A.6 flopt/search.py

```
from __future__ import annotations

from dataclasses import replace

import numpy as np
from scipy.optimize import differential_evolution

from .config import FLConfig
from .fedavg import federated_train
from .models import HARMLP


def ga_search(clients,base_cfg:FLConfig,bounds=None,maxiter:int=4,popsize:int=5,gamma:float=1e-8)->dict:
    if bounds is None:
        bounds=[(1,8),(3,min(20,len(clients))),(0.001,0.08)]
    history=[]

    def objective(raw):
        local_epochs=max(1,int(round(raw[0])))
        clients_per_round=max(1,int(round(raw[1])))
        lr=float(raw[2])
        cfg=replace(base_cfg,local_epochs=local_epochs,clients_per_round=clients_per_round,lr=lr)
        _,records=federated_train(HARMLP(),clients,cfg)
        last=records[-1]
        comm=sum(r["upload_bytes"]+r["download_bytes"] for r in records)
        penalty=max(0,0.80-last["accuracy"])*10
        fitness=last["loss"]+gamma*comm+penalty
        history.append({"evaluation":len(history)+1,"local_epochs":local_epochs,"clients_per_round":clients_per_round,"lr":lr,"loss":last["loss"],"accuracy":last["accuracy"],"comm":comm,"fitness":fitness})
        return fitness

    result=differential_evolution(objective,bounds,maxiter=maxiter,popsize=popsize,polish=False,seed=base_cfg.seed)
    return {"x":result.x.tolist(),"fitness":float(result.fun),"evaluations":int(result.nfev),"history":history}


def grid_search(clients,base_cfg:FLConfig,grid:list[tuple[int,int,float]],gamma:float=1e-8)->list[dict]:
    rows=[]
    for local_epochs,clients_per_round,lr in grid:
        cfg=replace(base_cfg,local_epochs=local_epochs,clients_per_round=clients_per_round,lr=lr)
        _,records=federated_train(HARMLP(),clients,cfg)
        last=records[-1]
        comm=sum(r["upload_bytes"]+r["download_bytes"] for r in records)
        rows.append({
            "local_epochs":local_epochs,
            "clients_per_round":clients_per_round,
            "lr":lr,
            "loss":last["loss"],
            "accuracy":last["accuracy"],
```

```
            "comm":comm,
            "fitness":last["loss"]+gamma*comm+max(0,0.80-last["accuracy"])*10,
        })
    return sorted(rows,key=lambda r:r["fitness"])
```

A.7 flopt/metrics.py

```
from __future__ import annotations

import numpy as np
from sklearn.metrics import classification_report,confusion_matrix,
    precision_recall_fscore_support

from .data import ClientData


def prediction_arrays(pred_rows:list[dict])->tuple[np.ndarray,np.ndarray,np.ndarray,np.
    ndarray]:
    y_true=np.array([r["y_true"] for r in pred_rows],dtype=int)
    y_pred=np.array([r["y_pred"] for r in pred_rows],dtype=int)
    conf=np.array([r["confidence"] for r in pred_rows],dtype=float)
    client=np.array([r["client_id"] for r in pred_rows],dtype=int)
    return y_true,y_pred,conf,client


def classification_rows(pred_rows:list[dict],activity_names:list[str])->list[dict]:
    y_true,y_pred,_,_=prediction_arrays(pred_rows)
    report=classification_report(y_true,y_pred,target_names=activity_names,output_dict=True,
        zero_division=0)
    rows=[]
    for label,vals in report.items():
        if isinstance(vals,dict):
            rows.append({"label":label,**{k:float(v) for k,v in vals.items()}})
        else:
            rows.append({"label":label,"score":float(vals)})
    return rows


def confusion_rows(pred_rows:list[dict],activity_names:list[str],normalize:bool=False)->list
    [dict]:
    y_true,y_pred,_,_=prediction_arrays(pred_rows)
    cm=confusion_matrix(y_true,y_pred,labels=list(range(len(activity_names))),normalize="
        true" if normalize else None)
    rows=[]
    for i,true_name in enumerate(activity_names):
        row={"true_label":true_name}
        for j,pred_name in enumerate(activity_names):
            row[pred_name]=float(cm[i,j])
        rows.append(row)
    return rows


def per_client_rows(pred_rows:list[dict],clients:list[ClientData])->list[dict]:
    y_true,y_pred,_,client_ids=prediction_arrays(pred_rows)
    rows=[]
    for idx,client in enumerate(clients):
        cid=client.client_id if client.client_id is not None else idx
        mask=client_ids==cid
        if not mask.any():
            continue
        rows.append({
            "client_id":int(cid),
            "test_samples":int(mask.sum()),
            "train_samples":int(len(client.x_train)),
            "accuracy":float((y_true[mask]==y_pred[mask]).mean()),
```

```
            "error_rate":float((y_true[mask]!=y_pred[mask]).mean()),
        })
    return rows


def aggregate_scores(pred_rows:list[dict])->dict:
    y_true,y_pred,_,_=prediction_arrays(pred_rows)
    precision,recall,f1,_=precision_recall_fscore_support(y_true,y_pred,average="macro",
        zero_division=0)
    w_precision,w_recall,w_f1,_=precision_recall_fscore_support(y_true,y_pred,average="
        weighted",zero_division=0)
    return {
        "accuracy":float((y_true==y_pred).mean()),
        "macro_precision":float(precision),
        "macro_recall":float(recall),
        "macro_f1":float(f1),
        "weighted_precision":float(w_precision),
        "weighted_recall":float(w_recall),
        "weighted_f1":float(w_f1),
    }


def top_confusions(pred_rows:list[dict],activity_names:list[str],limit:int=10)->list[dict]:
    y_true,y_pred,_,_=prediction_arrays(pred_rows)
    rows=[]
    for i,name_i in enumerate(activity_names):
        for j,name_j in enumerate(activity_names):
            if i==j:
                continue
            count=int(((y_true==i)&(y_pred==j)).sum())
            if count:
                rows.append({"true_label":name_i,"pred_label":name_j,"count":count})
    return sorted(rows,key=lambda r:r["count"],reverse=True)[:limit]


def per_class_error_rows(pred_rows:list[dict],activity_names:list[str])->list[dict]:
    y_true,y_pred,_,_=prediction_arrays(pred_rows)
    rows=[]
    for i,name in enumerate(activity_names):
        mask=y_true==i
        rows.append({
            "label":name,
            "support":int(mask.sum()),
            "error_rate":float((y_pred[mask]!=i).mean()) if mask.any() else 0.0,
            "recall":float((y_pred[mask]==i).mean()) if mask.any() else 0.0,
        })
    return rows
```

A.8 flopt/eda.py

```
from __future__ import annotations

from collections import Counter

import numpy as np

from .data import UCIBundle


def eda_tables(bundle:UCIBundle)->dict[str,list[dict]]:
    xtr=bundle.x_train_raw
    xte=bundle.x_test_raw
    train_counts=Counter(bundle.y_train.tolist())
    test_counts=Counter(bundle.y_test.tolist())
    class_rows=[]
```

```
    for i,name in enumerate(bundle.activity_names):
        class_rows.append({"class_id":i,"activity":name,"train_count":train_counts[i],"test_count":test_counts[i],"total_count":train_counts[i]+test_counts[i]})
    client_rows=[]
    label_rows=[]
    all_subjects=sorted(set(bundle.subject_train)|set(bundle.subject_test))
    for sid in all_subjects:
        tr=bundle.subject_train==sid
        te=bundle.subject_test==sid
        labels=np.concatenate([bundle.y_train[tr],bundle.y_test[te]])
        counts=np.bincount(labels,minlength=len(bundle.activity_names))
        total=int(counts.sum())
        probs=counts/total if total else counts
        entropy=float(-(probs[probs>0]*np.log2(probs[probs>0])).sum()) if total else 0.0
        client_rows.append({"client_id":int(sid),"train_samples":int(tr.sum()),"test_samples":int(te.sum()),"total_samples":total,"label_entropy":entropy,"dominant_class_ratio":float(probs.max()) if total else 0.0})
        for i,name in enumerate(bundle.activity_names):
            label_rows.append({"client_id":int(sid),"activity":name,"count":int(counts[i]),"proportion":float(probs[i]) if total else 0.0})
    summary=[{
        "train_rows":int(xtr.shape[0]),
        "test_rows":int(xte.shape[0]),
        "features":int(xtr.shape[1]),
        "clients":len(all_subjects),
        "classes":len(bundle.activity_names),
        "train_missing":int(np.isnan(xtr).sum()),
        "test_missing":int(np.isnan(xte).sum()),
        "train_duplicates":int(xtr.shape[0]-np.unique(xtr,axis=0).shape[0]),
        "feature_min":float(xtr.min()),
        "feature_max":float(xtr.max()),
        "feature_mean":float(xtr.mean()),
        "feature_std":float(xtr.std()),
    }]
    return {
        "dataset_summary":summary,
        "class_distribution":class_rows,
        "client_sample_counts":client_rows,
        "client_label_distribution":label_rows,
    }


def noniid_rows(bundle:UCIBundle)->list[dict]:
    rows=[]
    global_counts=np.bincount(np.concatenate([bundle.y_train,bundle.y_test]),minlength=len(bundle.activity_names)).astype(float)
    global_p=(global_counts+1e-9)/(global_counts.sum()+1e-9*len(global_counts))
    labels_by_subject={}
    for sid in sorted(set(bundle.subject_train)|set(bundle.subject_test)):
        labels=np.concatenate([bundle.y_train[bundle.subject_train==sid],bundle.y_test[bundle.subject_test==sid]])
        labels_by_subject[int(sid)]=labels
    for sid,labels in labels_by_subject.items():
        counts=np.bincount(labels,minlength=len(bundle.activity_names)).astype(float)
        p=(counts+1e-9)/(counts.sum()+1e-9*len(counts))
        m=0.5*(p+global_p)
        kl=float((p*np.log(p/global_p)).sum())
        js=float(0.5*(p*np.log(p/m)).sum()+0.5*(global_p*np.log(global_p/m)).sum())
        entropy=float(-(p*np.log2(p)).sum())
        rows.append({"client_id":sid,"kl_divergence":kl,"js_divergence":js,"label_entropy":entropy,"dominant_class_ratio":float(p.max()),"samples":int(counts.sum())})
    return rows
```

A.9 flopt/calibration.py

```python
from __future__ import annotations

import numpy as np

from .metrics import prediction_arrays


def calibration_bins(pred_rows:list[dict],bins:int=10)->tuple[list[dict],dict]:
    y_true,y_pred,conf,_=prediction_arrays(pred_rows)
    correct=(y_true==y_pred).astype(float)
    edges=np.linspace(0,1,bins+1)
    rows=[]
    ece=0.0
    mce=0.0
    for i in range(bins):
        lo,hi=edges[i],edges[i+1]
        mask=(conf>=lo)&(conf<=hi if i==bins-1 else conf<hi)
        if mask.any():
            acc=float(correct[mask].mean())
            avg_conf=float(conf[mask].mean())
            gap=abs(acc-avg_conf)
            ece+=float(mask.mean())*gap
            mce=max(mce,gap)
            count=int(mask.sum())
        else:
            acc=avg_conf=gap=0.0
            count=0
        rows.append({"bin":i,"lower":float(lo),"upper":float(hi),"count":count,"accuracy":acc,"confidence":avg_conf,"gap":gap})
    summary={"ece":float(ece),"mce":float(mce),"mean_confidence":float(conf.mean()),"accuracy":float(correct.mean())}
    return rows,summary
```

A.10 flopt/baselines.py

```python
from __future__ import annotations

from copy import deepcopy

import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader,TensorDataset

from .config import FLConfig
from .data import ClientData
from .fedavg import _device,evaluate_all


def centralized_train(model:nn.Module,clients:list[ClientData],cfg:FLConfig)->tuple[nn.Module,list[dict]]:
    device=_device()
    model=deepcopy(model).to(device)
    x=np.concatenate([c.x_train for c in clients]).astype("float32")
    y=np.concatenate([c.y_train for c in clients]).astype("int64")
    loader=DataLoader(TensorDataset(torch.tensor(x),torch.tensor(y)),batch_size=cfg.batch_size,shuffle=True)
    opt=torch.optim.SGD(model.parameters(),lr=cfg.lr)
    loss_fn=nn.CrossEntropyLoss()
    records=[]
    best=float("inf")
    stale=0
    max_rounds=cfg.max_rounds or cfg.rounds
    for epoch in range(1,max_rounds+1):
```

```
        model.train()
        for xb,yb in loader:
            xb=xb.to(device)
            yb=yb.to(device)
            opt.zero_grad()
            loss=loss_fn(model(xb),yb)
            loss.backward()
            opt.step()
        metrics=evaluate_all(model,clients,device)
        improved=metrics["loss"]<best-cfg.min_delta
        if improved:
            best=metrics["loss"]
            stale=0
        else:
            stale+=1
        metrics.update({"round":epoch,"upload_bytes":0,"download_bytes":0,"selected_clients":[],"best_loss_so_far":best,"rounds_since_improvement":stale,"stopped_early":bool(cfg.early_stopping and stale>=cfg.patience)})
        records.append(metrics)
        if metrics["stopped_early"]:
            break
    return model,records


def local_only_summary(model_factory,clients:list[ClientData],cfg:FLConfig)->tuple[list[dict],list[dict]]:
    device=_device()
    rows=[]
    round_rows=[]
    for idx,client in enumerate(clients):
        model=model_factory().to(device)
        local_cfg=cfg
        train_client_model(model,client,local_cfg,device)
        metrics=evaluate_all(model,[client],device)
        cid=client.client_id if client.client_id is not None else idx
        rows.append({"client_id":int(cid),"loss":metrics["loss"],"accuracy":metrics["accuracy"],"test_samples":len(client.x_test),"train_samples":len(client.x_train)})
        round_rows.append({"run_type":"local_only","seed":cfg.seed,"client_id":int(cid),"loss":metrics["loss"],"accuracy":metrics["accuracy"]})
    return rows,round_rows


def train_client_model(model:nn.Module,client:ClientData,cfg:FLConfig,device:torch.device)->None:
    x=torch.tensor(client.x_train,dtype=torch.float32)
    y=torch.tensor(client.y_train,dtype=torch.long)
    loader=DataLoader(TensorDataset(x,y),batch_size=cfg.batch_size,shuffle=True)
    opt=torch.optim.SGD(model.parameters(),lr=cfg.lr)
    loss_fn=nn.CrossEntropyLoss()
    epochs=cfg.local_epochs*max(1,min(cfg.max_rounds or cfg.rounds,50))
    for _ in range(epochs):
        for xb,yb in loader:
            xb=xb.to(device)
            yb=yb.to(device)
            opt.zero_grad()
            loss=loss_fn(model(xb),yb)
            loss.backward()
            opt.step()
```

A.11 flopt/stats.py

```
from __future__ import annotations

import math
```

```python

import numpy as np
from scipy import stats as scipy_stats


def confidence_rows(rows:list[dict],group_key:str,metrics:list[str])->list[dict]:
    groups={}
    for row in rows:
        groups.setdefault(row[group_key],[]).append(row)
    out=[]
    for group,items in groups.items():
        for metric in metrics:
            vals=np.array([float(r[metric]) for r in items if r.get(metric) not in {None,""}],dtype=float)
            if len(vals)==0:
                continue
            mean=float(vals.mean())
            std=float(vals.std(ddof=1)) if len(vals)>1 else 0.0
            se=std/math.sqrt(len(vals)) if len(vals)>1 else 0.0
            ci=1.96*se
            out.append({"group":group,"metric":metric,"n":len(vals),"mean":mean,"std":std,"ci95_low":mean-ci,"ci95_high":mean+ci})
    return out


def paired_tests(rows:list[dict],method_key:str,seed_key:str,metrics:list[str],baseline:str)->list[dict]:
    methods=sorted({r[method_key] for r in rows})
    out=[]
    for method in methods:
        if method==baseline:
            continue
        for metric in metrics:
            b={r[seed_key]:float(r[metric]) for r in rows if r[method_key]==baseline and r.get(metric) not in {None,""}}
            m={r[seed_key]:float(r[metric]) for r in rows if r[method_key]==method and r.get(metric) not in {None,""}}
            seeds=sorted(set(b)&set(m))
            if len(seeds)<2:
                continue
            diff=np.array([m[s]-b[s] for s in seeds],dtype=float)
            t_p=float(scipy_stats.ttest_rel([m[s] for s in seeds],[b[s] for s in seeds]).pvalue)
            try:
                w_p=float(scipy_stats.wilcoxon(diff).pvalue)
            except ValueError:
                w_p=1.0
            effect=float(diff.mean()/(diff.std(ddof=1)+1e-12)) if len(diff)>1 else 0.0
            out.append({"baseline":baseline,"method":method,"metric":metric,"n":len(seeds),"mean_diff":float(diff.mean()),"paired_t_p":t_p,"wilcoxon_p":w_p,"effect_size":effect})
    return out


def correlation_rows(left:list[dict],right:list[dict],key:str,left_metrics:list[str],right_metrics:list[str])->list[dict]:
    rmap={r[key]:r for r in right}
    out=[]
    for lm in left_metrics:
        for rm in right_metrics:
            xs=[]
            ys=[]
            for row in left:
                if row[key] in rmap and row.get(lm) not in {None,""} and rmap[row[key]].get(rm) not in {None,""}:
                    xs.append(float(row[lm]))
                    ys.append(float(rmap[row[key]][rm]))
            if len(xs)>2:
```

```
                corr,p=scipy_stats.pearsonr(xs,ys)
                out.append({"x_metric":lm,"y_metric":rm,"n":len(xs),"pearson_r":float(corr),"p_value":float(p)})
    return out
```

A.12 flopt/analysis.py

```
from __future__ import annotations

import numpy as np


def communication_efficiency_rows(summary_rows:list[dict],threshold:float=0.90)->list[dict]:
    out=[]
    for row in summary_rows:
        comm=float(row.get("total_comm_until_stop",row.get("total_comm",0)) or 0)
        acc=float(row.get("final_accuracy",row.get("accuracy",0)) or 0)
        worst=float(row.get("final_worst_client_accuracy",row.get("worst_client_accuracy",0)) or 0)
        loss=float(row.get("final_loss",row.get("loss",0)) or 0)
        mb=comm/1_000_000 if comm else 0.0
        out.append({
            **row,
            "comm_mb":mb,
            "accuracy_per_mb":acc/mb if mb else 0.0,
            "worst_accuracy_per_mb":worst/mb if mb else 0.0,
            "loss_per_mb":loss/mb if mb else 0.0,
            "reached_threshold":bool(acc>=threshold),
        })
    return out


def fairness_gap_rows(rows:list[dict])->list[dict]:
    out=[]
    for row in rows:
        acc=float(row.get("final_accuracy",row.get("accuracy",0)) or 0)
        worst=float(row.get("final_worst_client_accuracy",row.get("worst_client_accuracy",0)) or 0)
        out.append({**row,"fairness_gap":acc-worst})
    return out


def ablation_rows(rows:list[dict],baseline_name:str="fedavg_default")->list[dict]:
    baseline=[r for r in rows if r.get("method")==baseline_name]
    base_by_seed={r["seed"]:r for r in baseline}
    out=[]
    for row in rows:
        base=base_by_seed.get(row.get("seed"))
        if not base:
            continue
        out.append({
            **row,
            "delta_accuracy":float(row.get("final_accuracy",row.get("accuracy",0)))-float(base.get("final_accuracy",base.get("accuracy",0))),
            "delta_worst_client_accuracy":float(row.get("final_worst_client_accuracy",row.get("worst_client_accuracy",0)))-float(base.get("final_worst_client_accuracy",base.get("worst_client_accuracy",0))),
            "delta_comm":float(row.get("total_comm_until_stop",row.get("total_comm",0)))-float(base.get("total_comm_until_stop",base.get("total_comm",0))),
        })
    return out


def failure_mode_rows(rows:list[dict])->list[dict]:
    out=[]
```

```
    for row in rows:
        acc=float(row.get("final_accuracy",row.get("accuracy",0)) or 0)
        worst=float(row.get("final_worst_client_accuracy",row.get("worst_client_accuracy",0)) or 0)
        if worst<0.70:
            out.append({**row,"failure_type":"low_worst_client_accuracy","description":"Worst-client accuracy remains below 0.70."})
        if acc<0.90:
            out.append({**row,"failure_type":"low_average_accuracy","description":"Average accuracy remains below 0.90."})
        if row.get("stopped_early") is False and row.get("max_rounds")==row.get("stopped_round"):
            out.append({**row,"failure_type":"no_early_convergence","description":"Run reached max_rounds without early stopping."})
    return out


def selected_case_clients(per_client:list[dict],n:int=5)->list[int]:
    rows=sorted(per_client,key=lambda r:float(r["accuracy"]))
    ids=[int(rows[0]["client_id"]),int(rows[len(rows)//2]["client_id"]),int(rows[-1]["client_id"])]
    return list(dict.fromkeys(ids[:n]))


def summarize_rows(rows:list[dict],group_key:str,metrics:list[str])->list[dict]:
    groups={}
    for row in rows:
        groups.setdefault(row[group_key],[]).append(row)
    out=[]
    for group,items in groups.items():
        r={group_key:group,"n":len(items)}
        for metric in metrics:
            vals=np.array([float(item[metric]) for item in items if item.get(metric) not in {None,""}],dtype=float)
            if len(vals):
                r[f"{metric}_mean"]=float(vals.mean())
                r[f"{metric}_std"]=float(vals.std(ddof=1)) if len(vals)>1 else 0.0
        out.append(r)
    return out
```

A.13 flopt/plots.py

```
from __future__ import annotations

import os
from pathlib import Path

_mpl_cache=Path("outputs")/"mpl-cache"
_mpl_cache.mkdir(parents=True,exist_ok=True)
_xdg_cache=Path("outputs")/"cache"
_xdg_cache.mkdir(parents=True,exist_ok=True)
os.environ.setdefault("MPLCONFIGDIR",str(_mpl_cache))
os.environ.setdefault("XDG_CACHE_HOME",str(_xdg_cache))

import matplotlib.pyplot as plt
plt.switch_backend("Agg")
import numpy as np
from sklearn.decomposition import PCA


def plot_convergence(records:list[dict],path:str)->None:
    rounds=[r["round"] for r in records]
    loss=[r["loss"] for r in records]
    ref=loss[0]/np.sqrt(np.maximum(rounds,1))
    _prep(path)
```

```
    plt.figure(figsize=(7,4))
    plt.plot(rounds,loss,label="FedAvg loss")
    plt.plot(rounds,ref,label="O(1/sqrt(T)) reference",linestyle="--")
    plt.xlabel("Round")
    plt.ylabel("Loss")
    plt.legend()
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()


def plot_cvar(rows:list[dict],path:str)->None:
    _prep(path)
    plt.figure(figsize=(6,4))
    for row in rows:
        plt.scatter(row["accuracy"],row["worst_client_accuracy"])
        plt.text(row["accuracy"],row["worst_client_accuracy"],f"alpha={row['alpha']}")
    plt.xlabel("Average client accuracy")
    plt.ylabel("Worst-client accuracy")
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()


def plot_shadow_price(rows:list[dict],path:str)->None:
    clean=[r for r in rows if r.get("status") in {"optimal","optimal_inaccurate"}]
    _prep(path)
    plt.figure(figsize=(6,4))
    plt.plot([r["budget"] for r in clean],[r["lambda"] for r in clean],marker="o")
    plt.xlabel("Communication budget")
    plt.ylabel("lambda*")
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()


def plot_search_comparison(grid_rows:list[dict],ga_result:dict,path:str)->None:
    best_grid=min(float(r["fitness"]) for r in grid_rows)
    _prep(path)
    plt.figure(figsize=(5,4))
    plt.bar(["Best grid","GA"],[best_grid,float(ga_result["fitness"])])
    plt.ylabel("Fitness")
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()


def _prep(path:str)->None:
    Path(path).parent.mkdir(parents=True,exist_ok=True)


def bar(rows:list[dict],x:str,y:str,path:str,title:str="",rotation:int=45)->None:
    _prep(path)
    plt.figure(figsize=(8,4))
    plt.bar([str(r[x]) for r in rows],[float(r[y]) for r in rows])
    plt.title(title)
    plt.xticks(rotation=rotation,ha="right")
    plt.ylabel(y)
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()


def grouped_bar(rows:list[dict],x:str,ys:list[str],path:str,title:str="")->None:
    _prep(path)
    labels=[str(r[x]) for r in rows]
    pos=np.arange(len(labels))
    width=0.8/max(1,len(ys))
```

```
    plt.figure(figsize=(9,4))
    for i,y in enumerate(ys):
        plt.bar(pos+i*width,[float(r.get(y,0) or 0) for r in rows],width,label=y)
    plt.xticks(pos+width*(len(ys)-1)/2,labels,rotation=45,ha="right")
    plt.title(title)
    plt.legend()
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()


def line_mean_std(round_rows:list[dict],metric:str,path:str,title:str="")->None:
    _prep(path)
    rounds=sorted({int(r["round"]) for r in round_rows})
    mean=[]
    std=[]
    for rnd in rounds:
        vals=[float(r[metric]) for r in round_rows if int(r["round"])==rnd and r.get(metric) not in {None,""}]
        mean.append(np.mean(vals))
        std.append(np.std(vals))
    mean=np.array(mean)
    std=np.array(std)
    plt.figure(figsize=(8,4))
    plt.plot(rounds,mean,label=metric)
    plt.fill_between(rounds,mean-std,mean+std,alpha=0.2,label="std")
    plt.title(title)
    plt.xlabel("round")
    plt.ylabel(metric)
    plt.legend()
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()


def heatmap(matrix:np.ndarray,xlabels:list[str],ylabels:list[str],path:str,title:str="",xlabel:str="",ylabel:str="")->None:
    _prep(path)
    plt.figure(figsize=(9,6))
    plt.imshow(matrix,aspect="auto",cmap="viridis")
    plt.colorbar()
    plt.xticks(range(len(xlabels)),xlabels,rotation=45,ha="right")
    step=max(1,len(ylabels)//20)
    plt.yticks(range(0,len(ylabels),step),[ylabels[i] for i in range(0,len(ylabels),step)])
    plt.title(title)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()


def scatter(rows:list[dict],x:str,y:str,path:str,title:str="",color:str|None=None)->None:
    _prep(path)
    plt.figure(figsize=(6,4))
    c=[float(r[color]) for r in rows] if color else None
    sc=plt.scatter([float(r[x]) for r in rows],[float(r[y]) for r in rows],c=c,cmap="viridis" if color else None)
    if color:
        plt.colorbar(sc,label=color)
    plt.title(title)
    plt.xlabel(x)
    plt.ylabel(y)
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()
```

```
def scatter3(rows:list[dict],x:str,y:str,z:str,path:str,title:str="",color:str|None=None)->None:
    _prep(path)
    fig=plt.figure(figsize=(7,5))
    ax=fig.add_subplot(111,projection="3d")
    c=[float(r[color]) for r in rows] if color else None
    sc=ax.scatter([float(r[x]) for r in rows],[float(r[y]) for r in rows],[float(r[z]) for r in rows],c=c,cmap="viridis" if color else None)
    if color:
        fig.colorbar(sc,ax=ax,label=color,shrink=0.7)
    ax.set_title(title)
    ax.set_xlabel(x)
    ax.set_ylabel(y)
    ax.set_zlabel(z)
    plt.tight_layout()
    plt.savefig(path,dpi=160)
    plt.close()


def pca_plots(x:np.ndarray,labels:list,activity_names:list[str],path2d:str,path3d:str,title_prefix:str)->None:
    pca=PCA(n_components=3,random_state=0)
    pts=pca.fit_transform(x)
    _prep(path2d)
    plt.figure(figsize=(7,5))
    uniq=sorted(set(labels))
    for lab in uniq:
        mask=np.array(labels)==lab
        name=activity_names[int(lab)] if isinstance(lab,(int,np.integer)) and int(lab)<len(activity_names) else str(lab)
        plt.scatter(pts[mask,0],pts[mask,1],s=8,label=name,alpha=0.7)
    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.title(f"{title_prefix} 2D PCA")
    if len(uniq)<=12:
        plt.legend(fontsize=7)
    plt.tight_layout()
    plt.savefig(path2d,dpi=160)
    plt.close()
    fig=plt.figure(figsize=(8,6))
    ax=fig.add_subplot(111,projection="3d")
    for lab in uniq:
        mask=np.array(labels)==lab
        name=activity_names[int(lab)] if isinstance(lab,(int,np.integer)) and int(lab)<len(activity_names) else str(lab)
        ax.scatter(pts[mask,0],pts[mask,1],pts[mask,2],s=8,label=name,alpha=0.7)
    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.set_zlabel("PC3")
    ax.set_title(f"{title_prefix} 3D PCA")
    if len(uniq)<=12:
        ax.legend(fontsize=7)
    plt.tight_layout()
    plt.savefig(path3d,dpi=160)
    plt.close()
```

A.14 experiments/run_full_v2.py

```
from __future__ import annotations

import argparse
import json
import pickle
import platform
import sys
```

```
from dataclasses import replace
from pathlib import Path

import numpy as np
import torch

sys.path.insert(0,str(Path(__file__).resolve().parents[1]))

from flopt.analysis import ablation_rows,communication_efficiency_rows,fairness_gap_rows,
    failure_mode_rows,selected_case_clients,summarize_rows
from flopt.baselines import centralized_train,local_only_summary
from flopt.calibration import calibration_bins
from flopt.config import FLConfig
from flopt.data import load_uci_har
from flopt.duality import solve_policy_lp
from flopt.eda import eda_tables,noniid_rows
from flopt.fedavg import federated_train,predict_clients
from flopt.io import convergence_summary,ensure_dirs,flatten_round_records,write_csv,
    write_json
from flopt.metrics import aggregate_scores,classification_rows,confusion_rows,
    per_class_error_rows,per_client_rows,top_confusions
from flopt.models import HARMLP,count_parameters
from flopt.plots import bar,grouped_bar,heatmap,line_mean_std,pca_plots,scatter,scatter3
from flopt.profiling import timed
from flopt.search import ga_search,grid_search
from flopt.stats import confidence_rows,correlation_rows,paired_tests


ALPHAS=[0,0.5,0.75,0.9,0.95]
SEEDS=[7,11,19,23,29,31,37,41,43,47]
GRID=[(1,5,0.01),(1,10,0.03),(2,10,0.03),(4,10,0.02),(2,15,0.01),(4,15,0.02),(5,10,0.02)
    ,(3,12,0.04)]


def main()->None:
    parser=argparse.ArgumentParser()
    parser.add_argument("--out",default="outputs/full_uci_v2")
    parser.add_argument("--max-rounds",type=int,default=1000)
    parser.add_argument("--search-rounds",type=int,default=120)
    parser.add_argument("--patience",type=int,default=30)
    parser.add_argument("--min-delta",type=float,default=0.001)
    parser.add_argument("--seeds",default=",".join(map(str,SEEDS)))
    parser.add_argument("--ga-maxiter",type=int,default=4)
    parser.add_argument("--ga-popsize",type=int,default=6)
    parser.add_argument("--threads",type=int,default=12)
    args=parser.parse_args()
    torch.set_num_threads(args.threads)
    out=Path(args.out)
    ensure_dirs(out)
    runtime=[]
    seeds=[int(s) for s in args.seeds.split(",") if s.strip()]

    with timed("load_data",runtime):
        bundle=load_uci_har(Path("data"),seed=seeds[0])
        clients=bundle.clients
    activity_names=bundle.activity_names
    base=FLConfig(rounds=args.max_rounds,max_rounds=args.max_rounds,local_epochs=2,
        clients_per_round=min(10,len(clients)),lr=0.03,batch_size=32,early_stopping=True,
        patience=args.patience,min_delta=args.min_delta)
    search_base=replace(base,max_rounds=args.search_rounds,rounds=args.search_rounds,
        patience=max(10,args.patience//2))

    write_json(out/"run_metadata.json",{
        "max_rounds":args.max_rounds,
        "search_rounds":args.search_rounds,
        "patience":args.patience,
        "min_delta":args.min_delta,
        "seeds":seeds,
```

```
        "threads":args.threads,
        "platform":platform.platform(),
        "torch_version":torch.__version__,
        "model_parameters":count_parameters(HARMLP()),
    })
    with (out/"artifacts"/"scaler.pkl").open("wb") as f:
        pickle.dump(bundle.scaler,f)

    with timed("eda",runtime):
        run_eda(bundle,out)

    all_rounds=[]
    conv=[]
    method_seed_rows=[]
    final_models={}

    with timed("fedavg_default",runtime):
        rows,model=run_seeded_method("fedavg_default",clients,base,seeds,out,all_rounds,conv)
        method_seed_rows.extend(rows)
        final_models["fedavg_default"]=model

    cvar_summaries=[]
    cvar_models={}
    with timed("cvar_sweep",runtime):
        for alpha in ALPHAS:
            cfg=replace(base,cvar_alpha=alpha)
            rows,model=run_seeded_method(f"cvar_{alpha}",clients,cfg,seeds,out,all_rounds,conv,alpha=alpha)
            method_seed_rows.extend(rows)
            cvar_summaries.extend(rows)
            cvar_models[alpha]=model
    best_alpha=max(summarize_rows(cvar_summaries,"method",["final_accuracy","final_worst_client_accuracy"]),key=lambda r:r.get("final_worst_client_accuracy_mean",0))["method"]
    best_alpha_value=float(str(best_alpha).replace("cvar_",""))
    final_models["cvar_best"]=cvar_models[best_alpha_value]

    with timed("baselines",runtime):
        cent_rows,cent_model=run_seeded_centralized(clients,base,seeds,out,all_rounds,conv)
        method_seed_rows.extend(cent_rows)
        final_models["centralized"]=cent_model
        local_rows,local_detail=local_only_summary(HARMLP,clients,replace(base,seed=seeds[0],max_rounds=50,rounds=50))
        write_csv(out/"baselines"/"local_only_clients.csv",local_rows)
        write_csv(out/"raw"/"local_only_detail.csv",local_detail)
        method_seed_rows.append({"method":"local_only","seed":seeds[0],"final_loss":float(np.mean([r["loss"] for r in local_rows])),"final_accuracy":float(np.mean([r["accuracy"] for r in local_rows])),"final_worst_client_accuracy":float(np.min([r["accuracy"] for r in local_rows])),"total_comm_until_stop":0,"stopped_round":50,"stopped_early":False})

    with timed("search",runtime):
        grid_rows=grid_search(clients,replace(search_base,seed=seeds[0]),GRID)
        write_csv(out/"search"/"grid_search.csv",grid_rows)
        best_grid=grid_rows[0]
        ga=ga_search(clients,replace(search_base,seed=seeds[0]),maxiter=args.ga_maxiter,popsize=args.ga_popsize)
        write_json(out/"search"/"ga_result.json",ga)
        write_csv(out/"search"/"ga_history.csv",ga["history"])
        grid_cfg=replace(base,local_epochs=int(best_grid["local_epochs"]),clients_per_round=int(best_grid["clients_per_round"]),lr=float(best_grid["lr"]))
        ga_cfg=replace(base,local_epochs=max(1,int(round(ga["x"][0]))),clients_per_round=max(1,int(round(ga["x"][1]))),lr=float(ga["x"][2]))
        grid_valid,grid_model=run_seeded_method("grid_best_validated",clients,grid_cfg,seeds,out,all_rounds,conv)
        ga_valid,ga_model=run_seeded_method("ga_best_validated",clients,ga_cfg,seeds,out,all_rounds,conv)
```

```
        method_seed_rows.extend(grid_valid)
        method_seed_rows.extend(ga_valid)
        final_models["grid_best_validated"]=grid_model
        final_models["ga_best_validated"]=ga_model

    write_csv(out/"raw"/"all_round_metrics.csv",all_rounds)
    write_csv(out/"raw"/"convergence_summary.csv",conv)
    write_csv(out/"metrics"/"method_seed_results.csv",method_seed_rows)
    method_summary=summarize_rows(method_seed_rows,"method",["final_loss","final_accuracy","final_worst_client_accuracy","total_comm_until_stop","stopped_round"])
    write_csv(out/"metrics"/"method_summary.csv",method_summary)

    with timed("predictions_metrics",runtime):
        best_model=final_models["ga_best_validated"]
        pred_rows=predict_clients(best_model,clients)
        write_csv(out/"metrics"/"predictions.csv",pred_rows)
        write_csv(out/"metrics"/"classification_report.csv",classification_rows(pred_rows,activity_names))
        write_csv(out/"metrics"/"confusion_matrix.csv",confusion_rows(pred_rows,activity_names))
        write_csv(out/"metrics"/"confusion_matrix_normalized.csv",confusion_rows(pred_rows,activity_names,normalize=True))
        per_client=per_client_rows(pred_rows,clients)
        write_csv(out/"metrics"/"per_client_metrics.csv",per_client)
        write_csv(out/"errors"/"top_confusions.csv",top_confusions(pred_rows,activity_names))
        write_csv(out/"errors"/"per_class_error_rates.csv",per_class_error_rows(pred_rows,activity_names))
        cal_rows,cal_summary=calibration_bins(pred_rows)
        write_csv(out/"calibration"/"calibration_bins.csv",cal_rows)
        write_csv(out/"calibration"/"calibration_summary.csv",[cal_summary])
        write_json(out/"metrics"/"aggregate_scores.json",aggregate_scores(pred_rows))

    with timed("lp_diagnostics",runtime):
        lp_source=grid_rows+[{"loss":r["final_loss"],"comm":r["total_comm_until_stop"]} for r in method_seed_rows if r["method"] in {"fedavg_default","grid_best_validated","ga_best_validated"}]
        losses=[float(r["loss"]) for r in lp_source]
        costs=[float(r["comm"]) for r in lp_source]
        budgets=np.linspace(min(costs),max(costs),12).tolist()
        lp_rows=solve_policy_lp(losses,costs,budgets)
        write_json(out/"lp"/"lp_shadow_price.json",lp_rows)
        write_csv(out/"lp"/"lp_shadow_price.csv",flatten_lp(lp_rows))

    with timed("diagnostics",runtime):
        run_diagnostics(out,method_seed_rows,per_client,bundle,pred_rows,conv)

    with timed("plots",runtime):
        run_plots(out,bundle,all_rounds,method_summary,method_seed_rows,per_client,grid_rows,ga,lp_rows,pred_rows,activity_names)

    with timed("artifacts",runtime):
        for name,model in final_models.items():
            torch.save(model.state_dict(),out/"artifacts"/f"{name}.pt")

    write_csv(out/"runtime"/"runtime_by_stage.csv",runtime)
    write_report(out,method_summary,best_alpha_value,ga)
    print(f"wrote v2 artifacts to {out}")


def run_seeded_method(name,clients,cfg,seeds,out,all_rounds,conv,alpha=None):
    rows=[]
    last_model=None
    for seed in seeds:
        run_cfg=replace(cfg,seed=seed)
        model,records=federated_train(HARMLP(),clients,run_cfg,track_drift=name in {"fedavg_default",f"cvar_{alpha}"})
        last_model=model
```

```
        round_rows=flatten_round_records(records,name,seed,alpha)
        all_rounds.extend(round_rows)
        write_csv(out/"raw"/f"{name}_seed_{seed}_rounds.csv",round_rows)
        c=convergence_summary(records,name,seed,alpha,run_cfg.max_rounds or run_cfg.rounds)
        conv.append(c)
        rows.append({"method":name,"seed":seed,**{k:v for k,v in c.items() if k not in {"run_type","seed","alpha","max_rounds"}}})
        write_drift(out,name,seed,records)
    return rows,last_model


def run_seeded_centralized(clients,cfg,seeds,out,all_rounds,conv):
    rows=[]
    last_model=None
    for seed in seeds:
        run_cfg=replace(cfg,seed=seed)
        model,records=centralized_train(HARMLP(),clients,run_cfg)
        last_model=model
        rr=flatten_round_records(records,"centralized",seed)
        all_rounds.extend(rr)
        write_csv(out/"raw"/f"centralized_seed_{seed}_rounds.csv",rr)
        c=convergence_summary(records,"centralized",seed,None,run_cfg.max_rounds or run_cfg.rounds)
        conv.append(c)
        rows.append({"method":"centralized","seed":seed,**{k:v for k,v in c.items() if k not in {"run_type","seed","alpha","max_rounds"}}})
    return rows,last_model


def run_eda(bundle,out):
    tables=eda_tables(bundle)
    for name,rows in tables.items():
        write_csv(out/"eda"/f"{name}.csv",rows)
    niid=noniid_rows(bundle)
    write_csv(out/"noniid"/"client_distribution_metrics.csv",niid)
    class_rows=tables["class_distribution"]
    bar(class_rows,"activity","total_count",str(out/"plots"/"eda"/"class_distribution.png"),"Class Distribution")
    grouped_bar(class_rows,"activity",["train_count","test_count"],str(out/"plots"/"eda"/"train_test_class_distribution.png"),"Train/Test Class Distribution")
    bar(tables["client_sample_counts"],"client_id","total_samples",str(out/"plots"/"eda"/"client_sample_counts.png"),"Samples Per Client")
    bar(tables["client_sample_counts"],"client_id","label_entropy",str(out/"plots"/"eda"/"client_entropy.png"),"Client Label Entropy")
    matrix=np.array([[r["proportion"] for r in tables["client_label_distribution"] if r["client_id"]==cid] for cid in sorted({r["client_id"] for r in tables["client_label_distribution"]})])
    heatmap(matrix,bundle.activity_names,[str(cid) for cid in sorted({r["client_id"] for r in tables["client_label_distribution"]})],str(out/"plots"/"eda"/"client_label_heatmap.png"),"Client Label Distribution","activity","client")
    corr=np.corrcoef(bundle.x_train_raw[:,:30],rowvar=False)
    heatmap(corr,[str(i) for i in range(30)],[str(i) for i in range(30)],str(out/"plots"/"eda"/"feature_correlation_subset.png"),"Feature Correlation Subset")
    x=np.vstack([bundle.x_train_raw,bundle.x_test_raw])
    y=np.concatenate([bundle.y_train,bundle.y_test])
    s=np.concatenate([bundle.subject_train,bundle.subject_test])
    idx=np.linspace(0,len(x)-1,min(len(x),2500),dtype=int)
    pca_plots(x[idx],y[idx].tolist(),bundle.activity_names,str(out/"plots"/"eda"/"pca_by_activity.png"),str(out/"plots"/"eda"/"pca_3d_by_activity.png"),"Activity")
    pca_plots(x[idx],s[idx].tolist(),[],str(out/"plots"/"eda"/"pca_by_client.png"),str(out/"plots"/"eda"/"pca_3d_by_client.png"),"Client")


def write_drift(out,name,seed,records):
    rows=[]
    for r in records:
        ids=r.get("drift_client_ids",[])
        norms=r.get("drift_update_norms",[])
```

```
        cos=r.get("drift_cosine_to_mean",[])
        dist=r.get("drift_distance_to_mean",[])
        for i,cid in enumerate(ids):
            rows.append({"method":name,"seed":seed,"round":r["round"],"client_id":cid,"update_norm":norms[i],"cosine_to_mean":cos[i],"distance_to_mean":dist[i]})
    if rows:
        write_csv(out/"drift"/f"{name}_seed_{seed}_drift.csv",rows)


def flatten_lp(rows):
    out=[]
    for r in rows:
        kkt=r.get("kkt",{})
        out.append({"budget":r.get("budget"),"loss":r.get("loss"),"cost":r.get("cost"),"lambda":r.get("lambda"),"status":r.get("status"),**kkt})
    return out


def run_diagnostics(out,method_seed_rows,per_client,bundle,pred_rows,conv):
    metrics=["final_accuracy","final_worst_client_accuracy","final_loss","total_comm_until_stop","stopped_round"]
    write_csv(out/"stats"/"confidence_intervals.csv",confidence_rows(method_seed_rows,"method",metrics))
    write_csv(out/"stats"/"paired_tests.csv",paired_tests(method_seed_rows,"method","seed",metrics,"fedavg_default"))
    write_csv(out/"ablations"/"ablation_deltas.csv",ablation_rows(method_seed_rows))
    write_csv(out/"efficiency"/"communication_efficiency.csv",communication_efficiency_rows(method_seed_rows))
    write_csv(out/"failure_modes"/"failure_modes.csv",failure_mode_rows(method_seed_rows))
    noniid=noniid_rows(bundle)
    write_csv(out/"noniid"/"noniid_performance_correlations.csv",correlation_rows(noniid,per_client,"client_id",["js_divergence","label_entropy","samples"],["accuracy","error_rate"]))
    case_ids=selected_case_clients(per_client)
    write_csv(out/"case_studies"/"selected_clients.csv",[{"client_id":cid} for cid in case_ids])
    theoretical=[
        {"component":"FedAvg","optimization_concept":"local SGD and stochastic optimization"},
        {"component":"LP shadow price","optimization_concept":"duality and constrained optimization"},
        {"component":"KKT residuals","optimization_concept":"optimality diagnostics"},
        {"component":"CVaR weighting","optimization_concept":"robust and fairness-aware optimization"},
        {"component":"Differential evolution","optimization_concept":"global search"},
        {"component":"Early stopping","optimization_concept":"empirical convergence criterion"},
    ]
    write_csv(out/"reports"/"theoretical_alignment_table.csv",theoretical)


def run_plots(out,bundle,all_rounds,method_summary,method_seed_rows,per_client,grid_rows,ga,lp_rows,pred_rows,activity_names):
    fed=[r for r in all_rounds if r["run_type"]=="fedavg_default"]
    if fed:
        line_mean_std(fed,"loss",str(out/"plots"/"training"/"loss_vs_rounds_mean_std.png"),"FedAvg Loss")
        line_mean_std(fed,"accuracy",str(out/"plots"/"training"/"accuracy_vs_rounds_mean_std.png"),"FedAvg Accuracy")
        line_mean_std(fed,"worst_client_accuracy",str(out/"plots"/"training"/"worst_client_accuracy_vs_rounds.png"),"Worst-Client Accuracy")
        line_mean_std(fed,"best_loss_so_far",str(out/"plots"/"training"/"best_loss_so_far_vs_rounds.png"),"Best Loss So Far")
        line_mean_std(fed,"rounds_since_improvement",str(out/"plots"/"training"/"rounds_since_improvement.png"),"Rounds Since Improvement")
        scatter(fed,"round","total_comm_bytes",str(out/"plots"/"training"/"communication_vs_rounds.png"),"Communication Per Round")
```

```
        scatter(fed,"total_comm_bytes","accuracy",str(out/"plots"/"training"/"accuracy_vs_cumulative_communication.png"),"Accuracy vs Communication")
    bar(method_summary,"method","final_accuracy_mean",str(out/"plots"/"baselines"/"baseline_accuracy_comparison.png"),"Baseline Accuracy")
    bar(method_summary,"method","final_worst_client_accuracy_mean",str(out/"plots"/"baselines"/"baseline_worst_client_accuracy.png"),"Worst-Client Accuracy")
    bar(method_summary,"method","total_comm_until_stop_mean",str(out/"plots"/"baselines"/"baseline_communication_cost.png"),"Communication Cost")
    bar(method_summary,"method","stopped_round_mean",str(out/"plots"/"baselines"/"baseline_rounds_to_convergence.png"),"Rounds To Convergence")
    bar(per_client,"client_id","accuracy",str(out/"plots"/"fairness"/"per_client_accuracy_fedavg.png"),"Per-Client Accuracy")
    scatter(per_client,"train_samples","accuracy",str(out/"plots"/"noniid"/"sample_count_vs_accuracy.png"),"Sample Count vs Accuracy")
    cm=np.array([[r[name] for name in activity_names] for r in confusion_rows(pred_rows,activity_names)])
    heatmap(cm,activity_names,activity_names,str(out/"plots"/"classification"/"confusion_matrix_raw.png"),"Confusion Matrix","predicted","true")
    cmn=np.array([[r[name] for name in activity_names] for r in confusion_rows(pred_rows,activity_names,normalize=True)])
    heatmap(cmn,activity_names,activity_names,str(out/"plots"/"classification"/"confusion_matrix_normalized.png"),"Normalized Confusion Matrix","predicted","true")
    bar(per_class_error_rows(pred_rows,activity_names),"label","error_rate",str(out/"plots"/"errors"/"per_class_error_rate.png"),"Per-Class Error Rate")
    bar(top_confusions(pred_rows,activity_names),"true_label","count",str(out/"plots"/"errors"/"top_confused_pairs.png"),"Top Confusions")
    plot_lp(out,lp_rows)
    scatter(grid_rows,"comm","accuracy",str(out/"plots"/"search"/"accuracy_vs_communication_scatter.png"),"Accuracy vs Communication","fitness")
    scatter3(grid_rows,"local_epochs","clients_per_round","lr",str(out/"plots"/"search"/"hyperparameter_3d_fitness.png"),"3D Hyperparameter Fitness","fitness")
    write_csv(out/"search"/"ga_history_best_so_far.csv",ga_best_history(ga["history"]))
    if ga["history"]:
        line_mean_std(ga_best_history(ga["history"]),"best_fitness",str(out/"plots"/"search"/"ga_fitness_vs_evaluations.png"),"GA Best Fitness")
    eff=communication_efficiency_rows(method_seed_rows)
    bar(summarize_rows(eff,"method",["accuracy_per_mb","worst_accuracy_per_mb"]),"method","accuracy_per_mb_mean",str(out/"plots"/"efficiency"/"accuracy_per_mb.png"),"Accuracy per MB")
    scatter3(method_seed_rows,"final_accuracy","final_worst_client_accuracy","total_comm_until_stop",str(out/"plots"/"optimization"/"accuracy_fairness_communication_3d.png"),"Accuracy Fairness Communication")


def plot_lp(out,lp_rows):
    flat=flatten_lp(lp_rows)
    scatter(flat,"budget","lambda",str(out/"plots"/"optimization"/"shadow_price_vs_budget.png"),"Shadow Price vs Budget")
    scatter(flat,"budget","loss",str(out/"plots"/"optimization"/"loss_vs_budget.png"),"Loss vs Budget")
    scatter(flat,"budget","stationarity_residual",str(out/"plots"/"optimization"/"kkt_residuals.png"),"KKT Residual")
    scatter3(flat,"budget","loss","lambda",str(out/"plots"/"optimization"/"lp_budget_loss_lambda_3d.png"),"LP Budget Loss Lambda")


def ga_best_history(history):
    best=float("inf")
    rows=[]
    for row in history:
        best=min(best,float(row["fitness"]))
        rows.append({"round":row["evaluation"],"best_fitness":best})
    return rows


def write_report(out,method_summary,best_alpha,ga):
    best=max(method_summary,key=lambda r:r.get("final_accuracy_mean",0))
    text=f"""# Federated Learning Optimization V2 Report Draft
```

```

## Main Upgrade

This run uses early-stopped federated learning with organized diagnostics, EDA, baselines, search validation, LP shadow pricing, calibration, and extensive visualization.

## Headline Results

- Best mean accuracy method: `{best['method']}` with accuracy `{best.get('final_accuracy_mean',0):.4f}`.
- Best CVaR alpha selected from sweep: `{best_alpha}`.
- GA best raw vector: `{ga['x']}`.
- GA search evaluations: `{ga['evaluations']}`.

See `reports/overleaf_report_v2.tex` for the final formatted report after report generation.
"""
    (out/"reports"/"report_draft_v2.md").write_text(text,encoding="utf-8")


if __name__=="__main__":
    main()
```

Appendix B

Final Conclusion

The final conclusion is that federated learning can train a strong UCI HAR classifier without centralizing raw client data. Default FedAvg is already strong, grid tuning improves accuracy slightly, and differential evolution finds a more communication-efficient region of the hyperparameter space.

The major unresolved issue is fairness. Worst-client accuracy is lower than average accuracy, and simple CVaR-style weighting did not solve this. This is the correct research conclusion: the project succeeds at building, evaluating, and optimizing a federated system, while identifying client fairness as the next serious technical challenge.

The optimization value of the project comes from the complete chain: stochastic neural-network training, federated aggregation, early stopping, communication accounting, LP duality, shadow-price interpretation, hyperparameter search, statistical validation, and detailed diagnostics. The project is therefore not just a classifier; it is a communication-aware and fairness-aware federated optimization study.